



การพัฒนาแอปพลิเคชันเพื่อการจับคู่เพื่อนร่วมเดินทางด้วยการวิเคราะห์ความคล้ายคลึง โคไซน์

Development of an Application for Travel Companion Matching Using Cosine Similarity

จัดทำโดย

ธนวิชญ์ เสือจำศีล

วรกันต์ พงศ์ลัด

โครงการวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต

สาขาวิชาวิทยาศาสตร์และเทคโนโลยี ภาควิชาวิทยาศาสตร์และเทคโนโลยี

คณะวิทยาศาสตร์และเทคโนโลยี วิทยาเขตห้วยทราย

มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

ปีการศึกษา 2568

หัวข้อโครงการวิจัย

การพัฒนาแอปพลิเคชันเพื่อการจับคู่เพื่อนร่วม
เดินทางด้วยการวิเคราะห์ความคล้ายคลึง โคไซน์

ชื่อผู้เขียน

ธนวิชญ์ เสือจำศีล

วรกันต์ พงศ์ลัด

อาจารย์ที่ปรึกษา

ดร.วิเชพ ใจบุญ

อ.ทักษิณา คงสมลาภ

อ.สุนทรี แก่นแก้ว

ระดับการศึกษา

ปริญญาตรี

สาขาวิชา

วิทยาศาสตร์และเทคโนโลยี

ปีการศึกษา

2568

บทคัดย่อ

ปัจจุบันพฤติกรรมการท่องเที่ยวของคนรุ่นใหม่มีการเปลี่ยนแปลงอย่างเห็นได้ชัด โดยพบว่านักท่องเที่ยวหันมานิยมการเดินทางคนเดียว (Solo Travel) มากขึ้นเพื่ออิสระในการตัดสินใจ อย่างไรก็ตาม นักท่องเที่ยวกลุ่มนี้ยังคงเผชิญปัญหาหลัก 3 ประการ คือ 1) ด้านความปลอดภัย การเดินทางคนเดียวมีความเสี่ยงสูงและขาดระบบคัดกรองเพื่อนร่วมทางที่มีประสิทธิภาพ 2) ความยากในการหาเพื่อนร่วมทางที่เข้ากันได้ การหาเพื่อนที่มีสไตล์การท่องเที่ยว (User DNA) ตรงกัน ทั้งในด้านงบประมาณ กิจกรรม และวงจรเวลาเป็นเรื่องยาก และ 3) ความซับซ้อนในการวางแผน การจัดตารางการเดินทาง (Itinerary) ต้องใช้เวลาในการสืบค้นข้อมูลจำนวนมาก

โครงการนี้พัฒนา Go With Us เป็นแอปพลิเคชัน Native iOS สำหรับค้นหาและเข้าร่วมทริป พร้อมระบบ Backoffice ผ่านเว็บสำหรับผู้ดูแลระบบ ระบบ Smart Matching เปรียบเทียบข้อมูลผู้ใช้กับทริป 4 ด้าน ได้แก่ ความสนใจ ช่วงเวลาที่สะดวก งบประมาณ และจำนวนสถานที่ต่อวัน โดยใช้ Cosine Similarity เฉพาะข้อมูลหลายตัวเลือกด้านความสนใจและช่วงเวลา ส่วนงบประมาณและจำนวนสถานที่ต่อวันใช้สูตรคะแนนตามลักษณะข้อมูล ก่อนนำคะแนนย่อยที่คำนวณได้มาหาค่าเฉลี่ย นอกจากนี้ ระบบใช้ Gemini API เพื่อช่วยวางแผนการเดินทาง และใช้ OTP ยืนยันอีเมลระหว่างสมัครสมาชิก รวมทั้งการตรวจการเคลื่อนไหวของใบหน้าผ่านกล้องในขั้นตอนโปรไฟล์ โดยไม่ถือเป็นการเปรียบเทียบอัตลักษณ์ของบุคคล

สารบัญ

บทที่ 1 บทนำ.....	6
1.1 ความเป็นมาและความสำคัญของปัญหา.....	6
1.2 วัตถุประสงค์ของโครงการ.....	7
1.3 ขอบเขตของโครงการ (Scope).....	7
1.4 ตารางดำเนินการ.....	9
1.5 คำนิยามศัพท์เฉพาะ.....	10
1.6 ผลที่คาดว่าจะได้รับ.....	10
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง.....	12
2.1 วิวัฒนาการและพฤติกรรมการณ์ค้นหาผู้ร่วมเดินทาง.....	12
2.2 พฤติกรรมและแรงจูงใจในการค้นหาเพื่อนร่วมเดินทาง.....	13
2.3 ทฤษฎีการวิเคราะห์ความคล้ายคลึงและปัญญาประดิษฐ์.....	14
2.4 เทคโนโลยีปัญญาประดิษฐ์เชิงสร้างสรรค์.....	15
2.5 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาระบบ.....	15
2.6 ระบบความปลอดภัยและการพิสูจน์ตัวตนด้วยเทคโนโลยีชีวมาตร (Biometrics and Security).....	19
2.7 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง (Related Research and Studies).....	20

สารบัญ (ต่อ)

บทที่ 3 วิธีการดำเนินการ.....	22
3.1 การศึกษาปัญหาและกำหนดกรอบแนวคิดของระบบ.....	22
3.2 ขั้นตอนการดำเนินงานของระบบ.....	30
3.3 ขั้นตอนการดำเนินงานของระบบ.....	32
3.4 การวิเคราะห์และออกแบบระบบส่วนติดต่อผู้ใช้งาน.....	33
3.5 การออกแบบกระบวนการวิเคราะห์และแนะนำทริป.....	35
บทที่ 4 ผลการดำเนินงานและผลการทดสอบระบบ.....	39
4.1 ผลการพัฒนาระบบ.....	39
4.2 ผลการทำงานของระบบ Smart Matching.....	69
4.3 ผลการทดสอบระบบ.....	82
4.4 ตัวอย่างการตรวจสอบสูตร Smart Matching.....	102
4.5 ผลการประเมินความง่ายในการใช้งาน.....	105
4.6 สรุปผลการทดสอบ.....	106
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ.....	107
5.1 สรุปผลการดำเนินโครงการ.....	107
5.2 อภิปรายผล.....	107
5.3 ปัญหาและข้อจำกัดของระบบ.....	108
5.4 ข้อเสนอแนะและแนวทางการพัฒนาต่อ.....	109
อ้างอิง.....	110

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ปัจจุบันอุตสาหกรรมการท่องเที่ยวทั่วโลกมีการเติบโตอย่างต่อเนื่อง และพฤติกรรมของผู้บริโภคได้เปลี่ยนเข้าสู่ยุคดิจิทัลอย่างเต็มรูปแบบ โดยเฉพาะกลุ่มนักท่องเที่ยวที่นิยมเดินทางคนเดียว (Solo Traveler) ที่ต้องการอิสระในการวางแผน แต่ในขณะเดียวกันยังคงมีความต้องการพื้นฐานด้านสังคมและการเชื่อมต่อกับผู้อื่น ส่งผลให้แนวคิด เศรษฐกิจแบบแบ่งปัน (Sharing Economy) กลายเป็นทางเลือกสำคัญที่ทำให้นักท่องเที่ยวที่ไม่รู้จักกันสามารถรวมกลุ่มเพื่อเดินทางร่วมกันและประหยัดค่าใช้จ่ายได้

อย่างไรก็ตาม ปัญหาสำคัญที่พบในปัจจุบันคือ "ความไม่แน่นอนทางความสัมพันธ์" และความเสี่ยงจากการเผชิญกับผู้ใช้ไม่ประสงค์ดี การหาเพื่อนเที่ยวผ่านสื่อสังคมออนไลน์ทั่วไป เช่น Facebook หรือ Instagram ยังมีข้อจำกัดด้านขอบเขตการรับรู้ข้อมูลที่ไม่ได้เจาะจงกลุ่มเป้าหมายนักท่องเที่ยวโดยตรง และที่สำคัญคือขาดระบบคัดกรองที่มีประสิทธิภาพ ทำให้ผู้ใช้งานเสี่ยงต่อการถูกหลอกลวง (Catfishing) ที่อาจนำไปสู่การเสียชีวิตหรืออันตรายต่อร่างกาย

นอกจากนี้ ปัญหาเรื่องไลฟ์สไตล์การท่องเที่ยวที่ไม่สอดคล้องกัน หรือ Mismatch DNA เช่น ความต่างด้านงบประมาณ กิจกรรมที่สนใจ หรือวงจรเวลาการพักผ่อน (Travel Daily Rhythm) ยังเป็นปัจจัยหลักที่ส่งผลกระทบโดยตรงต่อความพึงพอใจและคุณภาพของประสบการณ์การเดินทาง แม้ปัจจุบันจะมีแพลตฟอร์มตัวกลางที่ให้บริการหาเพื่อนเที่ยว แต่ส่วนใหญ่ยังขาดระบบการวิเคราะห์พฤติกรรมผู้ใช้ในเชิงลึกเพื่อการจับคู่ที่แม่นยำ และยังไม่ได้บูรณาการปัญญาประดิษฐ์ (AI) เข้ามาช่วยอำนวยความสะดวกในลักษณะผู้ช่วยส่วนตัวอย่างแท้จริง

คณะผู้จัดทำพัฒนา Go With Us เป็นแอปพลิเคชัน Native iOS สำหรับค้นหาและเข้าร่วมทริป โดยระบบคำนวณความสอดคล้องระหว่างข้อมูลผู้ใช้กับทริปเป็น Match Score จากความสนใจ ช่วงเวลาที่สะดวก งบประมาณ และจำนวนสถานที่ต่อวัน ระบบใช้ Cosine Similarity สำหรับปัจจัยหลายตัวเลือกสองด้านแรก และคำนวณคะแนนอีกสองด้านตามสูตรเฉพาะ นอกจากนี้ยังใช้ Gemini API สนับสนุนการวางแผนการเดินทาง ใช้ OTP ยืนยันอีเมล และมีการตรวจการเคลื่อนไหวของใบหน้าผ่านกล้องในหน้าโปรไฟล์

1.2 วัตถุประสงค์ของโครงการ

1.2.1 เพื่อพัฒนาแอปพลิเคชัน **Go With Us** บน **iOS** สำหรับแนะนำทริปจากความสะดวกสบายระหว่างข้อมูลผู้ใช้กับทริป

1.2.2 เพื่อออกแบบและพัฒนาฐานข้อมูลและระบบบริการส่วนหลังของแอปพลิเคชัน **Go With Us**

1.2.3 เพื่อบูรณาการเทคโนโลยีในการสร้างแผนการเดินทางอัตโนมัติ โดยใช้ **MapKit** สำหรับการค้นหาและแสดงข้อมูลสถานที่ ร่วมกับปัญญาประดิษฐ์เชิงสร้างสรรค์ (**Gemini API**)

1.3 ขอบเขตของโครงการ (Scope)

1.3.1 ขอบเขตด้านข้อมูล (Data Scope)

ระบบมีการจัดเก็บ ดึงข้อมูล และประมวลผลข้อมูลในหมวดหมู่ต่าง ๆ ดังนี้

1.3.1.1 ข้อมูลความต้องการของผู้ใช้: จัดเก็บ 4 ปัจจัย ได้แก่ หมวดหมู่ความสนใจ ช่วงเวลาที่สะดวก งบประมาณ และจำนวนสถานที่ที่ต้องการเที่ยวต่อวัน

1.3.1.2 ข้อมูลการประมวลผลเวกเตอร์ (**Vector Data**): แปลงข้อมูลความชอบของผู้ใช้และคุณลักษณะของทริปให้เป็น **Preference Vector** และ **Trip Tags Vector** เพื่อใช้ในการคำนวณคะแนนความคล้ายคลึง.

1.3.1.3 ข้อมูลการตรวจสอบตัวตน: จัดเก็บสถานะการยืนยันอีเมลด้วย **OTP** และข้อมูลที่ใช้สำหรับการตรวจสอบคำขอในหน้าโปรไฟล์ การตรวจการเคลื่อนไหวของใบหน้าใช้กล้องและ **Apple Vision** ไม่ใช่การเปรียบเทียบโครงหน้ากับรูปโปรไฟล์โดยอัตโนมัติ

1.3.1.4 ข้อมูลการดำเนินกิจกรรม: จัดเก็บข้อมูลรายละเอียดทริป, รายการแจ้งเตือน (**Notifications**), และประวัติการสนทนาในรูปแบบแชทเรียลไทม์

1.3.1.5 ข้อมูลแผนการเดินทาง (**Itinerary Data**): ดึงข้อมูลสถานที่ท่องเที่ยวจาก **MapKit** (**Apple**) และข้อมูลแผนการเดินทางที่สร้างโดย **Gemini API**

1.3.2 ขอบเขตด้านเทคโนโลยีและเครื่องมือที่ใช้ (Technology Scope)

การพัฒนาจะถูกแบ่งออกเป็นส่วน ๆ ตามเทคโนโลยีดังนี้

1.3.2.1 แอปพลิเคชันบน iOS (Native iOS Application)

พัฒนาด้วยภาษา **Swift** โดยใช้ **SwiftUI** สำหรับสร้างส่วนติดต่อผู้ใช้งาน และใช้ **Xcode** เป็นเครื่องมือหลักในการพัฒนาแอปพลิเคชันบนระบบปฏิบัติการ **iOS**

1.3.2.2 ระบบจัดการหลังบ้าน (Backoffice)

พัฒนาเป็น **Web Application** ด้วยภาษา **TypeScript** โดยใช้ **React.js** ร่วมกับ **Vite** และจัดการส่วนติดต่อผู้ใช้งานด้วย **Tailwind CSS** สำหรับให้ผู้ดูแลระบบจัดการข้อมูลผู้ใช้งาน ทริป และข้อมูลต่าง ๆ ภายในระบบ

1.3.2.3 ส่วนประมวลผลหลังบ้าน (Backend)

ใช้ **Node.js** สำหรับจัดการตรรกะของระบบ การประมวลผลข้อมูล และการเชื่อมต่อกับ **API** รวมถึงการติดต่อกับฐานข้อมูล

1.3.2.4 ระบบจัดการฐานข้อมูล (Database)

ใช้ **PostgreSQL** บน **Supabase** ซึ่งเป็นฐานข้อมูลเชิงสัมพันธ์สำหรับจัดเก็บข้อมูลผู้ใช้งาน ข้อมูลทริป ข้อมูลการจอง และข้อความแชท

1.3.2.5 ปัญญาประดิษฐ์และ API ภายนอก

บูรณาการ **Generative AI** และ **API** ภายนอกที่เกี่ยวข้องกับระบบ เช่น **MapKit** สำหรับการแสดงผลและจัดการข้อมูลแผนที่ และใช้ **JSON Web Token (JWT)** สำหรับการยืนยันตัวตน และรักษาความปลอดภัยในการเข้าถึงข้อมูล

1.3.3 ขอบเขตด้านกรอบการทำงานและสถาปัตยกรรม (Framework & Methodology Scope)

1.3.3.1 วงจรการพัฒนา: ใช้กระบวนการทางวิศวกรรมซอฟต์แวร์ตามวงจร **SDLC** (**System Development Life Cycle**)

1.3.3.2 ออกแบบระบบในรูปแบบ **Client-Server Architecture** โดยแอปพลิเคชันหลักพัฒนาด้วยภาษา **Swift** และ **SwiftUI** สำหรับระบบปฏิบัติการ **iOS** เชื่อมต่อกับ **Backend** ผ่าน **API** และมีระบบ **Backoffice** ซึ่งพัฒนาเป็น **Web Application** สำหรับผู้ดูแลระบบ โดยจัดโครงสร้างซอฟต์แวร์ให้แยกส่วนการแสดงผล การประมวลผล และการจัดการข้อมูลอย่างเป็นระบบ

1.3.3.3 ยึดหลัก **Mobile-First Design** และออกแบบส่วนติดต่อผู้ใช้งานให้เหมาะสมกับอุปกรณ์ **iOS** โดยคำนึงถึงแนวทางการออกแบบ **Apple Human Interface Guidelines (HIG)** เพื่อให้ผู้ใช้งานสามารถเรียนรู้และใช้งานระบบได้อย่างสะดวกและเป็นธรรมชาติ ส่วนระบบ **Backoffice** ออกแบบให้เหมาะสมกับการใช้งานบน **Web Application**

1.3.3.4 ใช้ **Cosine Similarity** คำนวณคะแนนย่อยด้านหมวดหมู่ความสนใจและช่วงเวลาที่เหมาะสม แล้วนำคะแนนดังกล่าวมาเฉลี่ยร่วมกับคะแนนงบประมาณและจำนวนกิจกรรมต่อวัน เพื่อแสดง **Match Score** และจัดลำดับทริปที่เหมาะสมกับผู้ใช้งาน

1.5 คำนิยามศัพท์เฉพาะ

1.5.1 Go With Us หมายถึง ชื่อระบบแอปพลิเคชันสำหรับการจับคู่เพื่อนร่วมเดินทางและวางแผนเดินทางอัจฉริยะที่พัฒนาขึ้นในโครงการนี้ โดยแอปพลิเคชันหลักพัฒนาด้วยเทคโนโลยี Native iOS และมีระบบ Backoffice ในรูปแบบ Web Application สำหรับผู้ดูแลระบบ

1.5.2 User DNA หมายถึง ชื่อที่ใช้เรียกชุดข้อมูลความต้องการของผู้ใช้ได้แก่ หมวดหมู่ ความสนใจ ช่วงเวลาที่สะดวก งบประมาณ และจำนวนสถานที่ต่อวัน โดยแปลงเป็นเวกเตอร์เฉพาะหมวดหมู่และช่วงเวลา ส่วนอีกสองปัจจัยคำนวณจากค่าตัวเลข

1.5.3 Smart Matching หมายถึง กระบวนการคำนวณความสอดคล้องระหว่างข้อมูลผู้ใช้งานกับคุณลักษณะของทริปจากคะแนนย่อย 4 ปัจจัย โดยใช้ Cosine Similarity เฉพาะหมวดหมู่และช่วงเวลา แล้วแสดงผลเป็น Match Score

1.5.4 เศรษฐกิจแบบแบ่งปัน (Sharing Economy) หมายถึง แนวคิดทางเศรษฐกิจที่เน้นการแบ่งปันทรัพยากรหรือการใช้ประโยชน์ร่วมกันในกิจกรรมการเดินทาง เช่น การหารเฉลี่ยค่าใช้จ่ายในทริป เพื่อลดภาระต้นทุนและเพิ่มปฏิสัมพันธ์ทางสังคมระหว่างนักเดินทาง

1.5.5 การตรวจการเคลื่อนไหวของใบหน้า หมายถึง ขั้นตอนใช้กล้องและ Apple Vision ตรวจพบใบหน้าและการทำตามคำสั่งบนหน้าจอ ไม่ใช่ Face ID หรือระบบพิสูจน์ว่าผู้ใช้เป็นบุคคลเดียวกับภาพเอกสาร

1.5.6 MapKit หมายถึง Framework ของ Apple ที่นำมาใช้ในการจัดการและแสดงข้อมูลแผนที่ภายในแอปพลิเคชัน Go With Us บนระบบปฏิบัติการ iOS โดยสามารถใช้ในการแสดงตำแหน่งสถานที่ จุดหมายปลายทาง และข้อมูลที่เกี่ยวข้องกับเส้นทางการเดินทางเพื่อสนับสนุนการวางแผนและการเดินทางของผู้ใช้งาน

1.5.7 Cosine Similarity หมายถึง วิธีการทางคณิตศาสตร์ที่ใช้วัดความคล้ายคลึงระหว่างเวกเตอร์สองชุด โดยคำนวณจากค่ามุมในปริภูมิหลายมิติ เพื่อเปรียบเทียบความสนใจของผู้ใช้และคุณลักษณะของทริป

1.6 ผลที่คาดว่าจะได้รับ

1.6.1 ผู้ใช้งานสามารถเห็นทริปที่มีข้อมูลสอดคล้องกับความต้องการของตนเอง พร้อม Match Score และรายละเอียดคะแนนย่อยเพื่อประกอบการตัดสินใจ โดยคะแนนดังกล่าวไม่รับประกันความเข้ากันได้ระหว่างบุคคล

1.6.2 ได้ระบบ Smart Matching ที่สามารถวิเคราะห์ความเข้ากันได้ของผู้ใช้งานและทริป โดยใช้ Cosine Similarity เพื่อคำนวณค่าความเหมาะสมและช่วยจัดลำดับผลการจับคู่

1.6.3 สนับสนุนความน่าเชื่อถือของบัญชีผู้ใช้งานด้วยการยืนยันอีเมลผ่าน **OTP** และการตรวจการเคลื่อนไหวของใบหน้าผ่านกล้อง โดยผลการตรวจดังกล่าวไม่ใช่หลักฐานยืนยันอัตลักษณ์บุคคลในระดับเดียวกับการตรวจเอกสาร โดยผู้เชี่ยวชาญ

1.6.4 ได้ระบบ **Backoffice** สำหรับผู้ดูแลระบบในการจัดการข้อมูลและตรวจสอบข้อมูลที่เกี่ยวข้องกับระบบ **Go With Us** อย่างเป็นระบบ

1.6.5 ส่งเสริมภาพลักษณ์และการเติบโตของอุตสาหกรรมการท่องเที่ยว: ช่วยกระตุ้นให้นักท่องเที่ยวกลุ่มเดินทางคนเดียว (**Solo Traveler**) มีความกล้าในการออกเดินทางและพบปะสังคมใหม่ๆ ภายใต้แนวคิด เศรษฐกิจแบบแบ่งปัน (**Sharing Economy**) ซึ่งเป็นการส่งเสริมการท่องเที่ยวในยุคดิจิทัลอย่างยั่งยืน

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 วิวัฒนาการและพฤติกรรมในการค้นหาผู้ร่วมเดินทาง

2.1.1 วิวัฒนาการของการรวมกลุ่มท่องเที่ยว (Evolution of Group Travel) ในอดีตการเดินทางเป็นกลุ่มมักจำกัดอยู่ภายในเครือญาติ คนรู้จัก หรือการซื้อแพคเกจทัวร์ที่มีการกำหนดตารางล่วงหน้า ต่อมาเมื่อแพลตฟอร์มดิจิทัลเข้ามามีบทบาท การค้นหาผู้ร่วมเดินทางจึงถูกย้ายเข้าสู่เว็บบอร์ดและกลุ่มสังคมออนไลน์ (Social Media) ซึ่งเพิ่มความยืดหยุ่นให้นักเดินทางสามารถตั้งกระทู้ค้นหาเป้าหมายและสร้าง คอมมูนิตีใหม่ได้อย่างเสรี (Wang et al., 2018)

2.1.2 ปัญหาและความท้าทายในการหาผู้ร่วมเดินทาง แม้เครือข่ายสังคมจะช่วยให้หากกลุ่มทริปได้อิสระขึ้น แต่ยังคงเผชิญความท้าทาย 2 ประการ คือ 1) ด้านความปลอดภัย ความกังวลด้านความปลอดภัยในการพบคนแปลกหน้า (Pizam et al., 2020) และ 2) ด้านความเข้ากันได้ (Lifestyle Mismatch) โดยเฉพาะปัญหาความไม่เข้ากันทางไลฟ์สไตล์ รวมไปถึงงบประมาณที่ไม่สอดคล้องกัน เช่น งบประมาณ ตารางเวลา และความสนใจเฉพาะทาง ซึ่งนำไปสู่ความขัดแย้งระหว่างการเดินทาง (Zhang & Peng, 2021)

2.1.3 การเปลี่ยนผ่านสู่แพลตฟอร์มจับคู่ตามความสนใจ (Transition to Interest-Based Platforms) เพื่อแก้ไขช่องโหว่ด้านพฤติกรรมที่แตกต่าง การพัฒนาการท่องเที่ยวยุคใหม่จึงนำระบบอัลกอริทึมเข้ามาช่วยวิเคราะห์ความสนใจส่วนบุคคล (Personalized Interests) ทำให้ระบบแอปพลิเคชันสามารถคัดกรองทริปและสถานที่ให้สอดคล้องกับประวัติของผู้ใช้ได้แม่นยำขึ้น (Buhalis & Amaranggana, 2015) โดยในโครงการนี้ได้นำแนวคิดดังกล่าวมาพัฒนาเป็น "User DNA" และใช้ อัลกอริทึม Cosine Similarity เพื่อคำนวณค่าความคล้ายคลึงของพฤติกรรมอย่างเป็นระบบ

2.1.4 พฤติกรรมการเดินทางของแต่ละช่วงวัยอาจแตกต่างกันในด้านความคุ้มค่า ความยืดหยุ่น และรูปแบบกิจกรรม ข้อมูลนี้ใช้ประกอบการออกแบบคำถามและส่วนติดต่อผู้ใช้ ไม่ได้ใช้กำหนดลำดับความสำคัญของปัจจัยในสูตร Smart Matching

ตารางที่ 2.1 เปรียบเทียบพฤติกรรมการเรียนรู้และความต้องการในการท่องเที่ยวแยกตามช่วงอายุ (Generations)

กลุ่มช่วงอายุ (Generation)	อายุโดยประมาณ	รูปแบบพฤติกรรมการเรียนรู้และการเดินทาง	เป้าหมาย/สิ่งที่ให้ความสำคัญในการจัดทริป
Generation Z	15 - 28 ปี	เรียนรู้รวดเร็วจากโซเชียลมีเดีย นิยมความสนุกแบบคล่องตัวและแปลกใหม่	เน้นทริปที่คุ้มค่า (Budget) และสามารถยืดหยุ่นแผนกะทันหันได้
Generation Y (Millennials)	29 - 43 ปี	ให้ความสำคัญกับสมดุลชีวิตส่วนตัว (Work-Life Balance) และเชิงรักษ์โลก	เน้นตารางเวลาที่ลงตัว ความสะดวกสบาย และความผ่อนคลาย
Generation X	44 - 59 ปี	เน้นอ่านแหล่งข้อมูลที่น่าเชื่อถือ นิยมการเที่ยวแบบพักผ่อนจริงจัง ครอบครัว	เน้นความปลอดภัย ความคุ้มค่าด้านบริการและที่ชาร์จพลังงาน
Baby Boomers	60 ปีขึ้นไป	เรียนรู้ข้อมูลง่ายๆ ไม่ซับซ้อน มักเดินทางเป็นแพคเกจทัวร์หรือกลุ่มวัยเกษียณ	เน้นสุขภาพ ปลอดภัย และความสะดวกสบายระหว่างทริปขั้นสุด

(อ้างอิงและประยุกต์ทฤษฎีพฤติกรรมผู้บริโภคจาก: ศูนย์พัฒนาวิชาการด้านตลาดการท่องเที่ยว (TAT Academy) และวารสาร TAT Review Magazine, 2565-2566)
<https://so07.tci-thaijo.org/index.php/LiberalJ/article/view/9500>

2.2 พฤติกรรมและแรงจูงใจในการค้นหาเพื่อนร่วมเดินทาง

2.2.1 แรงจูงใจในการค้นหาเพื่อนร่วมทริปผ่านแพลตฟอร์ม (Motivations for Seeking Travel Companions) พฤติกรรมการหาเพื่อนเที่ยวกลุ่มใหม่มักเกิดจากข้อจำกัดของคนโสดตัว เช่น วันหยุดไม่ตรงกัน หรือความชอบคนละสไตล์ นักท่องเที่ยวจึงเกิดแรงจูงใจที่จะเปิดรับคนแปลกหน้าที่มีความสนใจเฉพาะทาง (Niche Interests) เหมือนกัน เช่น การชอบถ่ายรูปคาเฟ่ หรือกิจกรรมปีนเขา ตลอดจนความต้องการในการลดภาระค่าใช้จ่ายด้วยการหารเฉลี่ยตามแนวคิด เศรษฐกิจแบบแบ่งปัน (Sharing Economy) สิ่งเหล่านี้ล้วนเป็นแรงผลักดันให้เกิดความต้องการระบบจัดหาคู่หูที่มีความปลอดภัยสูงเพื่อสร้างความมั่นใจในการเริ่มปฏิสัมพันธ์กับคนใหม่ ๆ

2.2.2 อิทธิพลของความเข้ากันได้ทางพฤติกรรม (Influence of Behavioral Compatibility) ระบบ Go With Us ใช้ช่วงเวลาที่สะดวก งบประมาณ จำนวนกิจกรรมต่อวัน และหมวดหมู่ความสนใจเป็นปัจจัยประกอบการจับคู่ผู้ไปกับทริป หมวดหมู่และช่วงเวลาถูกแปลง

เป็นเวกเตอร์เพื่อคำนวณด้วย **Cosine Similarity** ส่วนงบประมาณและจำนวนกิจกรรมใช้สูตรคะแนนเฉพาะ ผลลัพธ์ช่วยให้ผู้ใช้พิจารณาความเหมาะสมของทริปก่อนตัดสินใจเข้าร่วม

2.2.3 นักท่องเที่ยวที่เดินทางคนเดียวอาจต้องการรวมกลุ่มชั่วคราวเพื่อทำกิจกรรมหรือแบ่งค่าใช้จ่าย ระบบจึงมีข้อมูลโปรไฟล์ การยืนยันอีเมล และช่องทางให้ตรวจสอบรายละเอียดทริปก่อนตัดสินใจเข้าร่วม อย่างไรก็ตาม ขั้นตอนเหล่านี้ไม่สามารถรับประกันความปลอดภัยของการเดินทางจริงได้

2.3 ทฤษฎีการวิเคราะห์ความคล้ายคลึงและปัญญาประดิษฐ์

2.3.1 อัลกอริทึมการคำนวณความคล้ายคลึงเชิงโคไซน์ (Cosine Similarity Algorithm) ใช้วัดมุมระหว่างเวกเตอร์สองชุด โดยค่าใกล้ 1 หมายถึงทิศทางของเวกเตอร์คล้ายกันมาก ในระบบนี้ใช้คำนวณคะแนนย่อยด้านหมวดหมู่ความสนใจและช่วงเวลาที่สะดวก จากนั้นจึงนำคะแนนย่อยไปเฉลี่ยร่วมกับคะแนนงบประมาณและกิจกรรมเพื่อให้ได้ **Match Score** ดังสูตรและขั้นตอนต่อไปนี้

Cosine Similarity คำนวณความคล้ายคลึงระหว่างเวกเตอร์ความสนใจของผู้ใช้กับเวกเตอร์หมวดหมู่ของทริป และระหว่างเวกเตอร์ช่วงเวลาที่สะดวกของทั้งสองฝั่ง ผลลัพธ์แต่ละด้านถูกแปลงเป็นคะแนนย่อย 0–100 ก่อนนำไปคำนวณ **Match Score** ร่วมกับปัจจัยอื่น

ขั้นตอนที่ 1 การคำนวณผลคูณจุด (Dot Product): เป็นการนำข้อมูล **Preference**

Vector ของผู้ใช้ (เช่น ความชอบสายลุย) และ **Trip Tags Vector** ของทริปเป้าหมาย (เช่น ทริปเดินป่า) มาทำผลคูณเชิงสเกลาร์เพื่อหา "จุดร่วม" ที่มีความสนใจตรงกัน ยังมีคุณลักษณะที่ซ้อนทับกันมาก ค่าคะแนนในส่วนนี้จะยิ่งสูงขึ้น

ขั้นตอนที่ 2: การปรับค่าเข้าสู่บรรทัดฐานเดียวกัน (Magnitude Division): หลังจากได้ผลคูณแล้วระบบจะทำการคำนวณค่าความยาวหรือขนาดปริภูมิบรรทัดฐานของเวกเตอร์

ระบบจะคำนวณขนาด (Magnitude) เพื่อใช้เป็นฐานหารในการทำ

Normalization กระบวนการนี้ช่วยให้เกิดความยุติธรรมในการประมวลผล โดยป้องกันไม่ให้ผู้ใช้ที่ระบุความชอบจำนวนมากได้เปรียบผู้ใช้ที่ระบุความชอบน้อย

ขั้นตอนที่ 3 การชี้วัดและประเมินผลลัพธ์ทิศทางความคล้ายคลึง (Similarity Evaluation)

ค่าสัมประสิทธิ์ที่ได้จะอยู่ระหว่าง 0 ถึง 1 เสมอ หากค่าเข้าใกล้ 1 หมายถึงมีความชอบสัมพันธ์กันอย่างสมบูรณ์แบบ (Perfectly Matched) แต่หากเข้าใกล้ 0 หมายถึงโปรไฟล์ข้อมูลไม่มีจุดร่วมหรือไลฟ์สไตล์ที่เกี่ยวข้องกันเลย (No Similarity)

2.3.2 อัลกอริทึม K-Nearest Neighbors (KNN) เป็นเทคนิคการจัดหมวดหมู่ข้อมูลโดยพิจารณาจาก "เพื่อนบ้านที่ใกล้ที่สุด" เพื่อระบุตัวตนของผู้ใช้ที่มีความชอบคล้ายคลึงกันในระดับโครงสร้าง และนำเสนอข้อมูลทริปหรือเพื่อนร่วมทางที่เหมาะสมที่สุดให้แก่ผู้ใช้งาน

2.3.3 อัลกอริทึม K-Means Clustering เป็นการเรียนรู้แบบไม่มีผู้สอน (Unsupervised Learning) ที่ใช้จัดกลุ่มสมาชิกในระบบตามพฤติกรรม (User Segmentation) เพื่อวิเคราะห์ภาพรวมของกลุ่มนักท่องเที่ยวและช่วยให้ระบบสามารถจำกัดขอบเขตการจับคู่ภายในกลุ่มที่มีลักษณะใกล้เคียงกันก่อนนำไปคำนวณความคล้ายคลึงเชิงลึก

2.4 เทคโนโลยีปัญญาประดิษฐ์เชิงสร้างสรรค์

2.4.1 โมเดลภาษาขนาดใหญ่ (Large Language Models: LLM) ศึกษาโครงสร้าง Transformer Architecture ที่ใช้กลไก Self-Attention ในการประมวลผลความสัมพันธ์ของข้อมูลเชิงบริบท คุณสมบัตินี้ช่วยให้ระบบสามารถสรุปข้อมูลสถานที่ท่องเที่ยวและวางแผนการเดินทาง (Itinerary Planning) ที่มีความซับซ้อนและมีความต่อเนื่องสอดคล้องกันตามเงื่อนไขที่กำหนดได้

2.4.2 Google Gemini API เป็นเทคโนโลยีปัญญาประดิษฐ์ที่ใช้วางแผนการเดินทางจากข้อมูลที่ใช้กรอก เช่น จุดหมาย สถานที่ที่เลือก วันเดินทาง งบประมาณ หมวกหมู่ และความสนใจ โดยข้อมูลตำแหน่งที่ผู้ใช้เลือกผ่านส่วนติดต่อแผนที่ถูกส่งเป็นข้อความประกอบคำสั่ง (Prompt) ผู้ใช้ยังต้องตรวจสอบและแก้ไขแผนก่อนบันทึก

2.4.3 วิศวกรรมคำสั่ง (Prompt Engineering) เทคนิคการออกแบบชุดคำสั่งเพื่อควบคุมผลลัพธ์ของ AI ให้มีประสิทธิภาพสูงสุด โดยในโครงการนี้มีการกำหนดให้ AI ตอบกลับในรูปแบบ JSON Format เพื่อให้ระบบส่วนหลัง (Backend) สามารถส่งต่อข้อมูลไปยังส่วนหน้า (Frontend) เพื่อประมวลผลและแสดงผลแผนการเดินทางในรูปแบบตารางหรือการ์ดข้อมูล que ผู้ใช้งานสามารถอ่านและเข้าใจได้ทันที

2.5 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาระบบ

ในการพัฒนาแอปพลิเคชัน Go With Us คณะผู้จัดทำได้เลือกใช้เครื่องมือ ภาษาโปรแกรม ไลบรารี และบริการที่เหมาะสมกับสถาปัตยกรรมของระบบ โดยแบ่งการทำงานออกเป็นแอปพลิเคชัน Native iOS ระบบ Backoffice Web ระบบ Backend และระบบจัดการฐานข้อมูล รวมถึงเทคโนโลยีด้านปัญญาประดิษฐ์และการประมวลผลข้อมูล ดังรายละเอียดต่อไปนี้

2.5.1 ภาษา Swift และ SwiftUI

Swift เป็นภาษาโปรแกรมที่พัฒนาโดยบริษัท Apple สำหรับใช้ในการพัฒนาแอปพลิเคชันบนระบบนิเวศของ Apple โดยมีคุณสมบัติด้านความปลอดภัยของชนิดข้อมูล

(Type Safety) และการจัดการหน่วยความจำที่เหมาะสมกับการพัฒนาแอปพลิเคชันสมัยใหม่ SwiftUI เป็น Framework สำหรับสร้างส่วนติดต่อผู้ใช้งาน (User Interface) ของแอปพลิเคชันบนระบบปฏิบัติการของ Apple โดยใช้แนวทางการพัฒนาแบบ Declarative UI ซึ่งช่วยให้สามารถกำหนดโครงสร้างและสถานะของหน้าจอได้อย่างเป็นระบบ

ในโครงการ Go With Us ได้ใช้ Swift และ SwiftUI เป็นเทคโนโลยีหลักในการพัฒนาแอปพลิเคชัน Native iOS สำหรับการแสดงผลข้อมูล การจัดการสถานะของผู้ใช้งาน การสร้างหน้าจอการใช้งาน และการเชื่อมต่อกับ Backend API

2.5.2 Xcode

Integrated Development Environment (IDE) ของบริษัท Apple สำหรับพัฒนาแอปพลิเคชันบนระบบปฏิบัติการ macOS และแพลตฟอร์มของ Apple โดยรองรับการเขียนโปรแกรม การทดสอบ การ Debugging การ Compile และการ Build แอปพลิเคชัน

ในโครงการ Go With Us ใช้ Xcode เป็นเครื่องมือหลักสำหรับพัฒนา ทดสอบ และ Build แอปพลิเคชัน Native iOS ที่พัฒนาด้วย Swift และ SwiftUI รวมถึงใช้สำหรับเชื่อมต่อและทดสอบความสามารถต่าง ๆ ของอุปกรณ์ iOS

2.5.3 MapKit

MapKit เป็น Framework ของ Apple สำหรับการทำงานเกี่ยวกับแผนที่และข้อมูลตำแหน่งบนระบบปฏิบัติการ iOS โดยสามารถนำมาใช้ในการแสดงแผนที่ ตำแหน่งสถานที่ จุดหมายปลายทาง และข้อมูลที่เกี่ยวข้องกับการเดินทาง

ในโครงการ Go With Us ได้นำ MapKit มาใช้เพื่อสนับสนุนการแสดงผลข้อมูลสถานที่และตำแหน่งทางภูมิศาสตร์ภายในแอปพลิเคชัน รวมถึงใช้เป็นส่วนประกอบในการสนับสนุนการวางแผนการเดินทางของผู้ใช้งาน

2.5.4 ภาษา TypeScript

TypeScript เป็นภาษาโปรแกรมที่พัฒนาต่อยอดจาก JavaScript โดยเพิ่มระบบตรวจสอบชนิดข้อมูลแบบ Static Type Checking เพื่อช่วยลดข้อผิดพลาดที่อาจเกิดขึ้นระหว่างการพัฒนาซอฟต์แวร์

ในโครงการ **Go With Us** ใช้ **TypeScript** สำหรับพัฒนาระบบ **Backoffice Web** และส่วนประกอบที่เกี่ยวข้องกับการจัดการข้อมูลของผู้ดูแลระบบ โดยช่วยให้การกำหนดโครงสร้างข้อมูลและการส่งผ่านข้อมูลระหว่างส่วนต่าง ๆ ของระบบมีความชัดเจนและลดข้อผิดพลาดจากชนิดข้อมูล

2.5.5 React.js และ Vite

React.js เป็นไลบรารีสำหรับพัฒนาส่วนติดต่อผู้ใช้งาน (**User Interface**) โดยมีแนวคิดหลักในการพัฒนาแบบ **Component-Based Architecture** ซึ่งช่วยแบ่งส่วนติดต่อผู้ใช้งานออกเป็น **Component** ย่อย ๆ และสามารถนำกลับมาใช้ซ้ำได้

2.5.6 Tailwind CSS

Tailwind CSS เป็น **CSS Framework** ที่ใช้แนวคิด **Utility-First CSS** โดยจัดเตรียมคลาสสำหรับกำหนดรูปแบบของส่วนติดต่อผู้ใช้งาน เช่น ขนาด ระยะห่าง สี ตัวอักษร และการจัดวางองค์ประกอบ

ในโครงการ **Go With Us** ใช้ **Tailwind CSS** สำหรับจัดรูปแบบส่วนติดต่อผู้ใช้งานของระบบ **Backoffice Web** เพื่อให้การออกแบบหน้าจอมีรูปแบบที่สอดคล้องกัน และสามารถปรับการแสดงผลให้เหมาะสมกับขนาดหน้าจอที่แตกต่างกัน

2.5.7 Node.js

Node.js เป็นสภาพแวดล้อมสำหรับประมวลผล **JavaScript** บนฝั่งเซิร์ฟเวอร์ (**Server-side Runtime Environment**) ซึ่งเหมาะสำหรับการพัฒนา **Backend** และ **API** ที่ต้องรองรับการรับส่งข้อมูลระหว่าง **Client** และ **Server**

ในโครงการ **Go With Us** ใช้ **Node.js** สำหรับพัฒนา **Backend** เพื่อจัดการตรรกะของระบบ การตรวจสอบข้อมูล การจัดการ **Authentication** การเชื่อมต่อฐานข้อมูล และการให้บริการ **API** แก่แอปพลิเคชัน **Native iOS** และระบบ **Backoffice**

2.5.8 PostgreSQL และ Supabase

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (**Relational Database Management System: RDBMS**) ที่รองรับการจัดเก็บข้อมูลในรูปแบบตารางและการสร้างความสัมพันธ์ระหว่างข้อมูล โดยมีความสามารถในการกำหนดข้อจำกัดของข้อมูล (**Constraints**) และความสัมพันธ์ระหว่างตาราง

ในโครงการ **Go With Us** ใช้ **PostgreSQL** ผ่านแพลตฟอร์ม **Supabase** สำหรับจัดเก็บข้อมูลผู้ใช้งาน ข้อมูล **User DNA** ข้อมูลทริป ข้อมูลการจับคู่ ข้อมูลการสนทนา และข้อมูลอื่น ๆ ที่เกี่ยวข้องกับระบบ โดยใช้ความสัมพันธ์ระหว่างตารางเพื่อรักษาความถูกต้องและความสอดคล้องของข้อมูล

2.5.9 Visual Studio Code

Visual Studio Code (VS Code) เป็นโปรแกรมแก้ไขซอร์สโค้ด (**Source Code Editor**) ที่พัฒนาโดยบริษัท **Microsoft** และรองรับการติดตั้งส่วนขยายสำหรับภาษาและเครื่องมือต่าง ๆ

ในโครงการ **Go With Us** ใช้ **Visual Studio Code** เป็นเครื่องมือสำหรับพัฒนา **Backend** และระบบ **Backoffice** รวมถึงใช้สำหรับการจัดการโครงสร้างโปรเจกต์ ตรวจสอบข้อผิดพลาดของโค้ด และเชื่อมต่อกับระบบควบคุมเวอร์ชัน **Git**

2.5.10 Generative AI (Gemini API)

Gemini API เป็นบริการ **Generative AI** ที่นำมาใช้ในการสร้างและประมวลผลข้อมูลแผนการเดินทาง โดยระบบจะนำข้อมูลความต้องการของผู้ใช้งาน เช่น งบประมาณ ช่วงเวลา และรูปแบบการเดินทาง มาใช้เป็นข้อมูลประกอบการสร้างแผนการเดินทางที่เหมาะสมกับผู้ใช้งาน

2.5.11 Cosine Similarity

Cosine Similarity เป็นวิธีวัดความคล้ายคลึงของเวกเตอร์จากมุมระหว่างเวกเตอร์สองชุดตามแนวคิดแบบจำลองเวกเตอร์ของ **Salton, Wong และ Yang (1975)** โครงการนี้ใช้กับหมวดหมู่ความสนใจและช่วงเวลาที่เลือกได้หลายรายการ ไม่ได้นำงบประมาณหรือจำนวนสถานที่ต่อวันเข้าเวกเตอร์เดียวกัน

Go With Us ใช้ **Cosine Similarity** เปรียบเทียบเวกเตอร์หมวดหมู่ความสนใจและช่วงเวลาระหว่างผู้ใช้กับทริป ส่วนงบประมาณและจำนวนสถานที่ต่อวันคำนวณเป็นคะแนนย่อยด้วยสูตรที่ต่างออกไป ก่อนนำคะแนนที่มีข้อมูลมาหาค่าเฉลี่ย

2.5.12 เครื่องมือสนับสนุนการพัฒนาด้วยปัญญาประดิษฐ์ (AI-Assisted Development Tool)

เครื่องมือปัญญาประดิษฐ์ถูกนำมาใช้เป็นเครื่องมือสนับสนุนการพัฒนาซอฟต์แวร์ โดยช่วยในด้านการวิเคราะห์โค้ด การค้นหาแนวทางแก้ไขข้อผิดพลาด (**Debugging**) การระดมแนวคิดในการออกแบบระบบ และการช่วยจัดทำเอกสารทางเทคนิค

เครื่องมือดังกล่าวทำหน้าที่เป็นเครื่องมือสนับสนุนกระบวนการพัฒนาเท่านั้น และไม่ได้เป็นองค์ประกอบที่ทำงานอยู่ภายในระบบ **Go With Us** โดยตรง

2.6 ระบบความปลอดภัยและการพิสูจน์ตัวตนด้วยเทคโนโลยีชีวมาตร (Biometrics and Security)

2.6.1 การยืนยันอีเมลและการตรวจการเคลื่อนไหวของใบหน้า

ระหว่างสมัครสมาชิก ระบบส่ง **OTP** ไปยังอีเมลและสร้างบัญชีเมื่อยืนยันรหัสสำเร็จ ส่วนขั้นตอนในหน้าโปรไฟล์ใช้กล้องและ **Apple Vision** ตรวจพบใบหน้าและการทำตามคำสั่ง เช่น หันซ้าย หันขวา หรือยิ้ม ขั้นตอนดังกล่าวเป็นการตรวจการตอบสนองต่อคำสั่ง ไม่ใช่การเปรียบเทียบใบหน้ากับภาพโปรไฟล์หรือเอกสารราชการ และไม่ควรใช้ผลนี้เพียงอย่างเดียวเพื่อรับรองอัตลักษณ์บุคคล

2.6.2 การรักษาความปลอดภัยข้อมูล (Data Security) ระบบใช้การเข้ารหัสผ่านและการตรวจสอบสิทธิ์สำหรับ **API** ที่ต้องเข้าสู่ระบบ ส่วนการเข้ารหัสข้อมูลระหว่างทางขึ้นอยู่กับการตั้งค่า **HTTPS/TLS** ของสภาพแวดล้อมที่นำระบบไปใช้งาน รายละเอียดแบ่งเป็น 2 ส่วนดังนี้

2.6.2.1 การบริหารจัดการสิทธิ์เข้าถึงระบบ (Authentication): ระบบใช้ **JSON Web Token (JWT)** ตรวจสอบสิทธิ์เฉพาะ **API** ที่ต้องเข้าสู่ระบบ เช่น การสร้างทริปและการเข้าร่วมทริป ขณะที่ **API** สำหรับสมัครสมาชิก เข้าสู่ระบบ และอ่านข้อมูลทริปบางรายการเปิดให้เรียกโดยไม่ต้องมี **Token**

2.6.2.2 การปกป้องข้อมูล (Data Protection): ระบบเข้ารหัสผ่านของผู้ใช้ด้วย **bcrypt** ก่อนบันทึกลงฐานข้อมูล การรับส่งข้อมูลผ่าน **HTTPS/TLS** ต้องกำหนดและ

ตรวจสอบในสภาพแวดล้อมที่ให้บริการจริง จึงไม่สรุปว่าทุกการเชื่อมต่อเข้ารหัสแล้วจากโค้ด Backend เพียงอย่างเดียว

2.7 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง (Related Research and Studies)

2.7.1 Remigijus Paulavičius และคณะ (2023)

งานวิจัยเรื่องระบบแนะนำแผนการเดินทางแบบปรับตามความชอบรายบุคคล ภายใต้กรอบปัญหา **Tourist Trip Design Problem (TTDP)** ซึ่งเกี่ยวข้องกับ **Orienteering Problem** งานวิจัยเสนอวิธีจัดแผนการเดินทางโดยพิจารณาความชอบของผู้ใช้และข้อจำกัด เช่น ระยะเวลา จุดเริ่มต้น จุดสิ้นสุด และสถานที่ที่ต้องแวะ

2.7.2 Dietmar Jannach และคณะ (2010)

หนังสือ **Recommender Systems: An Introduction** ได้อธิบายหลักการของระบบแนะนำ โดยเน้นย้ำถึงเทคนิคการกรองข้อมูลร่วมกัน (**Collaborative Filtering**) และการคำนวณความคล้ายคลึงของข้อมูลผ่านวิธีวัดค่าคอไซน์ (**Cosine Similarity**) เพื่อประเมินระดับความใกล้เคียงระหว่างเวกเตอร์แสดงความสนใจของผู้ใช้งาน ซึ่งเป็นแนวทางเชิงคณิตศาสตร์ที่สามารถสร้างระบบแนะนำที่มีประสิทธิภาพสูงได้โดยไม่ต้องผ่านกระบวนการฝึกสอนแบบจำลอง (**Model Training**) ที่ซับซ้อน

2.7.3 J. B. MacQueen (1967)

การนำเสนออัลกอริทึม **K-Means** สำหรับการจัดกลุ่มข้อมูล (**Clustering**) โดยใช้หลักการกำหนดจุดศูนย์กลางของกลุ่ม (**Centroids**) และประมวลผลปรับค่าซ้ำ (**Iterative Process**) จนกระทั่งได้ผลลัพธ์ที่เสถียร ซึ่งอัลกอริทึมนี้ได้รับการยอมรับอย่างแพร่หลายในการวิเคราะห์และการแบ่งกลุ่มพฤติกรรมหรือลักษณะความสนใจของผู้ใช้งาน

2.7.4 นันทพล เหลืองอร่าม, ภูมิพันธ์ รักยิ้ม และ สุนันทา ศรีม่วง (2566)

งานวิจัยเรื่อง “การพัฒนากระบวนแนะนำสถานที่ท่องเที่ยวรายบุคคล” มีวัตถุประสงค์เพื่อพัฒนาระบบที่สามารถจัดอันดับสถานที่ท่องเที่ยวให้เหมาะสมกับผู้ใช้งาน โดยระบบรองรับการปรับปรุงแบบจำลองการจัดลำดับเมื่อมีข้อมูลใหม่เข้าสู่ระบบ ซึ่งผลการประเมินจากผู้ใช้งานจริงพบว่าความพึงพอใจโดยรวมอยู่ในระดับมาก (ค่าเฉลี่ยเท่ากับ 4.38)

2.7.5 ฐรา อังสกุล และ จิตมินต์ อังสกุล (2557)

งานวิจัยเรื่อง “ระบบแนะนำสถานที่ท่องเที่ยวรายบุคคลโดยใช้เทคนิคการจัดกลุ่มและกระบวนการลำดับชั้นเชิงวิเคราะห์” นำเสนอการบูรณาการเทคนิคการจัดกลุ่มข้อมูลร่วมกับกระบวนการลำดับชั้นเชิง

วิเคราะห์ (Analytic Hierarchy Process: AHP) เพื่อสนับสนุนการตัดสินใจเลือกสถานที่ท่องเที่ยวให้เหมาะสมกับผู้ใช้รายบุคคล โดยการคำนวณความสำคัญของปัจจัยที่เกี่ยวข้อง

บทที่ 3

วิธีการดำเนินการ

ในการดำเนินโครงการ การพัฒนาแอปพลิเคชันวางแผนการเดินทางอัจฉริยะ (Go With Us) ในรูปแบบ **Native iOS Application** โดยประยุกต์ใช้อัลกอริทึมการวิเคราะห์ความคล้ายคลึงเชิงโคไซน์ (Cosine Similarity) คณะผู้จัดทำได้มุ่งเน้นการใช้กระบวนการทางวิศวกรรมซอฟต์แวร์ตามวงจรการพัฒนาระบบ (System Development Life Cycle: SDLC) ควบคู่กับหลักการออกแบบแอปพลิเคชันสำหรับอุปกรณ์เคลื่อนที่ โดยพัฒนาแอปพลิเคชันหลักด้วยภาษา **Swift** และ **SwiftUI** เพื่อให้ได้ซอฟต์แวร์ที่มีความสมบูรณ์ทั้งในด้านสถาปัตยกรรมข้อมูลประสิทธิภาพการทำงาน และความสะดวกต่อการใช้งานจริง โดยในบทนี้จะอธิบายถึงรายละเอียดขั้นตอนการดำเนินงาน การออกแบบโครงสร้างการทำงาน และกลไกหลักของระบบที่เน้นการประมวลผลผ่านอัลกอริทึม **Cosine Similarity** เพื่อวิเคราะห์และจับคู่ผู้ใช้งานหรือทริปที่มีความสนใจและรูปแบบการเดินทางสอดคล้องกัน

3.1 การศึกษาปัญหาและกำหนดกรอบแนวคิดของระบบ

การพัฒนาระบบเริ่มจากปัญหาที่ผู้ใช้ต้องเปรียบเทียบความเหมาะสมของทริปด้วยตนเอง ระบบจึงออกแบบให้แอปพลิเคชัน **iOS** ส่งข้อมูลผ่าน **Backend** เพื่อคำนวณคะแนนและแสดงรายการทริปที่สอดคล้องกับความต้องการ

ด้วยเหตุนี้ คณะผู้จัดทำจึงกำหนดกรอบแนวคิดของแอปพลิเคชัน **Go With Us** ให้ใช้ **Smart Matching** วิเคราะห์ผู้เข้ากับทริปจาก 4 ปัจจัย ได้แก่ หมวดหมู่งบประมาณ จำนวนกิจกรรมเฉลี่ยต่อวัน และช่วงเวลา ระบบคำนวณคะแนนด้วยวิธีที่เหมาะสมกับข้อมูลแต่ละชนิด แล้วหาค่าเฉลี่ยของปัจจัยที่มีข้อมูลเพื่อสร้าง **Match Score** และเรียงลำดับทริป

นอกจากนี้ ระบบได้รับการออกแบบในรูปแบบ **Client-Server Architecture** โดยแอปพลิเคชันหลักพัฒนาเป็น **Native iOS Application** ด้วยภาษา **Swift** และ **SwiftUI** และใช้ **Xcode** เป็นเครื่องมือหลักในการพัฒนาและทดสอบระบบ ขณะที่ระบบ **Backoffice** พัฒนาในรูปแบบ **Web Application** ด้วย **TypeScript, React.js, Vite** และ **Tailwind CSS** สำหรับการบริหารจัดการข้อมูลโดยผู้ดูแลระบบ ทั้งสองส่วนเชื่อมต่อกับ **Backend** ผ่าน **API** เพื่อแลกเปลี่ยนข้อมูลและประมวลผลร่วมกับฐานข้อมูล **PostgreSQL** บน **Supabase** อันช่วยให้ระบบสามารถจัดการข้อมูลผู้ใช้งาน ข้อมูลทริป และข้อมูลที่เกี่ยวข้องกับการจับคู่ได้อย่างเป็นระบบ

3.1.1 ความต้องการเชิงฟังก์ชัน (Functional Requirements)

ความต้องการเชิงฟังก์ชันคือพฤติกรรมหรือความสามารถเฉพาะที่ระบบต้องกระทำได้ เมื่อผู้ใช้งานโต้ตอบกับแอปพลิเคชัน Go With Us โดยแบ่งตามกลุ่มฟังก์ชันดังนี้

ระบบจัดการบัญชีผู้ใช้งานและการยืนยันตัวตน (User Account Management & Authentication)

1. ระบบต้องอนุญาตให้ผู้ใช้งานใหม่ทำการ สมัครสมาชิก ด้วยอีเมล รหัสผ่าน และชื่อผู้ใช้งานได้
2. ระบบต้องรองรับการ เข้าสู่ระบบ (Login) ด้วยอีเมลและรหัสผ่านที่ลงทะเบียนไว้
3. ระบบต้องรองรับการยืนยันคำขอยืนยันตัวตนจากหน้าโปรไฟล์โดยผู้ใช้งานส่งข้อมูลและภาพที่ระบบ
กำหนดให้ผู้ดูแลตรวจสอบการยืนยันตัวตนเป็นกระบวนการเพิ่มเติมหลังสมัครสมาชิก
ไม่ใช่เงื่อนไขที่บังคับก่อนเข้าใช้งานครั้งแรก
4. ระบบต้องมีหน้า โปรไฟล์ผู้ใช้ (Profile) ที่แสดงชื่อ รูปโปรไฟล์ ประวัติการเดินทาง
และคำแนะนำความเหมาะสม
5. ระบบต้องอนุญาตให้ผู้ใช้งานแก้ไขโปรไฟล์ รูปภาพ และข้อมูลความสนใจของตนเองได้ตลอดเวลา
6. ระบบต้องรองรับการ ออกจากระบบ (Logout) และล้าง Session เพื่อความปลอดภัย

ระบบตั้งค่าพฤติกรรมและอัตลักษณ์ความชอบ (User DNA Settings)

7. ระบบต้องรองรับการให้ผู้ใช้งานเลือกหมวดหมู่กิจกรรมท่องเที่ยวที่สนใจในรูปแบบตัว
เลือกหลายรายการ (Multi-selection) เช่น ทะเล คาเฟ่ ภูเขา ประวัติศาสตร์ หรือ
กิจกรรมแอดเวนเจอร์ เพื่อรวบรวมข้อมูลความสนใจเบื้องต้นของผู้ใช้งาน
8. ระบบต้องให้ผู้ใช้งานระบุงบประมาณเป็นจำนวนเงิน และเลือกจำนวนสถานที่ที่สะดวกเที่ยวต่อ
วันตั้งแต่ 1-10 สถานที่
9. ระบบต้องรองรับการให้ผู้ใช้งานระบุวงจรเวลาการท่องเที่ยวประจำวัน (Travel
Daily Rhythm) ตามความต้องการรายบุคคล โดยแบ่งโครงสร้างเวลาออกเป็น 4
ช่วงหลัก ได้แก่ ช่วงเช้า ช่วงกลางวัน ช่วงเย็น และช่วงกลางคืน เพื่อนำไปใช้วิเคราะห์
พฤติกรรมการเดินทาง
10. ระบบต้องบันทึกหมวดหมู่ความสนใจและช่วงเวลาที่ถูกเลือกไว้เพื่อสร้างเวกเตอร์หลายตัว
เลือก ส่วนงบประมาณและจำนวนสถานที่ต่อวันเก็บเป็นค่าตัวเลข แล้วใช้ข้อมูลทั้ง 4 ปัจจัย
เปรียบเทียบผู้ใช้งานกับทริป

ระบบค้นหาและสร้างทริป (Trip Search & Creation)

11. ระบบต้องแสดงรายการทริปพร้อม **Match Score** ที่คำนวณจากข้อมูลผู้ใช้และทริป และให้ผู้ใช้เปิดดูรายละเอียดก่อนตัดสินใจเข้าร่วม
12. ระบบต้องคำนวณและแสดงค่าคะแนนความเหมาะสมเป็นเปอร์เซ็นต์ (**Match Score %**) บนบัตรข้อมูลทริปแต่ละใบ เพื่อให้ผู้ใช้ประเมินระดับความเข้ากันได้และประกอบการตัดสินใจเข้าร่วมทริปได้ทันที
13. ระบบต้องจัดวางปุ่มคำสั่ง **Create Trip** ไว้บนหน้าฟีด เพื่อเป็นช่องทางให้ผู้ใช้สามารถกดเรียกเปิดฟอร์มสำหรับตั้งค่าและกรอกข้อมูลการสร้างแผนการเดินทางใหม่ได้อย่างสะดวกรวดเร็ว
14. ระบบต้องอนุญาตให้ผู้ใช้สร้างทริปใหม่ โดยระบุสถานที่ วันเดินทาง งบประมาณ จำนวนสมาชิก หมวดหมู่ ช่วงเวลา และแผนการเดินทาง ระบบคำนวณจำนวนสถานที่เฉลี่ยต่อวันจากแผนการเดินทางเพื่อใช้ใน **Smart Matching**
15. ระบบต้องบูรณาการร่วมกับ **Gemini API** เพื่อประมวลผลและสร้างแผนกำหนดการเดินทางโดยละเอียดแบบอัตโนมัติ (**AI Itinerary**) ให้สอดคล้องตามเงื่อนไขและข้อจำกัดที่ผู้ใช้กำหนดในขั้นตอนการสร้างทริป
16. ระบบต้องมีฟังก์ชันบันทึกทริปที่สนใจ (**Bookmark**) เพื่อเก็บไว้เรียกดูในภายหลัง พร้อมทั้งมีระบบตัวกรอง (**Filter**) ที่ช่วยให้ผู้ใช้สามารถจำกัดขอบเขตการค้นหาทริปตามช่วงงบประมาณหรือประเภทกิจกรรมได้อย่างแม่นยำ

ระบบเข้าร่วมและจัดการกลุ่ม (Join & Group Management)

17. ระบบต้องจัดวางปุ่มคำสั่ง เข้าร่วมทริป (**Join Trip**) บนหน้ารายละเอียดทริป เพื่อให้ผู้ใช้สามารถเข้าร่วมกลุ่มเดินทางได้ทันทีโดยไม่ต้องผ่านกระบวนการอนุมัติจากเจ้าของทริป
18. เมื่อกระบวนการเข้าร่วมเสร็จสิ้น ระบบต้องนำผู้ใช้เข้าสู่ห้องแชทกลุ่มของทริปนั้นโดยอัตโนมัติ พร้อมทั้งส่งสัญญาณแจ้งเตือนการเข้าร่วมของสมาชิกใหม่ให้สมาชิกเดิมในกลุ่มรับทราบ
19. ระบบต้องแสดงรายชื่อสมาชิกทั้งหมดในทริป โดยกำหนดสิทธิ์ให้เจ้าของทริป (**Host**) สามารถบริหารจัดการสมาชิก เช่น การลบสมาชิกออกจากกลุ่มเดินทาง (**Remove Member**) รวมถึงสิทธิ์ในการแก้ไขปรับปรุงรายละเอียดของทริปได้
20. ระบบต้องมีฟังก์ชัน ออกจากทริป (**Leave Trip**) เพื่อรองรับกรณีที่สมาชิกทั่วไปต้องการยกเลิกการเข้าร่วมกลุ่มเดินทางด้วยตนเอง

ระบบการสื่อสารและแจ้งเตือน (Communication & Notification)

21. ระบบต้องจัดเตรียมห้องแชทกลุ่มแบบเวลาจริง (Real-time) สำหรับทริปที่ได้รับ การยืนยัน โดยรองรับการส่งข้อความตัวอักษร (Text Message) และไฟล์รูปภาพ (Image)
22. ระบบต้องแสดงชื่อและรูปภาพประวัติส่วนตัว (Profile Picture) ของผู้ส่ง ข้อความกำกับไว้ทุกครั้ง และต้องจัดวางปุ่มคำสั่ง ย้อนกลับ (Back) บนหน้าจอแชท เพื่อให้ผู้ ใช้สามารถกลับสู่หน้ารายละเอียดทริปหรือหน้าฟีดหลักได้
23. ระบบส่งสัญญาณและแสดงผลการแจ้งเตือน (Notification System) ระบบ ต้องรองรับการส่งการแจ้งเตือนแบบผลัก (Push Notification) ไปยังอุปกรณ์ของผู้ใช้ ทันทีเมื่อมีข้อมูลเข้าใหม่ เช่น ข้อความแชท คำขอเข้าร่วมกลุ่ม หรือการอัปเดตสถานะของทริป
24. ระบบต้องมีหน้าจอศูนย์รวมการแจ้งเตือน (Notifications Feed) เพื่อแสดง รายการความเคลื่อนไหวทั้งหมดพร้อมระบุวันและเวลาที่เกิดขึ้นอย่างชัดเจน

ระบบการประเมินผลและความน่าเชื่อถือ (Review & Rating System)

25. การให้คะแนนและเขียนคำวิจารณ์ (Review & Rating) ระหว่างสมาชิกหลังจบ ทริปเป็นข้อเสนอสำหรับการพัฒนาต่อไป ระบบรุ่นที่อธิบายในรายงานนี้ยังไม่มีฟังก์ชันดังกล่าว

3.1.2 ความต้องการเชิงไม่ใช่ฟังก์ชัน (Non-Functional Requirements)

ประสิทธิภาพ (Performance)

1. ระบบควรแสดงสถานะกำลังโหลดและผลลัพธ์ของหน้า Feed อย่างชัดเจน โดย ประสิทธิภาพจริงต้องประเมินจากการวัดบนอุปกรณ์และเครือข่ายที่ระบุ
2. การคำนวณคะแนนและการตอบกลับ API ควรทำงานโดยไม่ทำให้หน้าจอค้าง และบันทึก เวลาในการทดสอบจริง
3. ระบบสนทนาควรแสดงสถานะการส่งและข้อผิดพลาดเมื่อส่งข้อความไม่สำเร็จ
4. การรองรับผู้ใช้พร้อมกันต้องประเมินด้วยการทดสอบภาระงานก่อนระบุจำนวนผู้ใช้ที่ รองรับได้

ความน่าเชื่อถือและเสถียรภาพ (Reliability)

5. ระบบควรจัดการความผิดพลาดจากบริการออนไลน์และบันทึกเหตุการณ์ที่จำเป็นต่อการ ตรวจสอบ

6. ระบบต้องมีกลไกจัดการข้อผิดพลาดเมื่อขาดการเชื่อมต่ออินเทอร์เน็ต โดยแสดงผล

Offline Notification แทนการค้างของหน้าจอ

7. ระบบควรรักษาความถูกต้องของข้อมูลผู้ใช้และทริป และกำหนดวิธีสำรองข้อมูลตามการตั้งค่าของบริการฐานข้อมูลที่ใช้งานจริง

ความปลอดภัย (Security)

8. ข้อมูลรหัสผ่านต้องถูกเข้ารหัสด้วย Hash Algorithm (bcrypt) ก่อนบันทึกลงฐานข้อมูล

9. API ที่ต้องเข้าสู่ระบบต้องตรวจสอบ JWT (JSON Web Token) เพื่อยืนยันสิทธิ์ ส่วน API สาธารณะ เช่น สมัครสมาชิก เข้าสู่ระบบ และอ่านข้อมูลทริปบางรายการ ไม่บังคับให้แนบ Token

10. เมื่อเผยแพร่ระบบใช้งานจริง ต้องกำหนด HTTPS/TLS สำหรับการรับส่งข้อมูลและตรวจสอบการตั้งค่าที่จุดให้บริการ API เพื่อป้องกันการดักจับข้อมูลระหว่างทาง

11. ระบบขออนุญาตใช้กล้องเฉพาะเมื่อผู้ใช้เข้าสู่ขั้นตอนตรวจการเคลื่อนไหวของใบหน้า และต้องอธิบายวัตถุประสงค์การใช้งานให้ชัดเจน

ความสามารถในการใช้งาน (Usability)

12. แอปพลิเคชันหลักออกแบบสำหรับ iOS ตามแนวทาง Apple Human Interface Guidelines ส่วน Backoffice ใช้งานผ่านเว็บเบราว์เซอร์

13. องค์ประกอบควบคุมและปุ่มกดต้องมีขนาดอย่างน้อย 44x44 pt เพื่อความสะดวกในการใช้งานผ่านหน้าจอสัมผัส

14. ประเมินความง่ายในการใช้งานด้วยแบบประเมิน SUS เมื่อมีผู้ทดสอบจริง และรายงานคะแนนตามข้อมูลที่เก็บได้

ความสามารถในการบำรุงรักษาและขยายตัว (Maintainability & Scalability)

15. แยกส่วนติดต่อผู้ใช้ iOS ที่พัฒนาด้วย SwiftUI ออกจาก Backoffice ที่พัฒนาด้วย React เพื่อให้บำรุงรักษาแต่ละส่วนได้ตามเทคโนโลยีที่ใช้จริง

16. ระบบควรบันทึกข้อผิดพลาดที่จำเป็นต่อการวิเคราะห์ปัญหา โดยหลีกเลี่ยงการบันทึกข้อมูลส่วนบุคคลเกินความจำเป็น

สถาปัตยกรรมฐานข้อมูลต้องแยกข้อมูลผู้ใช้ ทริป สมาชิกทริป ข้อความ การแจ้งเตือน และคำขอยืนยันตัวตนอย่างเป็นสัดส่วน พร้อมกำหนดความสัมพันธ์และดัชนีที่จำเป็นเพื่อให้อ่านและเรียกข้อมูลได้อย่างเหมาะสม

3.1.3 รายการปุ่มและองค์ประกอบควบคุมในแอปพลิเคชัน (Application Buttons & Controls)

ปุ่ม / ควบคุม	หน้าที่พบ	หน้าที่ของปุ่ม
สมัครสมาชิก (Sign Up)	หน้าเริ่มต้น	นำผู้ใช้ไปยังหน้ากรอกข้อมูลสมัครสมาชิก
เข้าสู่ระบบ (Login)	หน้าเริ่มต้น	ส่งข้อมูลอีเมล-รหัสผ่านเพื่อตรวจสอบสิทธิ์
ลืมรหัสผ่าน?	หน้า Login	เปิดฟอร์มส่งอีเมล Reset รหัสผ่าน
Create Trip / สร้างทริป	หน้า Feed / Home	เปิดฟอร์มให้ผู้ใช้กรอกรายละเอียดทริปใหม่
ค้นหา (Search)	หน้า Feed	เปิดช่องค้นหาทริปตามคำสำคัญหรือสถานที่
กรองผลลัพธ์ (Filter)	หน้า Feed	เปิดแถบตัวกรองจำกัดทริปตามงบหรือประเภท
บันทึก / Bookmark	การ์ดทริปใน Feed	บันทึกทริปที่สนใจไว้ในรายการโปรด
การ์ดทริป (Trip Card)	หน้า Feed	กดเปิดหน้ารายละเอียดเต็มของทริปนั้น
เข้าร่วมทริป (Join Trip)	หน้ารายละเอียดทริป	เข้าร่วมทริปและเข้าสู่ห้องแชทกลุ่มได้ทันที
นำออกจากทริป (Remove Member)	หน้าจัดการทริปของ Host	Host นำสมาชิกออกจากทริปได้หากจำเป็น
ออกจากทริป (Leave)	หน้ารายละเอียดทริป	ยกเลิกการเป็นสมาชิกในทริปนั้น
แก้ไขทริป (Edit Trip)	หน้าทริปของ Host	เปิดฟอร์มแก้ไขรายละเอียดทริปที่สร้างไว้
ลบทริป (Delete Trip)	หน้าทริปของ Host	ลบทริปออกจากระบบพร้อมแจ้งสมาชิก
แชต (Chat)	หน้ารายละเอียดทริป	เข้าสู่ห้องสนทนากลุ่มของทริปนั้น

ปุ่ม / ควบคุม	หน้าที่พบ	หน้าที่ของปุ่ม
ส่งข้อความ (Send)	หน้า Chat	ส่งข้อความที่พิมพ์ไปยังห้องแชตกลุ่ม
แนบรูปภาพ (Attach)	หน้า Chat	เปิดคลังรูปภาพเพื่อเลือกรูปส่งในแชต
ย้อนกลับ (Back)	ทุกหน้าย่อย	กลับไปยังหน้าก่อนหน้าตาม Navigation Stack
แถบเมนูหลัก (Bottom Nav Bar)	ทุกหน้าหลัก	สลับระหว่าง: Feed / Notifications / Profile
แจ้งเตือน (Notifications)	Bottom Nav Bar	ไปยังรายการแจ้งเตือนทั้งหมด
แก้ไขโปรไฟล์ (Edit Profile)	หน้า Profile	เปิดฟอร์มให้แก้ไขชื่อ รูป และความสนใจ
ออกจากระบบ (Logout)	หน้า Profile	ล้าง Session และกลับสู่หน้า Login

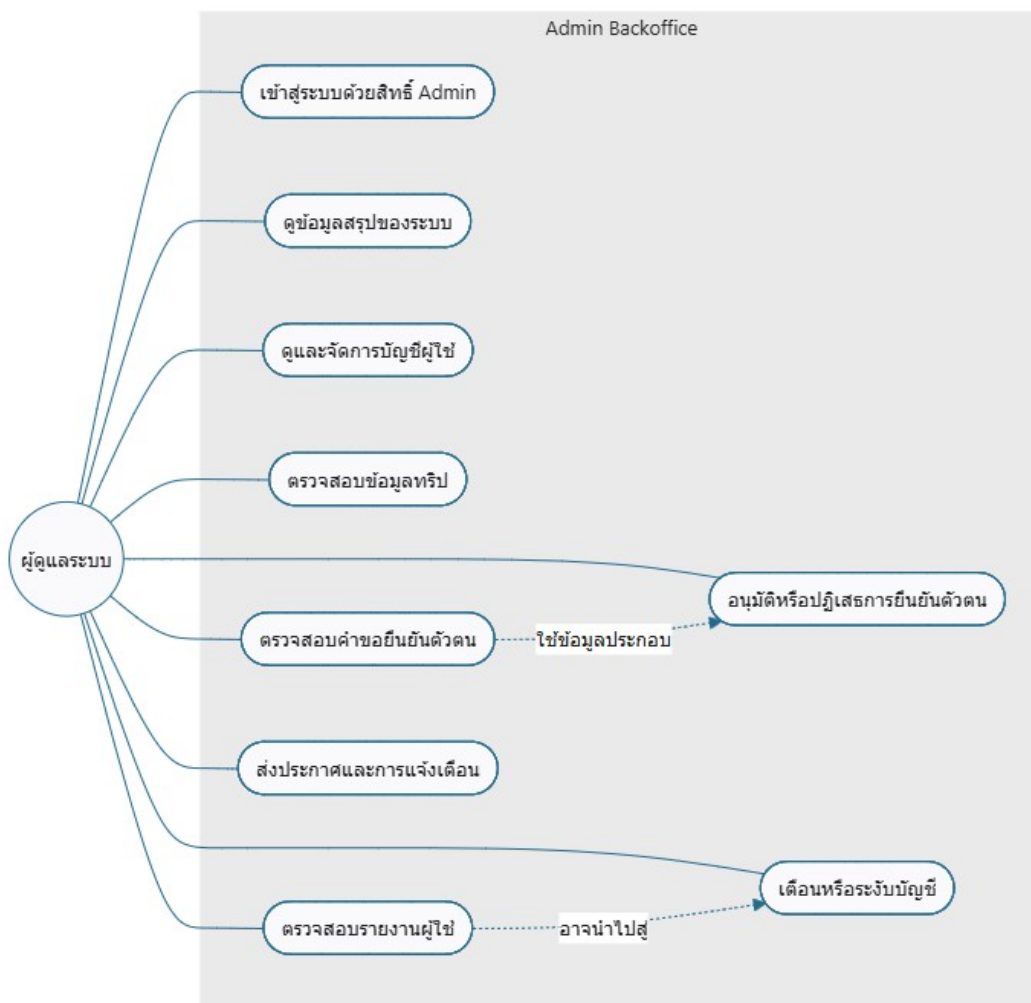
3.1.4 รายละเอียดข้อมูลที่ใช้ในระบบ Smart Matching

ระบบ Smart Matching เปรียบเทียบข้อมูลผู้ใช้งานกับทริป 4 ปัจจัย ได้แก่ หมวดหมู่ความสนใจ งบประมาณ จำนวนสถานที่ต่อวัน และช่วงเวลาี่สะดวก โดยคำนวณคะแนนย่อยตามชนิดข้อมูล และหาค่าเฉลี่ยของคะแนนที่มีข้อมูล

3.1.4.1 งบประมาณต่อทริป (Budget): ผู้ใช้กำหนดจำนวนเงินที่สะดวกจ่ายผ่านแถบเลื่อน ตั้งแต่ 100–5,000 บาทขึ้นไป ระบบเก็บเป็นค่าจำนวนเงินจริงและนำไปเปรียบเทียบกับ budget ของทริปโดยตรง

3.1.4.2 ช่วงเวลาที่สะดวก (Time of Day): ผู้ใช้เลือกได้มากกว่าหนึ่งช่วงจาก morning, noon, evening และ night ระบบแปลงรายการที่เลือกเป็น Multi-hot Vector 4 ตำแหน่งเพื่อคำนวณ Cosine Similarity กับช่วงเวลาของทริป

3.1.4.3 คะแนนความเข้ากันได้ (Match Score): ระบบคำนวณคะแนนย่อย Category, Budget, Activity Style และ Time of Day ในช่วง 0–100 จากนั้นหาค่าเฉลี่ยเลขคณิตของเฉพาะปัจจัยที่มีข้อมูล ผลลัพธ์ใช้เรียงทริปจากคะแนนสูงไปต่ำ



3.2 ขั้นตอนการดำเนินงานของระบบ

3.2.1 เทคโนโลยีสำหรับการพัฒนาแอปพลิเคชัน iOS และระบบ Backoffice

3.2.1.1 ภาษา Swift และ Framework SwiftUI

ใช้เป็นเทคโนโลยีหลักในการพัฒนาแอปพลิเคชัน Go With Us บนระบบปฏิบัติการ iOS โดย Swift ใช้สำหรับพัฒนาตรรกะการทำงานของระบบและการจัดการข้อมูล ส่วน SwiftUI ใช้สำหรับสร้างส่วนติดต่อผู้ใช้งาน (User Interface) ในรูปแบบ Declarative UI ซึ่งช่วยให้สามารถจัดการโครงสร้างหน้าจอและสถานะของแอปพลิเคชันได้อย่างเป็นระบบ

3.2.1.2 MapKit

ใช้สำหรับการจัดการและแสดงผลข้อมูลแผนที่ภายในแอปพลิเคชัน iOS เช่น ตำแหน่งสถานที่ จุดหมายปลายทาง และข้อมูลที่เกี่ยวข้องกับตำแหน่งทางภูมิศาสตร์ เพื่อสนับสนุนการวางแผนและการเดินทางของผู้ใช้งาน

3.2.1.3 ภาษา TypeScript

ใช้สำหรับพัฒนาระบบ **Backoffice Web** โดยช่วยกำหนดชนิดข้อมูลและโครงสร้างข้อมูลให้มีความชัดเจน รวมถึงใช้ในการจัดการตรรกะและการรับส่งข้อมูลระหว่างส่วนติดต่อผู้ใช้งานกับ **Backend**

3.2.1.4 โลบารี React.js และเครื่องมือ Vite

React.js ใช้สำหรับสร้างส่วนติดต่อผู้ใช้งานของระบบ **Backoffice** ในรูปแบบ **Component-Based Architecture** ซึ่งสามารถแบ่งหน้าจอออกเป็น ส่วนประกอบย่อยและนำกลับมาใช้ซ้ำได้ ส่วน **Vite** ใช้สำหรับจัดการสภาพแวดล้อมในการพัฒนาและการ **Build** ระบบ **Web Application** เพื่อเพิ่มความรวดเร็วในการพัฒนาและทดสอบระบบ

3.2.1.5 Tailwind CSS

ใช้สำหรับจัดรูปแบบและตกแต่งส่วนติดต่อผู้ใช้งานของระบบ **Backoffice Web** โดยใช้แนวคิด **Utility-First CSS** เพื่อให้สามารถควบคุมรูปแบบขององค์ประกอบต่าง ๆ ได้อย่างเป็นระบบและมีความสอดคล้องกัน

3.2.2 การเชื่อมต่อระหว่างแอปพลิเคชันกับ Backend

ระบบ **Go With Us** ออกแบบในรูปแบบ **Client-Server Architecture** โดยแอปพลิเคชัน **Native iOS** และระบบ **Backoffice Web** ทำหน้าที่เป็นส่วน **Client** ซึ่งเชื่อมต่อกับ **Backend** ผ่าน **API** เพื่อรับส่งข้อมูลและเรียกใช้บริการต่าง ๆ ของระบบ โดยมีรายละเอียดดังนี้

3.2.2.1 ผังแอปพลิเคชัน iOS

พัฒนาด้วยภาษา **Swift** และ **SwiftUI** โดยใช้ **Xcode** เป็นเครื่องมือหลักในการพัฒนาและทดสอบแอปพลิเคชัน สามารถเชื่อมต่อกับ **Backend API** เพื่อรับส่งข้อมูลผู้ใช้งาน ข้อมูลทริป ข้อมูลการจับคู่ และข้อมูลอื่น ๆ ที่เกี่ยวข้องกับระบบ นอกจากนี้ยังสามารถเรียกใช้งานความสามารถของอุปกรณ์ **iOS** เช่น กล้องและข้อมูลตำแหน่งผ่าน **Framework** ที่เกี่ยวข้องของระบบปฏิบัติการ

3.2.2.2 ผังระบบ Backoffice Web

พัฒนาด้วย **TypeScript, React.js, Vite** และ **Tailwind CSS** เพื่อให้ผู้ดูแลระบบสามารถจัดการและตรวจสอบข้อมูลต่าง ๆ ของระบบ โดยเชื่อมต่อกับ **Backend API** สำหรับรับส่งข้อมูลกับฐานข้อมูลและบริการต่าง ๆ ของระบบ

3.2.2.3 ผัง Backend

พัฒนาด้วย **Node.js** เพื่อทำหน้าที่ประมวลผลตรรกะของระบบ จัดการ **API** ตรวจสอบ

สอบสิทธิ์ผู้ใช้งาน ประมวลผลข้อมูล และเชื่อมต่อกับฐานข้อมูล PostgreSQL บน Supabase รวมถึงเชื่อมต่อกับบริการภายนอกที่ระบบนำมาใช้งาน

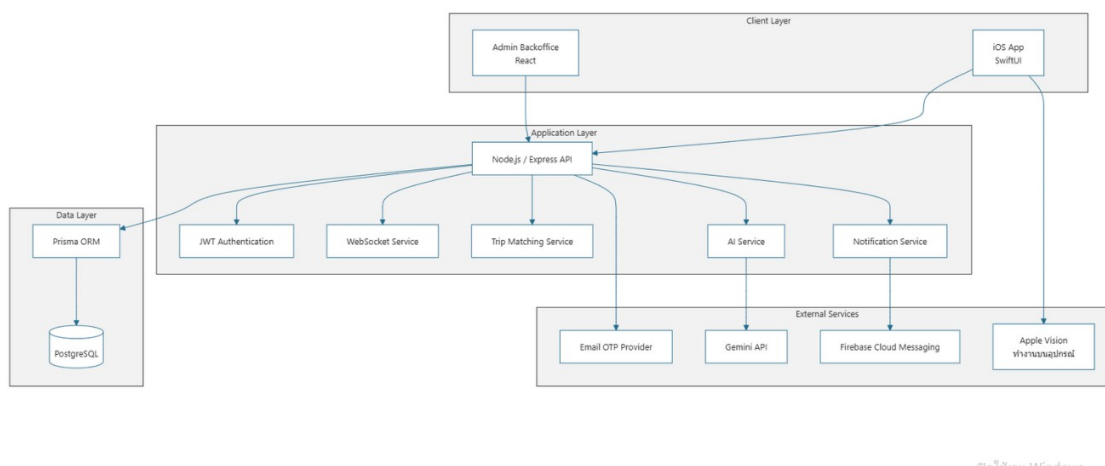
3.3 ขั้นตอนการดำเนินงานของระบบ

3.3.1 การวิเคราะห์ปัญหาและความต้องการ (Problem Analysis)

ระบบทั่วไปมักแสดงทริปจากข้อมูลพื้นฐานเพียงอย่างเดียว ทำให้ผู้ใช้ต้องตรวจสอบเองว่าทริปเหมาะกับงบประมาณ จำนวนกิจกรรม ความสนใจ และเวลาที่สะดวกหรือไม่ ระบบ Go With Us จึงกำหนดให้มี Smart Matching แบบ User-to-Trip ที่คำนวณคะแนนย่อยจากข้อมูล 4 ด้านและแสดงเป็น Match Score เพื่อช่วยประกอบการตัดสินใจ โดยยังคงให้ผู้ใช้เปิดดูรายละเอียดทริปก่อนเข้าร่วม

3.3.2 การออกแบบระบบ (System Design)

ระบบออกแบบเป็น Client-Server Architecture ประกอบด้วยแอปพลิเคชัน iOS ระบบ Backoffice Backend API และฐานข้อมูล PostgreSQL บน Supabase แอป iOS และ Backoffice ไม่เชื่อมต่อฐานข้อมูลโดยตรง แต่ส่งคำขอผ่าน Backend ซึ่งตรวจสอบ JWT ประมวลผลกฎธุรกิจ เรียกใช้ Prisma และส่งผลลัพธ์กลับเป็น JSON โครงสร้างนี้ครอบคลุมบัญชีผู้ใช้ แบบสอบถาม ทริป สมาชิกทริป Smart Matching แชนแนล การแจ้งเตือน รายงาน และคำขอยืนยันตัวตน



3.3.3 แผนผังสถาปัตยกรรมระบบ (System Architecture Diagram)

3.3.4 การพัฒนาระบบ (System Development)

แอปพลิเคชัน iOS พัฒนาด้วย Swift และ SwiftUI ระบบ Backoffice พัฒนาด้วย TypeScript, React, Vite และ Tailwind CSS ส่วน Backend

พัฒนาด้วย Node.js และ Express เชื่อมต่อ PostgreSQL ผ่าน Prisma การคำนวณ Smart Matching ทำที่ Backend โดยใช้ Cosine Similarity กับหมวดหมู่และช่วงเวลา ใช้อัตราส่วนกับงบประมาณ ใช้ระยะห่างบนช่วง 1-10 กับกิจกรรม แล้วหาค่าเฉลี่ยคะแนนย่อยที่มีข้อมูล

3.3.5 การทดสอบระบบ (System Testing)

กำหนดการทดสอบ 4 ระดับ ได้แก่ Unit Testing สำหรับสูตรคำนวณ, Integration Testing สำหรับ App-API-Database, System Testing แบบ Black Box สำหรับลำดับการใช้งาน และ User Acceptance Testing สำหรับการยอมรับของผู้ใช้ การรายงานผลต้องอ้างอิงหลักฐานการรันทดสอบจริง และไม่ใส่ค่าผ่านหรือคะแนนประเมินแทนข้อมูลที่ยังไม่ได้เก็บ

3.3.6 การประเมินผล (Evaluation)

ประเมินความถูกต้องของคะแนนย่อยและ Match Score ด้วยกรณีข้อมูลตรงกัน แตกต่างบางส่วน และแตกต่างทั้งหมด ตรวจสอบการคำนวณจำนวนกิจกรรมเฉลี่ยจาก itinerary การเลือกหมวดหมู่ทริปสูงสุด 3 รายการ การละเว้นปัจจัยที่ไม่มีข้อมูล และการเรียงคะแนนจากมากไปน้อย รวมทั้งวัด Response Time และประเมิน SUS เมื่อมีข้อมูลจริง

3.4 การวิเคราะห์และออกแบบระบบส่วนติดต่อผู้ใช้งาน

3.4.1 หน้าจอหลักและการควบคุมทริป

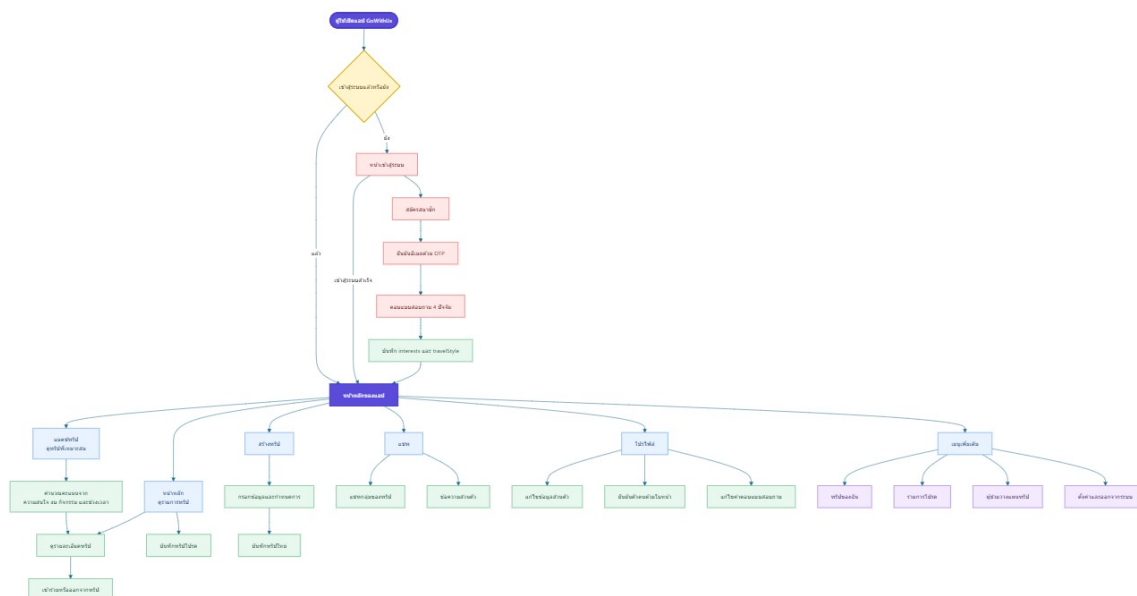
หลังเข้าสู่ระบบ ผู้ใช้จะเห็นรายการทริปพร้อมข้อมูลสำคัญและ Match Score สามารถค้นหา เปิดรายละเอียด แสดงความสนใจ ขอเข้าร่วม หรือสร้างทริปใหม่ได้ การสร้างทริปแบ่งเป็นข้อมูลทั่วไป วันที่ สถานที่ งบประมาณ จำนวนสมาชิก category ระดับกิจกรรม ช่วงเวลา และแผนการเดินทาง

3.4.2 การควบคุมสถานะของหน้าจอ (Application State)

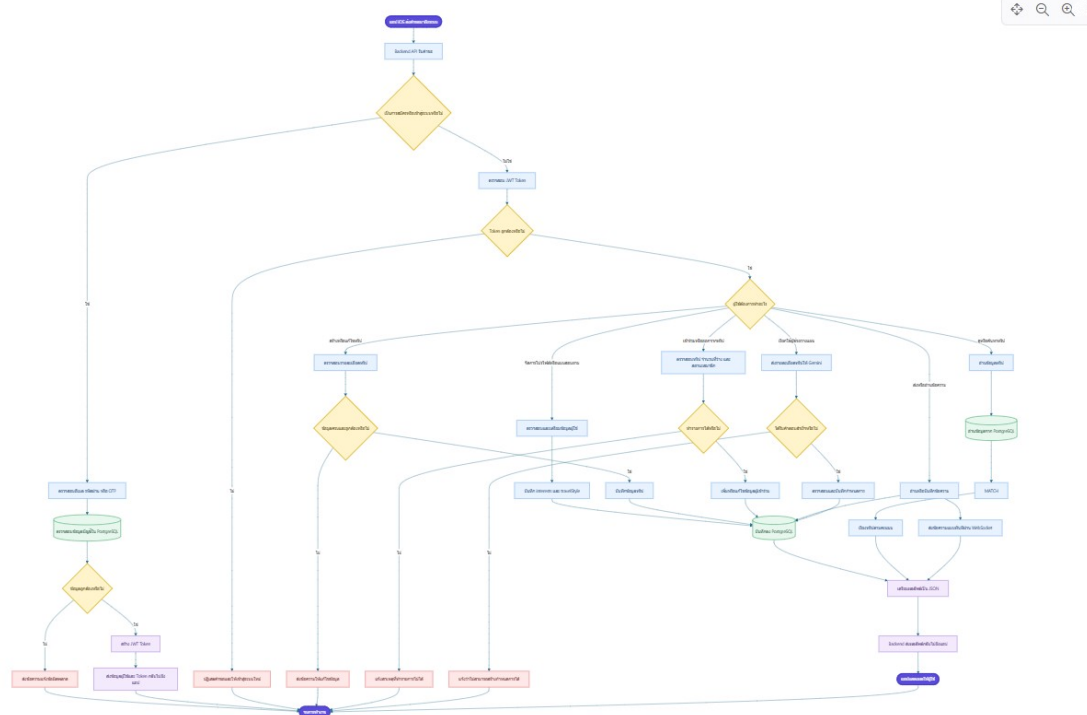
แอปจัดการสถานะการเข้าสู่ระบบ ข้อมูลผู้ใช้ แบบสอบถาม รายการทริป รายละเอียดทริป และผลการส่งคำขอ เมื่อข้อมูลกำลังโหลดจะแสดงสถานะรอ เมื่อเกิดข้อผิดพลาดจะแสดงข้อความที่ผู้ใช้เข้าใจได้ และเมื่อบันทึกสำเร็จจะปรับข้อมูลบนหน้าจอให้ตรงกับผลจาก Backend

3.4.3 เงื่อนไขการเปลี่ยนหน้าจอ (Navigation Flow)

ผู้ใช้ใหม่ดำเนินการตามลำดับ สมัครสมาชิก; ยืนยัน OTP; ตอบแบบสอบถาม 4 ส่วน; เข้าหน้าหลัก ส่วนการยืนยันตัวตนด้วยเอกสารหรือใบหน้าอยู่ในหน้าโปรไฟล์และทำภายหลังได้ สำหรับการสร้างทริป เมื่อ Backend บันทึกสำเร็จ แอปจะเปิดหน้ารายละเอียดทริปที่สร้างขึ้น



3.4.4 แผนผังการทำงานของผู้ใช้งาน (User Activity Diagram)



3.4.5 แผนผังการทำงานของระบบ (System Activity Diagram)

3.5 การออกแบบกระบวนการวิเคราะห์และแนะนำทริป

ระบบวิเคราะห์ความเหมาะสมระหว่างผู้ไปกับทริปจาก **category**, **budget**, **activityStyle** และ **timeOfDay** โดยสร้างเวกเตอร์เฉพาะ **category** และ **timeOfDay** ส่วน **budget** กับ **activityStyle** ใช้ค่าตัวเลขโดยตรง คะแนนรวมเป็นค่าเฉลี่ยของคะแนนย่อยที่มีข้อมูล

ข้อมูลผู้ใช้งานจากฟิลด์ **interests** และ **travelStyle** ส่วนข้อมูลทริปมาจากฟิลด์ **category**, **budget**, **activityStyle** และ **timeOfDay** Backend ปรับชื่อและรูปแบบค่าให้เป็นมาตรฐานก่อนคำนวณ เพื่อให้ข้อมูลจากแอปและฐานข้อมูลเปรียบเทียบกันได้

ผลลัพธ์ประกอบด้วย **matchScore** และ **matchBreakdown** ได้แก่ **category**, **budget**, **activityStyle** และ **timeOfDay** ระบบคำนวณคะแนนย่อยตามชนิดข้อมูลแล้วหาค่าเฉลี่ยของคะแนนที่มีข้อมูล

3.5.1 การเตรียมข้อมูลและสกัดคุณลักษณะ (Feature Preparation)

3.5.1.1 ข้อมูลนำเข้าจากผู้ใช้งานและทริป

แบบสอบถามผู้ใช้งานมี 4 ส่วนและบันทึกค่าที่นำไปใช้คำนวณดังนี้

1. งบประมาณต่อทริป (**Budget**): เก็บเป็นจำนวนเงินบาท เช่น 2,000 ระบบเปรียบเทียบกับราคาทริปโดยตรง ไม่แปลงเป็นเวกเตอร์
2. จำนวนสถานที่ต่อวัน (**Activity Style**): ผู้ใช้เลือกค่าจริง 1–10 แห่งต่อวัน ค่าเริ่มต้นคือ 3 ส่วนทริปคำนวณจากจำนวนกิจกรรมทั้งหมดใน **itinerary** ÷ จำนวนวัน แล้วปัดเป็นจำนวนเต็มอย่างน้อย 1
3. ช่วงเวลาที่สะดวก (**Time of Day**): เลือกได้หลายค่า ได้แก่ **morning**, **noon**, **evening** และ **night** แล้วแปลงเป็น **Multi-hot Vector** 4 ตำแหน่ง
4. หมวดหมู่ความสนใจ (**Interests**): ผู้ใช้เลือกได้ไม่เกิน 5 จาก 13 หมวด ทริปมี **category** หลักและ **interestTags** เพิ่มเติมไม่เกิน 2 รายการ รวมสูงสุด 3 หมวด ทั้งสองฝั่งถูกแปลงเป็น **Multi-hot Vector** 13 ตำแหน่ง

3.5.1.2 โครงสร้างข้อมูลในฐานข้อมูลและ JSON

ข้อมูลแบบสอบถามไม่ได้แยกเป็นคอลัมน์ทุกคำตอบ แต่เก็บร่วมกับข้อมูลผู้ใช้งานเป็นข้อมูลที่มีโครงสร้าง โดยแบ่งเป็น **interests** และ **travelStyle** เพื่อให้ Backend อ่านค่ารายปัจจัยได้

1. **interests** เก็บรายการหมวดหมู่ที่ผู้ใช้เลือก เช่น ["กาแฟ", "อาหาร", "ถ่ายรูป"] และใช้สร้างเวกเตอร์หมวดหมู่ 13 ตำแหน่ง
2. **travelStyle** เก็บ JSON เช่น {"budget": 2000, "activityStyle": 3, "timeOfDay": ["morning", "evening"]} หมายถึงงบ 2,000 บาทต่อทริป ต้องการเที่ยว 3 แห่งต่อวัน และสะดวกช่วงเช้ากับเย็น
3. ข้อมูลทริปที่จับคู่ประกอบด้วย **budget**, **activityStyle**, **timeOfDay**, **category** และ **interestTags** โดย **activityStyle** คำนวณจาก **itinerary** ตอนสร้างหรือแก้ไขทริป หากทริปไม่มี **activityStyle** หรือ **timeOfDay** ระบบใช้ข้อมูลสไลด์ของผู้สร้างทริปแทนเมื่อมีข้อมูล หากยังไม่มีข้อมูลที่ใช้ได้ ระบบเว้นคะแนนปัจจัยนั้นและหารเฉลี่ยจากจำนวนคะแนนที่คำนวณได้จริง
4. ระบบสร้างเวกเตอร์สำหรับคำนวณแยกกัน 2 ชุด ได้แก่ เวกเตอร์หมวดหมู่ 13 ตำแหน่ง และเวกเตอร์ช่วงเวลา 4 ตำแหน่ง (**morning**, **noon**, **evening**, **night**) โดยแต่ละชุดใช้ **Cosine Similarity** ของตนเองและให้คะแนนย่อยคนละค่า หากปัจจัยใดไม่มีข้อมูล ระบบจะเว้นคะแนนปัจจัยนั้นและหารเฉลี่ยด้วยจำนวนคะแนนที่คำนวณได้จริง หากไม่มีคะแนนย่อยเลยให้คะแนนรวม 0% ส่วนงบประมาณและจำนวนสถานที่ต่อวันใช้สูตรคะแนนเฉพาะ ไม่สร้างเป็นเวกเตอร์

3.5.2 ขั้นตอนและสูตรคำนวณคะแนนความเข้ากันได้

3.5.2.1 การคำนวณคะแนนย่อยทั้ง 4 ปัจจัย

1) คะแนนหมวดหมู่ความสนใจ (**CategoryScore**) ใช้ **Cosine Similarity** เปรียบเทียบเวกเตอร์หมวดหมู่ของผู้ใช้ (A^c) กับทริป (B^c) จำนวน 13 ตำแหน่ง ตำแหน่งที่เลือกมีค่า 1 และตำแหน่งอื่นมีค่า 0 หมวดหมู่ของทริปได้จาก **category** รวมกับ **interestTags** แบบไม่ซ้ำกันสูงสุด 3 หมวด

$$CategoryScore = round[100 \times (A^c \cdot B^c) \div (\|A^c\| \times \|B^c\|)]$$

ตัวอย่างคะแนนหมวดหมู่: ผู้ใช้เลือก [กาแฟ, อาหาร] และทริปเลือก [กาแฟ] เมื่อแสดงเฉพาะ 2 ตำแหน่งนี้จะได้ $A^c = [1, 1]$ และ $B^c = [1, 0]$ ส่วนอีก 11 ตำแหน่งมีค่า 0 ทั้งสองฝั่ง ผลคูณเชิงจุดเท่ากับ 1 ขนาดเวกเตอร์เท่ากับ $\sqrt{2}$ และ 1 ดังนั้น $CategoryScore = round(100 \times 1 \div \sqrt{2}) = 71\%$

2) คะแนนช่วงเวลาที่ชอบทำกิจกรรม (**TimeScore**) ใช้ **Cosine Similarity** เปรียบเทียบเวกเตอร์ช่วงเวลาของผู้ใช้ (A^t) กับทริป (B^t) จำนวน 4 ตำแหน่ง ตามลำดับ [เช้า, กลางวัน, เย็น, กลางคืน] โดยช่วงที่เลือกมีค่า 1 และช่วงอื่นมีค่า 0

$$TimeScore = round[100 \times (A^t \cdot B^t) \div (\|A^t\| \times \|B^t\|)]$$

ตัวอย่างคะแนนช่วงเวลา: ผู้ใช้เลือก [เช้า, เย็น] และทริปเลือก [เช้า, เย็น] จะได้ $A^t = [1, 0, 1, 0]$ และ $B^t = [1, 0, 1, 0]$ ผลคูณเชิงจุดเท่ากับ 2 และขนาดเวกเตอร์ทั้งสองเท่ากับ $\sqrt{2}$ ดังนั้น $TimeScore = round(100 \times 2 \div (\sqrt{2} \times \sqrt{2})) = 100\%$

แต่ละคะแนนตรวจสอบข้อมูลแยกกัน หากเวกเตอร์ของผู้ใช้หรือทริปในด้านนั้นเป็นศูนย์ทุกตำแหน่ง ระบบเว้นคะแนนด้านนั้นเป็น **null** เพื่อหลีกเลี่ยงการหารด้วยศูนย์ แล้วคำนวณคะแนนด้านที่มีข้อมูลต่อไป

3) คะแนนงบประมาณ: หากทริปฟรีหรือ $UserBudget \geq TripBudget$ ได้ 100% มิฉะนั้นคำนวณตามสัดส่วนงบประมาณผู้ใช้ต้องบทripและปัดเป็นจำนวนเต็ม เช่น $1,500 \div 2,000 \times 100 = 75\%$ และ $400 \div 1,000 \times 100 = 40\%$

$$BudgetScore = round(100 \times UserBudget / TripBudget)$$

เมื่อ $0 \leq UserBudget < TripBudget$

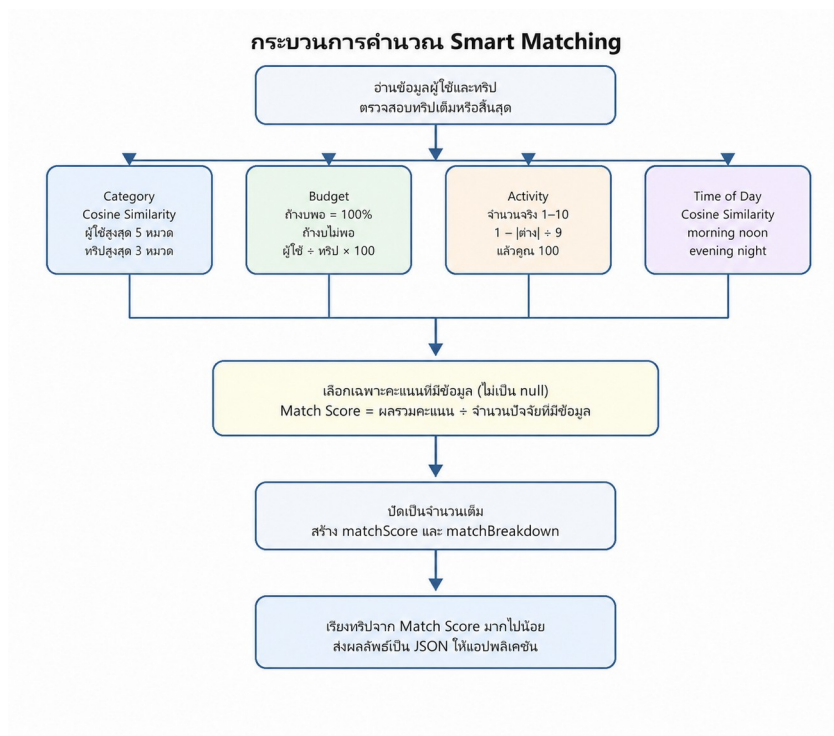
4) คะแนนกิจกรรม: $ActivityScore = round\{100 \times \max[0, 1 - |U - T| / 9]\}$ โดย U และ T คือจำนวนสถานที่ต่อวันของผู้ใช้และทริปที่จำกัดให้อยู่ในช่วง 1-10 ตัวอย่าง 3 กับ 3 ได้ 100%, 3 กับ 6 ได้ 67% และ 1 กับ 10 ได้ 0%

การคำนวณคะแนนรวม: นำคะแนนย่อยที่มีข้อมูลจริงมาหาค่าเฉลี่ยเลขคณิตแล้วปัดเป็นจำนวนเต็ม ไม่มีการถ่วงน้ำหนัก ตัวอย่าง $Category = 71, Budget = 75, Activity = 67$ และ $Time = 100$ ได้ $(71 + 75 + 67 + 100) \div 4 = 78.25$ จึงแสดง Match Score 78% หากมีข้อมูลเพียง 3 ปังจยให้หารด้วย 3

$$MatchScore = round(\sum availableScores / n), n > 0$$

3.5.3 แผนผังกระบวนการคำนวณอัลกอริทึม (Algorithm Flow Process)

ลำดับการทำงานคือ รับรหัสผู้ใช้; อ่าน interests และ travelStyle อ่านทริป ตรวจสอบทริปเต็มหรือสิ้นสุด รวม category กับ interestTags สูงสุด 3 หมวด; คำนวณ Category ด้วย Cosine คำนวณ Budget ด้วยอัตราส่วน คำนวณ Activity จากระยะห่างบนช่วง 1-10; คำนวณ Time ด้วย Cosine หาค่าเฉลี่ยของคะแนนที่มีข้อมูล ส่ง matchScore และ matchBreakdown เป็น JSON เรียงจากมากไปน้อย



3.5.4 การแปลผล Match Score

ช่วงคะแนนในตารางเป็นการจัดกลุ่มเพื่อช่วยอ่านผลบนหน้าจอ ไม่ใช่เกณฑ์ที่พิสูจน์พฤติกรรมหรือความปลอดภัยในการเดินทาง คะแนนรวมควรอ่านร่วมกับรายละเอียดคะแนนย่อยและข้อมูลทริป

ช่วงคะแนน (%)	ระดับความสอดคล้อง	คำอธิบาย
76-100	สูง	ข้อมูลผู้ใช้และทริปสอดคล้องกันมากตามปัจจัยที่คำนวณได้
51-75	ปานกลาง	ข้อมูลสอดคล้องกันบางด้าน ควรดูคะแนนย่อยประกอบ
26-50	บางส่วน	มีปัจจัยที่ตรงกันและต่างกัน ควรอ่านรายละเอียดทริปเพิ่มเติม
0-25	น้อย	ข้อมูลตามปัจจัยที่ใช้คำนวณสอดคล้องกันน้อย

บทที่ 4

ผลการดำเนินงานและผลการทดสอบระบบ

การดำเนินโครงการพัฒนาแอปพลิเคชัน Go With Us ซึ่งเป็นแอปพลิเคชันสำหรับแนะนำและจับคู่ผู้ใช้งานกับทริปการเดินทางโดยประยุกต์ใช้อัลกอริทึม Cosine Similarity คณะผู้จัดทำได้ดำเนินการพัฒนาระบบตามขอบเขตและแนวทางที่กำหนดไว้ในบทก่อนหน้า โดยระบบประกอบด้วยแอปพลิเคชัน Native iOS ระบบจัดการหลังบ้าน (Backoffice Web) และระบบ Backend ที่เชื่อมต่อกับฐานข้อมูล PostgreSQL บน Supabase

4.1 ผลการพัฒนาระบบ

จากการออกแบบและพัฒนาระบบตามสถาปัตยกรรม Client-Server คณะผู้จัดทำสามารถพัฒนาระบบออกเป็น 3 ส่วนหลัก ได้แก่ แอปพลิเคชันบนอุปกรณ์ iOS (Native iOS Application) ระบบจัดการหลังบ้าน (Backoffice Web) และระบบบริการส่วนหลัง (Backend & Database) โดยแต่ละส่วนมีหน้าที่สนับสนุนการทำงานร่วมกันของระบบ ดังนี้

4.1.1 ผลการพัฒนาแอปพลิเคชัน Native iOS

จากการพัฒนาแอปพลิเคชัน Go With Us คณะผู้จัดทำได้พัฒนาส่วนติดต่อผู้ใช้งานให้รองรับกระบวนการใช้งานตั้งแต่การสมัครสมาชิก การยืนยันตัวตน การกำหนดข้อมูลความชอบส่วนบุคคล การค้นหาและสร้างทริป ตลอดจนการติดตามสถานะและการสื่อสารระหว่างสมาชิกภายในทริป โดยออกแบบส่วนติดต่อผู้ใช้งานให้มีรูปแบบเรียบง่าย เน้นการแสดงผลข้อมูลที่จำเป็น และลดความซับซ้อนของขั้นตอนการใช้งานบนอุปกรณ์เคลื่อนที่

1.) หน้าจอสมัครสมาชิก (Registration)

หน้าสมัครสมาชิกรับชื่อ อีเมล รหัสผ่าน และยืนยันรหัสผ่าน เมื่อข้อมูลผ่านการตรวจสอบ ระบบส่ง OTP เพื่อยืนยันอีเมล หลังยืนยันสำเร็จ ผู้ใช้ตอบแบบสอบถาม 4 ส่วน ได้แก่ งบประมาณ จำนวนสถานที่ต่อวัน ช่วงเวลาและหมวดหมู่ความสนใจ แล้วจึงเข้าสู่หน้าหลัก การตรวจการเคลื่อนไหวของใบหน้าเป็นขั้นตอนในหน้าโปรไฟล์ที่ทำภายหลังได้

18:56



GoWithUs

ไปกับเรา สนุกกว่า

อีเมล

อีเมลของคุณ@email.com

รหัสผ่าน

.....

เข้าสู่ระบบ

ยังไม่มีบัญชี? [สมัครสมาชิก](#)

รูปที่ 4.1 หน้าจอเข้าสู่ระบบของแอปพลิเคชัน Go With Us

18:58

←

สมัครสมาชิก

เริ่มต้นการผจญภัยของคุณ

ชื่อ

ชื่อของคุณ

อีเมล

อีเมลของคุณ@email.com

รหัสผ่าน

อย่างน้อย 6 ตัวอักษร

ยืนยันรหัสผ่าน

พิมพ์รหัสผ่านอีกครั้ง

สมัครสมาชิก

รูปที่ 4.2 หน้าจอสมัครสมาชิกของแอปพลิเคชัน Go With Us

2.) หน้าจอระบุไลฟ์สไตล์และข้อมูลอัตลักษณ์ (User DNA Settings)

หลังยืนยัน OTP ระบบแสดงแบบสอบถามเพื่อเก็บข้อมูล 4 ปัจจัยสำหรับ Smart Matching ข้อมูลดังกล่าวเรียกรวมว่า User Preferences โดยมีเพียงหมวดหมู่และช่วงเวลาที่แปลงเป็นเวกเตอร์ ส่วนงบประมาณและจำนวนสถานที่ต่อวันเก็บเป็นค่าตัวเลขสำหรับคำนวณโดยตรง

2.1) หน้าจอกำหนดงบประมาณต่อทริป (Budget Preference Screen)

ผู้ใช้ปรับแถบเลื่อนตั้งแต่ 100–5,000 บาทขึ้นไป ระบบเก็บเป็นจำนวนเงินบาทใน `travelStyle.budget` และใช้เปรียบเทียบกับ `budget` ของทริป ไม่ได้ใช้เป็นระดับ 1–10 และไม่ใช้งบประมาณเฉลี่ยต่อวัน



รูปที่ 4.3 แบบสอบถามเรื่องงบประมาณ

2.2) หน้าจอระบุสไตล์กิจกรรมและความลุย (Activity & Adventure Style Screen)

หน้าจอแสดงตัวเลือกจำนวนสถานที่ต่อวันตั้งแต่ 1–10 แห่ง ผู้ใช้เลือกหนึ่งค่า โดยค่าเริ่มต้นคือ 3 ค่าที่เลือกถูกบันทึกใน `travelStyle.activityStyle` เป็นจำนวนเต็มและนำไปเปรียบเทียบกับจำนวนกิจกรรมเฉลี่ยต่อวันของทริป

ค่าผู้ใช้ 1 หมายถึงต้องการเที่ยวประมาณ 1 สถานที่ต่อวัน ส่วนค่า 10 หมายถึงต้องการตารางกิจกรรมหนาแน่นประมาณ 10 สถานที่ต่อวัน

ระบบแสดงแต่ละค่าเป็นการ์ด “N สถานที่ต่อวัน” พร้อมเครื่องหมายเลือกที่ชัดเจน

คะแนนกิจกรรมพิจารณาจากความต่างของจำนวนสถานที่ โดยหารด้วยความต่างสูงสุด

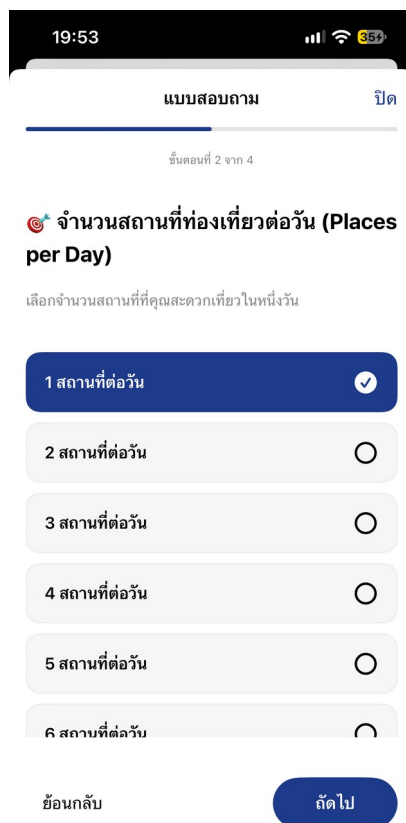
3

ตัวอย่าง ผู้ใช้เลือก 3 แห่งต่อวันและทริปมีค่าเฉลี่ย 6 กิจกรรมต่อวัน ความต่างเท่ากับ

คะแนนกิจกรรมจึงเท่ากับ $(1-3\div 9)\times 100 = 66.67$ และปัดเป็น 67%

วิธีนี้ทำให้ข้อมูลที่ใกล้เคียงกันได้คะแนนสูง และลดคะแนนตามจำนวนกิจกรรมที่แตกต่างกัน

จริง



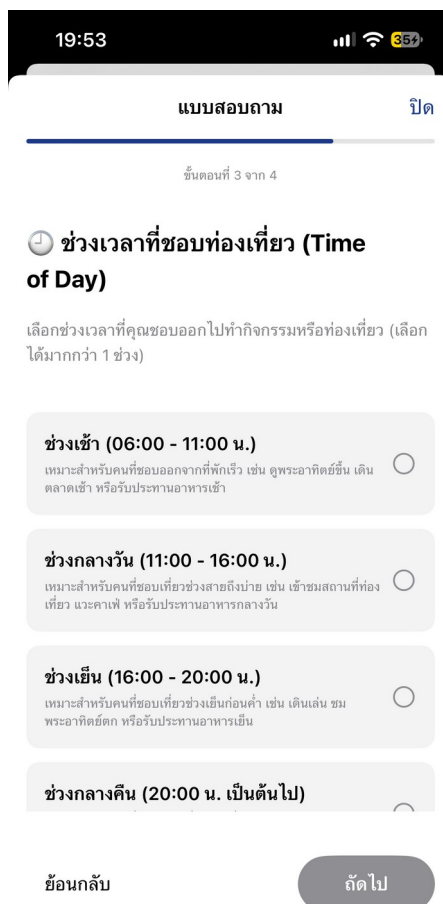
รูปที่ 4.4 แบบสอบถามสไตล์กิจกรรมที่ชื่นชอบ

2.3) หน้าจอระบุวงจรเวลาการท่องเที่ยว (Travel Daily Rhythm Screen)

ผลการแสดงหน้าจอระบบแสดงหน้าจอแบบสอบถามขั้นตอนที่ 3 เพื่อรับข้อมูลวงจรเวลาการท่องเที่ยว โดยออกแบบอินเทอร์เฟซเป็น รูปแบบปุ่มตัวเลือกที่สามารถเลือกได้มากกว่า 1 ข้อ (Multiple Selection) เพื่อรองรับผู้ที่มีไลฟ์สไตล์ยืดหยุ่น โดยแบ่งออกเป็น 4 ช่วงเวลาหลัก พร้อมระบุกรอบเวลาและตัวอย่างกิจกรรมไว้อย่างชัดเจน เพื่อให้ผู้ใช้เข้าใจตรงกัน ดังนี้

1. เช้า (06:00 - 11:00): ดูพระอาทิตย์ขึ้น, เดินตลาดเช้า
2. กลางวัน (11:00 - 16:00): หาของอร่อยทาน, นั่งกาแฟ, หลบแดดเข้าพิพิธภัณฑ
3. เย็น (16:00 - 20:00): เดินเล่นชิลๆ, ดูพระอาทิตย์ตก, ทานมื้อค่ำ
4. มีด/ราตรี (20:00 เป็นต้นไป): ท่องราตรี, ชมแสงสีเมือง, ปาร์ตี้

ระบบจัดเก็บค่าที่เลือกใน `travelStyle.timeOfDay` เป็นรายการของ morning, noon, evening และ night จากนั้นสร้าง Multi-hot Vector 4 ตำแหน่งเมื่อกำหนดคะแนนช่วงเวลา



19:53

แบบสอบถาม ปิด

ขั้นตอนที่ 3 จาก 4

🕒 **ช่วงเวลาที่ชอบท่องเที่ยว (Time of Day)**

เลือกช่วงเวลาที่คุณชอบออกไปทำกิจกรรมหรือท่องเที่ยว (เลือกได้มากกว่า 1 ช่วง)

ช่วงเช้า (06:00 - 11:00 น.)
เหมาะสำหรับคนที่ชอบออกจากที่พักเร็ว เช่น ดูพระอาทิตย์ขึ้น เดินตลาดเช้า หรือรับประทานอาหารเช้า

ช่วงกลางวัน (11:00 - 16:00 น.)
เหมาะสำหรับคนที่ชอบเที่ยวช่วงสายถึงบ่าย เช่น เข้าชมสถานที่ท่องเที่ยว และคาเฟ่ หรือรับประทานอาหารกลางวัน

ช่วงเย็น (16:00 - 20:00 น.)
เหมาะสำหรับคนที่ชอบเที่ยวช่วงเย็นก่อนค่ำ เช่น เดินเล่น ชมพระอาทิตย์ตก หรือรับประทานอาหารเย็น

ช่วงกลางคืน (20:00 น. เป็นต้นไป)

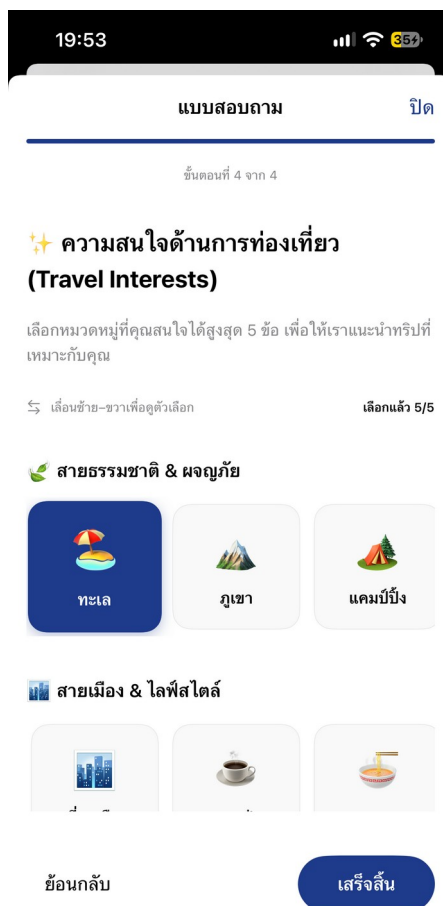
ย้อนกลับ ถัดไป

รูปที่ 4.5 แบบสอบถามเลือกช่วงเวลาที่ท่องเที่ยว

2.4) หน้าจอเลือกหมวดหมู่สิ่งที่ชอบ (Interest Category Selection Screen)

หน้าจอขั้นตอนสุดท้ายแสดงหมวดหมู่ความสนใจ 13 รายการ ได้แก่ ทะเล ภูเขา แคมป์ปิ้ง เทียวเมือง คาเฟ่ อาหาร แสงอาทิตย์ ถ่ายรูป ช้อปปิ้ง คอนเสิร์ต ผจญภัย ไหว้พระ และอื่น ๆ โดยผู้ใช้เลือกได้ไม่เกิน 5 รายการ ผู้ใช้เลือกได้หลายรายการ แต่ไม่เกิน 5 หมวด ตัวเลือกที่เลือกจะแสดงสถานะอย่างชัดเจนก่อนกดบันทึก

รายการถูกจัดเก็บใน **interests** เมื่อคำนวณ ระบบจับคู่แต่ละหมวดกับตำแหน่งในเวกเตอร์ 13 ตำแหน่ง โดยเลือก = 1 และไม่เลือก = 0 จากนั้นเปรียบเทียบกับเวกเตอร์ทริปที่รวม **category** กับ **interestTags** แบบไม่ซ้ำกันสูงสุด 3 หมวด ด้วย **Cosine Similarity**



รูปที่ 4.6 เลือกสไตล์การท่องเที่ยวของคุณ

3) หน้าจอหลักและฟีด (Home / Feed)

ผลการแสดงหน้าจอหน้าจอหลักของแอปพลิเคชันถูกออกแบบให้แสดงข้อมูลทริปการเดินทางที่มีอยู่ในระบบในรูปแบบฟีด (Feed) โดยประกอบด้วยส่วนควบคุมที่ช่วยอำนวยความสะดวกในการค้นหา ได้แก่ ส่วนค้นหาและตัวกรอง (Search & Filter): มีแถบค้นหา (Search bar) และปุ่มคัดกรองด่วน เช่น เลือกตามจังหวัด, วันที่เดินทาง, และสไตล์การท่องเที่ยว แท็บแสดงผล (Display Tabs): ผู้ใช้สามารถเลือกดูฟีดทริปได้ 3 หมวดหมู่ คือ "แนะนำ", "มาใหม่", และ "ยอดนิยม"

การ์ดข้อมูลทริป (Trip Card): ข้อมูลแต่ละทริปจะถูกแสดงผลในรูปแบบการ์ด ซึ่งประกอบด้วย ภาพหน้าปก, ชื่อทริป, สถานที่, วันที่เดินทาง, สัดส่วนจำนวนสมาชิก (เช่น 1/4), ข้อมูลผู้สร้างทริป, และป้ายกำกับ (Hashtags) เพื่อบอกเอกลักษณ์ของทริปนั้นๆ

การ์ดทริปแสดงป้าย “เข้ากันได้ X%” จากค่าเฉลี่ยคะแนน Category, Budget, Activity Style และ Time of Day เฉพาะด้านที่มีข้อมูล Backend เรียงรายการจาก Match Score มากไปน้อย



รูป 4.7 หน้าหลักของแอปหลังจากทำแบบสอบถามเสร็จสิ้น

4) หน้าจอรายละเอียดทริป (Trip Detail)

ผลการแสดงหน้าจอ: หน้าจอรายละเอียดทริปถูกออกแบบมาเพื่อให้ข้อมูลเชิงลึกเกี่ยวกับแผนการเดินทางอย่างครบถ้วน โดยแบ่งพื้นที่การแสดงผลออกเป็นส่วนต่างๆ เพื่อให้อ่านง่าย ดังนี้:

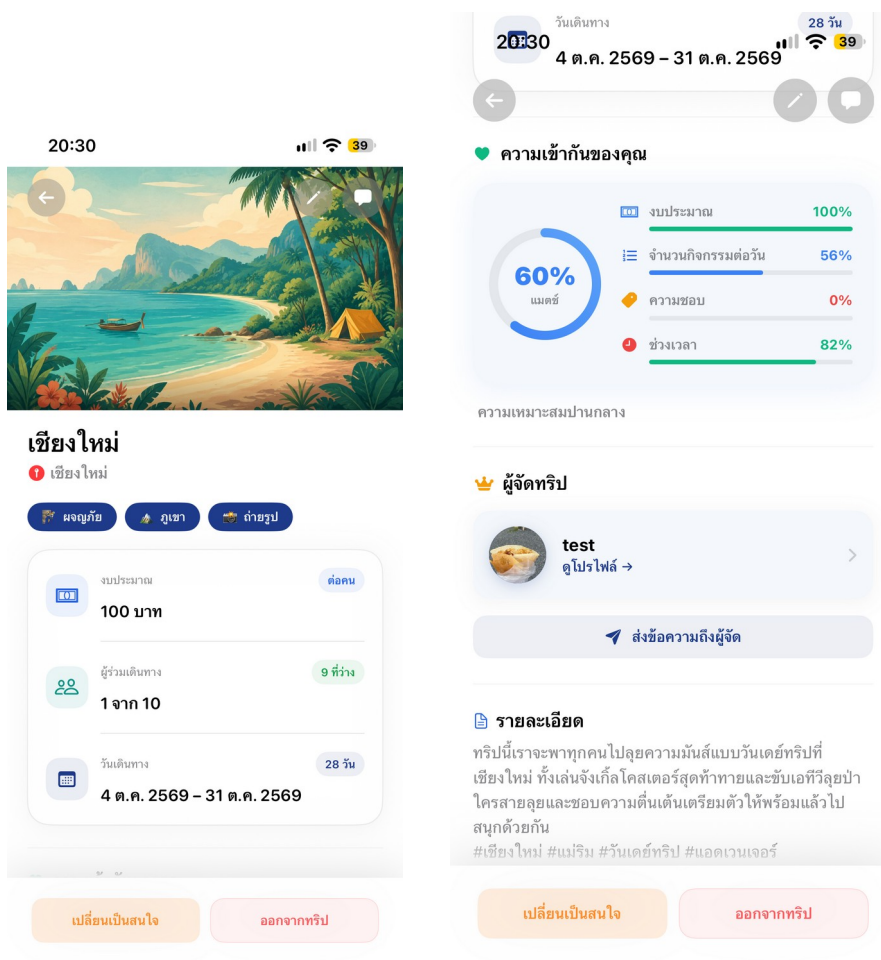
ส่วนข้อมูลทั่วไป (General Info): แสดงภาพหน้าปก, ชื่อทริป, สถานที่, และป้ายสถานะจำนวนที่นั่งว่าง พร้อมทั้งแสดงการ์ดข้อมูลสำคัญ 3 ส่วน ได้แก่ งบประมาณต่อคน, จำนวนสมาชิกรวม, และระยะเวลาเดินทาง

ส่วนวิเคราะห์ความเข้ากันได้ (Match Breakdown): แสดงกราฟคะแนนความเข้ากันได้ (Match Score) แบบรวม และแยกย่อยรายละเอียดเป็นแถบกราฟเส้นในแต่ละมิติ (งบประมาณ, ไลฟ์สไตล์, ความชอบ, ช่วงเวลา) พร้อมประโยคสรุปผลลัพธ์น่ารักๆ (เช่น "ไปได้แน่นอนไปกันรอด")

ส่วนรายละเอียดและแผนการเดินทาง (Description & Itinerary): อธิบายลักษณะของทริป และแสดงตารางกำหนดการเดินทางแต่ละวัน (Timeline) ที่ระบุเวลา สถานที่ และกิจกรรมอย่างเป็นลำดับขั้นตอน

ส่วนข้อมูลบุคคล (Creator & Participants): แสดงโปรไฟล์ผู้จัดทริปพร้อมปุ่มกดส่งข้อความ (Direct Message) และแสดงรายชื่อสมาชิกทั้งหมดที่แบ่งสถานะชัดเจน (เช่น ไปแน่นอน!, สนใจทริปนี้)

ส่วนการตัดสินใจ (Action Bar): ด้านล่างสุดของหน้าจอจะมีปุ่มแบบตรึง (Sticky Button) ให้ผู้ใช้สามารถกด "ดาว สนใจ" หรือกด "บวก จะไปด้วย" เพื่อยืนยันการเข้าร่วมได้ทันที





รูปที่ 4.8 รายละเอียดทริป

5.) หน้าจอการสร้างทริปใหม่ (Create New Trip Screen)

ผลลัพธ์การพัฒนาระบบในส่วนของการสร้างทริปใหม่ ถูกออกแบบมาในรูปแบบฟอร์มที่ครอบคลุมและใช้งานง่ายสำหรับเจ้าของทริป (Host) เพื่อรองรับการนำข้อมูลไปประมวลผลขั้นสูง โดยแบ่งผลการดำเนินงานออกเป็น 3 ส่วนหลัก ดังนี้

1. การระบุข้อมูลและบริบทของทริป (Input Context)

ผลการแสดงหน้าจอ: ระบบแสดงฟอร์มรับข้อมูลพื้นฐานที่จำเป็นอย่างครบถ้วน ได้แก่ ชื่อทริป, การอัปโหลดรูปภาพหน้าปก (รองรับหลายรูป), การเลือกจังหวัดเป้าหมาย, วันเริ่ม-สิ้นสุดการเดินทาง, งบประมาณ (ระบุได้ทั้งแบบต่อคนและรวมทั้งทริป), และจำนวนสมาชิกที่รับสมัคร นอกจากนี้ยังมีสวิตช์สำหรับตั้งค่าความเป็นสาธารณะ (Public) ของทริป

ผลการประมวลผล: ข้อมูลบริบทเหล่านี้จะถูกบันทึกและจัดเก็บเข้าสู่ฐานข้อมูล เพื่อนำไปใช้ประกอบหลักในการสร้างการ์ดทริป (Trip Card) สำหรับแสดงผลบนหน้า Feed หลักของระบบ

2. ข้อมูลทริปที่ใช้ใน Smart Matching

ผู้สร้างทริประบุ category หลัก เลือก interestTags เพิ่มเติมได้ไม่เกิน 2 หมวด
 ระบุ budget และ timeOfDay ส่วน activityStyle ไม่มีค่าที่กรอกโดยตรง แต่
 Backend คำนวณจากจำนวนกิจกรรมใน itinerary หาค่าด้วยจำนวนวัน
 Backend เปรียบเทียบข้อมูลความต้องการของผู้ใช้กับทริปที่ละปัจจัย แล้วหาค่าเฉลี่ยของ
 คะแนนย่อยที่มีข้อมูลเพื่อจัดลำดับทริป

3. การบูรณาการแผนการเดินทางอัจฉริยะ (AI Itinerary Integration)

ผลการแสดงหน้าจอ:ระบบได้นำบูรณาการเครื่องมือปัญญาประดิษฐ์ไว้ในหน้าสร้างทริปอย่างเป็น
 รูปธรรม โดยปรากฏในรูปแบบปุ่ม "AI ช่วยจัดทริป" บริเวณช่องกรอกรายละเอียด และมีปุ่ม
 ฟังก์ชัน "รูปปฏิทิน เริ่มสร้างแผนท่องเที่ยว" สำหรับใช้จัดการตารางการเดินทางแต่ละวัน (Daily
 Itinerary)

ผลการประมวลผล: เมื่อมีการเรียกใช้งานปุ่ม AI ระบบจะทำการเชื่อมต่อกับ API ของปัญญา
 ประดิษฐ์เชิงสร้างสรรค์ (Gemini AI) โดยจะดึงข้อมูลบริบททริปที่ถูกพิมพ์ไว้เบื้องต้นส่งไป
 ประมวลผล เพื่อช่วยร่างคำอธิบายทริปหรือจัดตารางเวลาเดินทางให้อัตโนมัติ ผลลัพธ์ที่ได้ช่วย
 ลดความยุ่งยากและอำนวยความสะดวกให้เจ้าของทริปในการจัดการแผนการเดินทางได้อย่าง
 รวดเร็วและชาญฉลาด

20:32

ยกเลิก

แก้ไขทริป

ข้อมูลทริป

ขั้นตอนที่ 1 จาก 4

ชื่อทริป

เชียงใหม่

รูปภาพ

เลือกรูปภาพ (ได้หลายรูป)
รูปแรกจะเป็นรูปหน้าปก

หรือวาง URL รูปภาพ

จังหวัด

เชียงใหม่

ระบุสถานที่เจาะจง (ไม่จำเป็น)

เช่น คอยอินทนนท์

ถัดไป

20:32

ยกเลิก

แก้ไขทริป

ข้อมูลทริป

ขั้นตอนที่ 1 จาก 4

สไตล์ของทริป

เลือกได้สูงสุด 3 รายการ เพื่อใช้จับคู่กับความสนใจของผู้ใช้ 3/3

ทะเล

ภูเขา

แคมป์ปิ้ง

ผจญภัย

เที่ยวเมือง

คาเฟ่

อาหาร

ช้อปปิ้ง

ถ่ายรูป

คอนเสิร์ต

แฮงค์เอาท์

ไหว้พระ

รายละเอียด

AI ช่วยจัดทริป

ถัดไป

20:32

ยกเลิก

แก้ไขทริป

ข้อมูลทริป

ขั้นตอนที่ 1 จาก 4

คอนเสิร์ต

แข่งเอตต์

ไหว้พระ

รายละเอียด

AI ช่วยจัดทริป

ความต้องการพิเศษให้ AI (ไม่จำเป็น)

ทริปนี้เราจะพาทุกคนไปลุยความมันส์แบบวันเดีย
ทริปที่เชียงใหม่ ทั้งเล่นจิ้งเกิ้ลโคสเตอร์สุดท้าทาย
และขับเอทีวีลุยป่า ใครสายลุยและชอบความตื่น
เต้นเตรียมตัวให้พร้อมแล้วไปสนุกด้วยกัน
#ผจญภัยในป่า #แข่งเอตต์ #ไหว้พระ #คอนเสิร์ต

แท็ก / คีย์เวิร์ด

เช่น ทะเล, คาเฟ่, ธรรมชาติ

สาธารณะ

ทุกคนสามารถเห็นทริปนี้ได้

ถัดไป

20:32

ยกเลิก

แก้ไขทริป

วันและงบประมาณ

ขั้นตอนที่ 2 จาก 4

วันเริ่ม

วันสิ้นสุด

ลบออก

4 ต.ค. 2569

31 ต.ค. 2569

งบประมาณ (บาท)

จำนวนคน

100

10

ประเภทงบ

ต่อคน

ต่อทริป (รวม)

ช่วงเวลาของทริป

ช่วงเช้า (06:00 - 11:00 น.)

เช่น ชมพระอาทิตย์ขึ้น, เยี่ยมชมตลาดเช้า

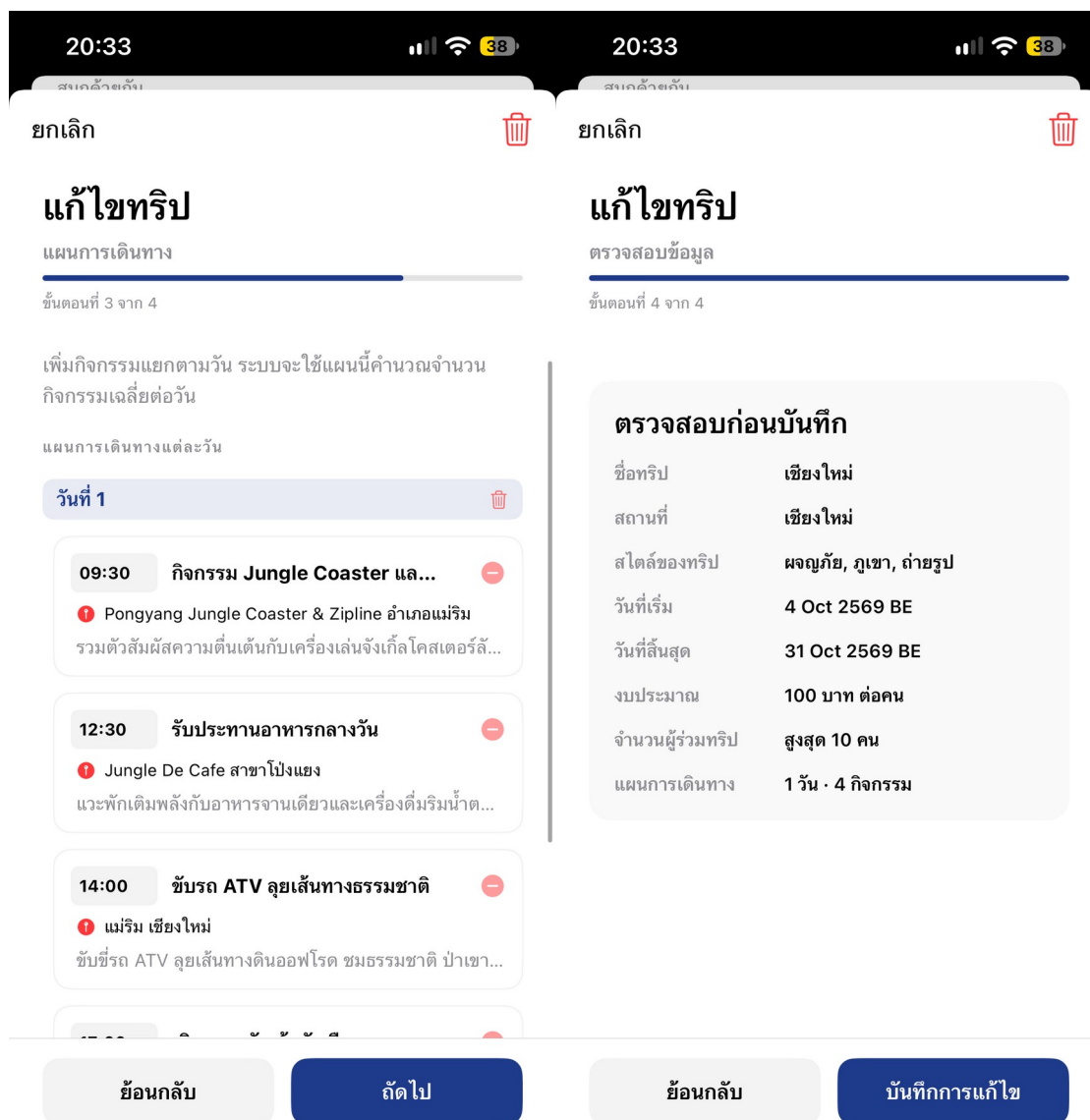
ช่วงกลางวัน (11:00 - 16:00 น.)

เช่น รับประทานอาหาร, พักผ่อนในคาเฟ่, เข้าชมพิพิธภัณฑ์

ช่วงเย็น (16:00 - 20:00 น.)

ย้อนกลับ

ถัดไป



รูปที่ 4.9 หน้าการสร้างทริปของผู้สร้าง (Host)

6.) หน้าจอการแจ้งเตือน (Notification Screen)

หน้าจอนี้ทำหน้าที่เป็นศูนย์กลางในการสื่อสารและแจ้งสถานะความเคลื่อนไหวต่างๆ ภายในแอปพลิเคชัน Go With Us เพื่อให้ผู้ใช้ไม่พลาดข้อมูลสำคัญในการประสานงานทริป โดยมีผลการดำเนินงานดังนี้:

1. ผลการแสดงผลหน้าจอและการติดตามสถานะ (UI & Update Tracking)

ผลการแสดงผลหน้าจอ: ระบบแสดงหน้าต่างศูนย์กลางการแจ้งเตือนในรูปแบบเลื่อนทับหน้าจอหลัก (Modal/Sheet) เพื่อให้ผู้ใช้เข้าถึงได้อย่างรวดเร็ว ในกรณีที่ยังไม่มีข้อมูลอัปเดต ระบบจะแสดงหน้าจอสถานะว่าง (Empty State) พร้อมไอคอนกระดิ่งและข้อความ "ไม่มีการแจ้งเตือน" เพื่อความชัดเจนในการสื่อสารกับผู้ใช้งาน

ผลการประมวลผล: เมื่อมีความเคลื่อนไหวเกิดขึ้น หน้าจอนี้จะรับข้อมูลและแสดงผลรายการแจ้งเตือนแบบเรียลไทม์ เช่น มีข้อความแชทใหม่, ข่าวสารจากระบบ, และที่สำคัญคือ การแจ้งเตือน

2. การบริหารจัดการรายการแจ้งเตือน (Notification Management)

ผลการใช้งาน: อินเทอร์เฟซถูกออกแบบมาเพื่อให้ผู้ใช้งานมีอำนาจในการจัดการข้อมูลของตนเอง โดยสามารถลบรายการแจ้งเตือนเฉพาะรายการที่อ่านแล้ว หรือใช้ฟังก์ชันการจัดการเพื่อดำรงรายการแจ้งเตือนทั้งหมดออกจากหน้าจอ



ไม่มีการแจ้งเตือน

รูปที่ 4.10 หน้าการแจ้งเตือนของแอปพลิเคชัน

7.) หน้าจอการจัดการทริป (Trip Management / My Trip)

ผลการแสดงหน้าจอ: ระบบแสดงหน้าจอ "ทริปของฉัน" ซึ่งทำหน้าที่เป็นศูนย์บัญชาการ (Dashboard) สำหรับบริหารจัดการทริปทั้งหมดของผู้ใช้งาน โดยออกแบบส่วนติดต่อผู้ใช้งานให้มีการจัดระเบียบข้อมูลผ่านระบบแท็บ (Tab Navigation) เพื่อให้ง่ายต่อการเข้าถึง ซึ่งแบ่งออกเป็น 3 หมวดหมู่หลัก ได้แก่

1. สร้างเอง (Created by me): สำหรับแสดงรายการทริปที่ผู้ใช้งานเป็นเจ้าของ (Host) หรือเป็นผู้ริเริ่มสร้างขึ้น
2. เข้าร่วมแล้ว (Joined): สำหรับแสดงรายการทริปที่ผู้ใช้งานได้ส่งคำขอและได้รับการยืนยันสิทธิ์ให้เข้าร่วมเป็นสมาชิกเรียบร้อยแล้ว
3. รายการโปรด (Favorites): สำหรับแสดงรายการทริปที่ผู้ใช้งานเคยกดปุ่ม "สนใจ" บันทึกเก็บไว้เพื่อพิจารณาหรือเปรียบเทียบในภายหลัง

การจัดการสถานะว่าง (Empty State Management): ในกรณีที่ระบบตรวจพบว่ายังไม่มีข้อมูลทริปในหมวดหมู่นั้นๆ หน้าจอจะแสดงผลในรูปแบบสถานะว่าง (Empty State) โดยปรากฏสัญลักษณ์ภาพเครื่องบินควบคู่กับข้อความอธิบายสถานะที่ชัดเจน (เช่น "คุณยังไม่มีทริปที่สร้างไว้" หรือ "คุณยังไม่ได้เข้าร่วมทริปใดเลย") พร้อมทั้งแสดงปุ่มคำสั่ง (Call-to-Action) เช่น ปุ่ม "สร้างทริป" เพื่อจูงใจให้ผู้ใช้ออกไปสร้างปฏิสัมพันธ์กับแอปพลิเคชันต่อไป

ผลการประมวลผลและประโยชน์ที่ได้รับ: หน้าจอนี้ช่วยอำนวยความสะดวกให้ผู้ใช้งานสามารถติดตามและบริหารจัดการวงจรชีวิตของทริป (Trip Lifecycle) ของตนเองได้อย่างเป็นระบบ ช่วยลดความสับสนในการใช้งาน และทำให้สามารถเรียกดูข้อมูลทริปที่ตนเองมีส่วนร่วมได้รวดเร็วและมีประสิทธิภาพ



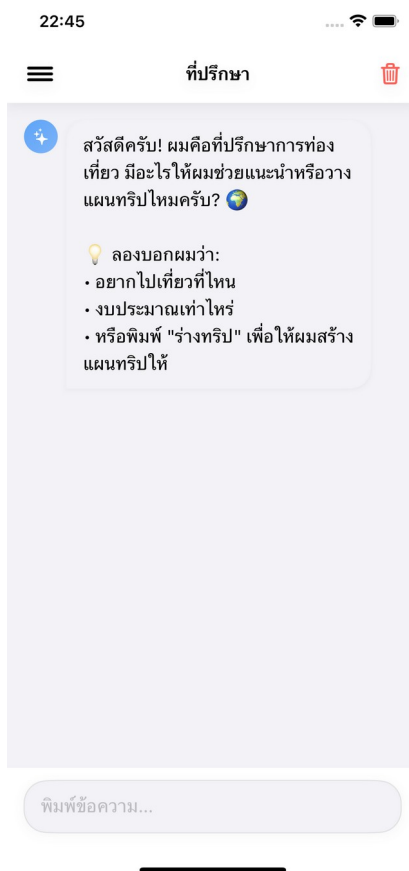
รูปที่ 4.11 หน้าจัดการทริปของผู้ใช้

8.) หน้าจอระบบแชทและผู้ช่วยอัจฉริยะ (Chat / Ai assistant)

ผลการแสดงหน้าจอ: ระบบแบ่งการทำงานออกเป็น 2 ส่วนหลัก ได้แก่

1) หน้ารายการสนทนา (Chat List): ทำหน้าที่รวบรวมประวัติการติดต่อสื่อสารระหว่างผู้ใช้งาน (หากยังไม่มีการสนทนา ระบบจะแสดงหน้า Empty State พร้อมข้อความ "ยังไม่มีการสนทนา") และ 2) หน้าต่างแชทกับผู้ช่วยอัจฉริยะ (AI Chat): ซึ่งปรากฏในชื่อ "ที่ปรึกษา"

ผลการบูรณาการปัญญาประดิษฐ์ (AI Integration): ในหน้าจอที่ปรึกษา ระบบได้เชื่อมต่อกับเทคโนโลยีปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI) เพื่อทำหน้าที่เป็นผู้ช่วยส่วนตัวด้านการท่องเที่ยว โดย AI สามารถโต้ตอบและให้คำแนะนำเบื้องต้นได้ทันที เช่น การสอบถามจุดหมายปลายทาง, งบประมาณ, หรือรับคำสั่งเสียง (เช่น พิมพ์ "ร่างทริป") เพื่อให้ AI ช่วยออกแบบและนำเสนอแผนการเดินทาง พี่เจอรี่นี้ช่วยยกระดับประสบการณ์ผู้ใช้งานให้มีความอัจฉริยะและตอบสนองความต้องการได้แบบเรียลไทม์



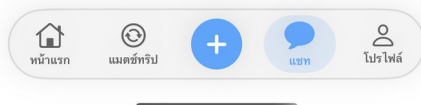
22:52



ข้อความ



ยังไม่มีการสนทนา

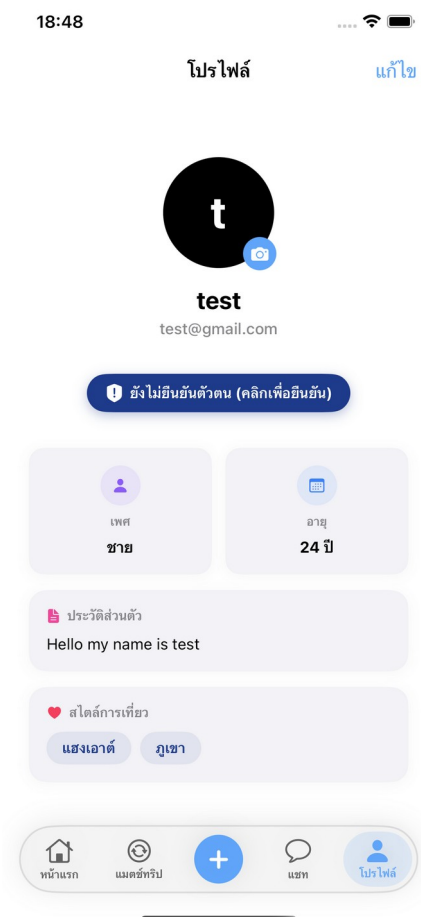


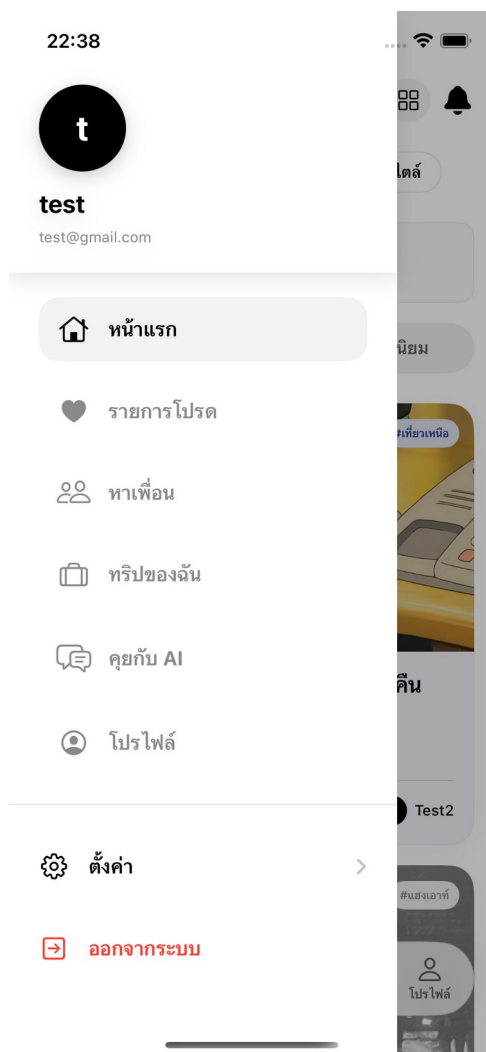
รูปที่ 4.12 หน้าแชทและ ai สำหรับการแปลนหรือคิดสถานที่ท่องเที่ยว

9.) หน้าจอโปรไฟล์และการตั้งค่า (Profile & Settings Screen)

1. ผลการจัดการข้อมูลส่วนบุคคล (Profile Management): หน้าจอโปรไฟล์ถูกออกแบบมาเพื่อแสดงตัวตนของผู้ใช้งานอย่างรอบด้าน ประกอบด้วย รูปโปรไฟล์, ชื่อผู้ใช้, แถบแจ้งเดือนสถานะการยืนยันตัวตน (Identity Verification), เพศ, อายุ, ประวัติส่วนตัวแบบย่อ (Bio), และป้ายกำกับ (Tags) สไตล์การท่องเที่ยว โดยผู้ใช้สามารถเข้าสู่โหมด "แก้ไขโปรไฟล์" ซึ่งระบบรองรับการอัปโหลดรูปภาพส่วนตัวสูงสุดถึง 6 รูป และอนุญาตให้ปรับปรุงข้อมูลพื้นฐาน (เช่น วันเกิด และ Username) ได้ตามเงื่อนไขที่ระบบกำหนด

2. ผลการเข้าถึงเมนูและการตั้งค่า (Side Menu Navigation): ระบบใช้การออกแบบเมนูด้านข้าง (Side Menu / Drawer) เป็นศูนย์กลางทิศทางการนำทาง (Navigation Hub) เพื่อให้ผู้ใช้สลับหน้าจอไปยังฟังก์ชันหลักต่างๆ (เช่น รายการโปรด, หาเพื่อน, ทริปของฉัน) ได้อย่างสะดวก นอกจากนี้ยังเป็นจุดศูนย์กลางในการเข้าถึงเมนู "ตั้งค่า" (Settings) สำหรับจัดการการทำงานเชิงลึกของแอปพลิเคชัน และมีเมนู "ออกจากระบบ" (Logout) เพื่อความปลอดภัยขั้นสูงสุดในการบริหารจัดการบัญชีผู้ใช้



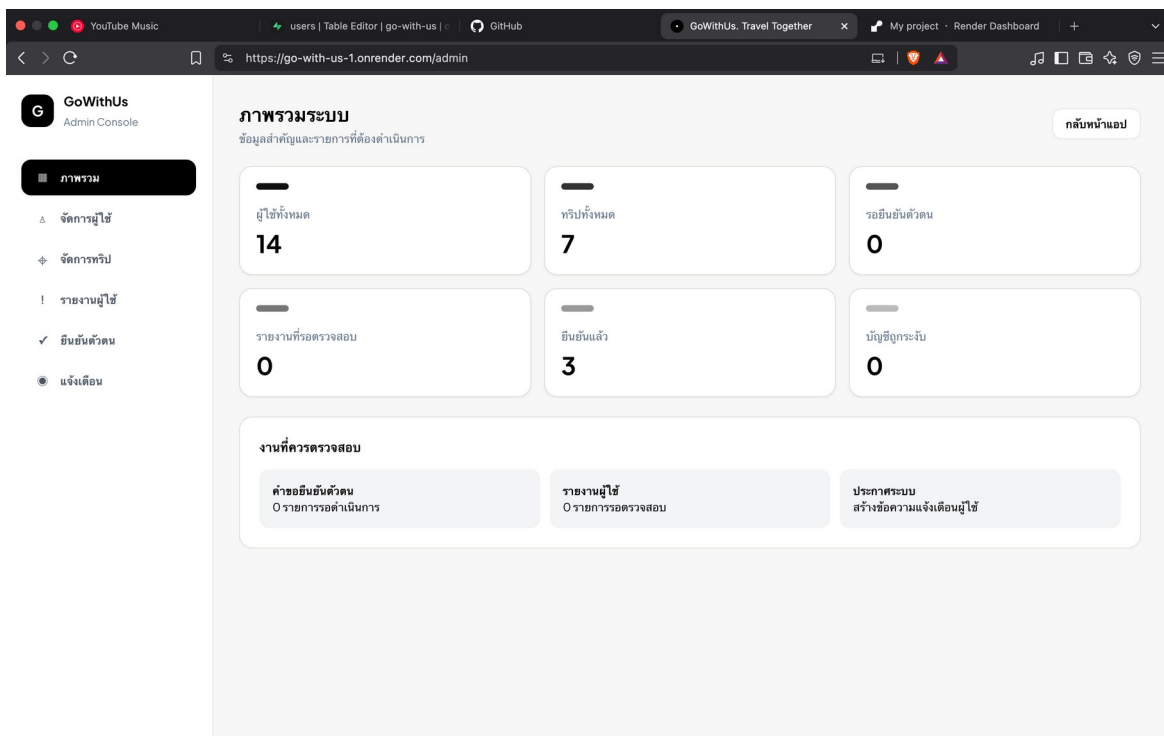


รูปที่ 4.13 หน้าโปรไฟล์และการตั้งค่าของแอปพลิเคชัน

4.1.2 ผลการพัฒนาระบบ Backoffice Web ระบบจัดการหลังบ้าน (Backoffice) พัฒนาขึ้นมาเพื่อให้ผู้ดูแลระบบ (Administrator) สามารถควบคุม ตรวจสอบ และบริหารจัดการแพลตฟอร์ม GoWithUs ได้อย่างมีประสิทธิภาพ โดยแบ่งออกเป็นหน้าการใช้งานหลัก ๆ ดังนี้

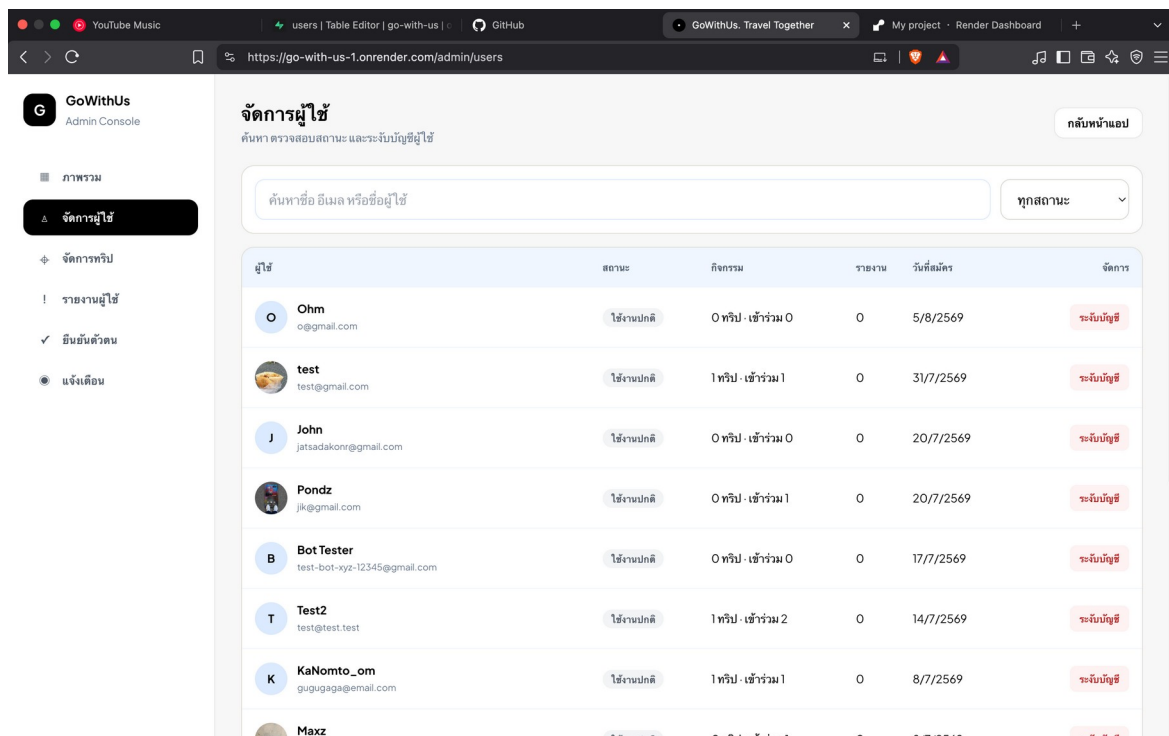
1. หน้าภาพรวมระบบ (Backoffice Dashboard) วัตถุประสงค์: ทำหน้าที่เป็นหน้าหลัก (Landing Page) ของผู้ดูแลระบบเพื่อสรุปข้อมูลสถิติที่สำคัญและการดำเนินงานเร่งด่วนภายในระบบในรูปแบบสรุปย่อ (Real-time Overview) ฟังก์ชันการทำงานหลัก: ระบบแสดงผลการ์ดสถิติ (Metric Cards): แสดงจำนวนข้อมูลสำคัญ ได้แก่ จำนวนผู้ใช้ทั้งหมดในระบบ, จำนวนทริปทั้งหมด, คำขอที่รอยืนยันตัวตน, รายงานปัญหาที่รอการตรวจสอบ, จำนวนบัญชีที่ยืนยันตัวตนแล้ว และจำนวนบัญชีที่ถูกระงับการใช้งาน ส่วนงานที่ควรตรวจสอบ (Quick Action Tasks): แจ้งเตือนผู้ดูแลระบบถึงรายการที่ยังค้างดำเนินการ เช่น จำนวนคำขอยืนยันตัวตนและรายงานผู้ใช้ที่รอการ

ตรวจสอบ เพื่อความรวดเร็วในการจัดการข้อมูล ระบบทางลัดประกาศระบบ: ลิงก์ด่วนสำหรับการสร้างข้อความแจ้งเตือนหาผู้ใช้



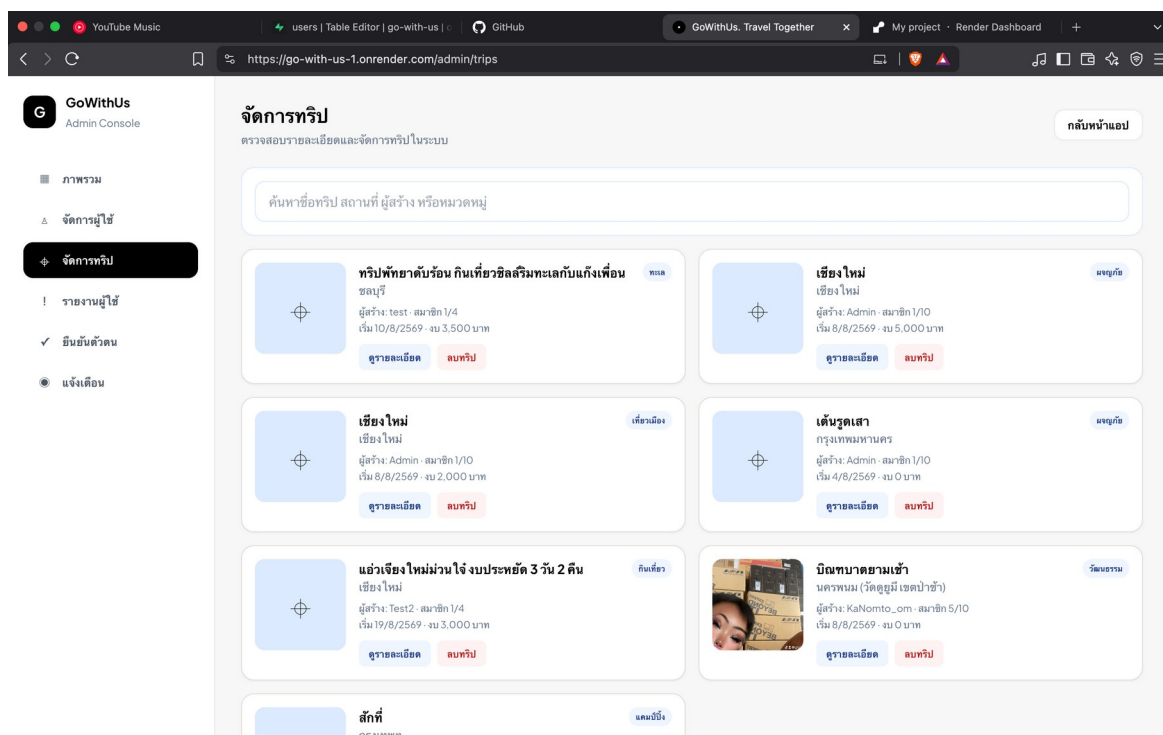
รูปที่ 4.14 หน้าภาพรวมระบบ

- หน้าจัดการผู้ใช้ (User Management) วัตถุประสงค์: ใ้ควบคุม ตรวจสอบสถานะ และจัดการสิทธิ์การใช้งานบัญชีของผู้ใช้งานทั้งหมดในแพลตฟอร์ม ฟังก์ชันการทำงานหลัก: ตารางรายชื่อผู้ใช้ (User Data Table): แสดงรายละเอียดส่วนบุคคล เช่น ชื่อบัญชี (Name), อีเมล (Email), รูปโปรไฟล์, วันที่สมัครสมาชิก (Register Date) ระบบแสดงสถานะและกิจกรรมผู้ใช้ (Status & Activity Tracker): แสดงสถานะการใช้งานปัจจุบัน (เช่น ใช้งานปกติ, ระงับการใช้งาน) และประวัติกิจกรรมในระบบ (จำนวนทริปที่ เป็นผู้สร้าง และจำนวนทริปที่เข้าร่วม) รวมถึงจำนวนครั้งที่ผู้ใช้นี้โดนรายงานปัญหา (Reports) ระบบค้นหาและกรองข้อมูล: ผู้ดูแลระบบสามารถค้นหาผู้ใช้งานผ่านชื่อ หรืออีเมล และเลือกกรองสถานะผู้ใช้เพื่อความสะดวกในการค้นหา การระงับบัญชีใช้งาน (Account Suspension): มีปุ่มดำเนินการควบคุมสถานะความปลอดภัย สามารถสั่งระงับการใช้งานชั่วคราวหรือถาวร (Ban) สำหรับบัญชีผู้ใช้ที่ทำผิดกฎระเบียบของแพลตฟอร์ม



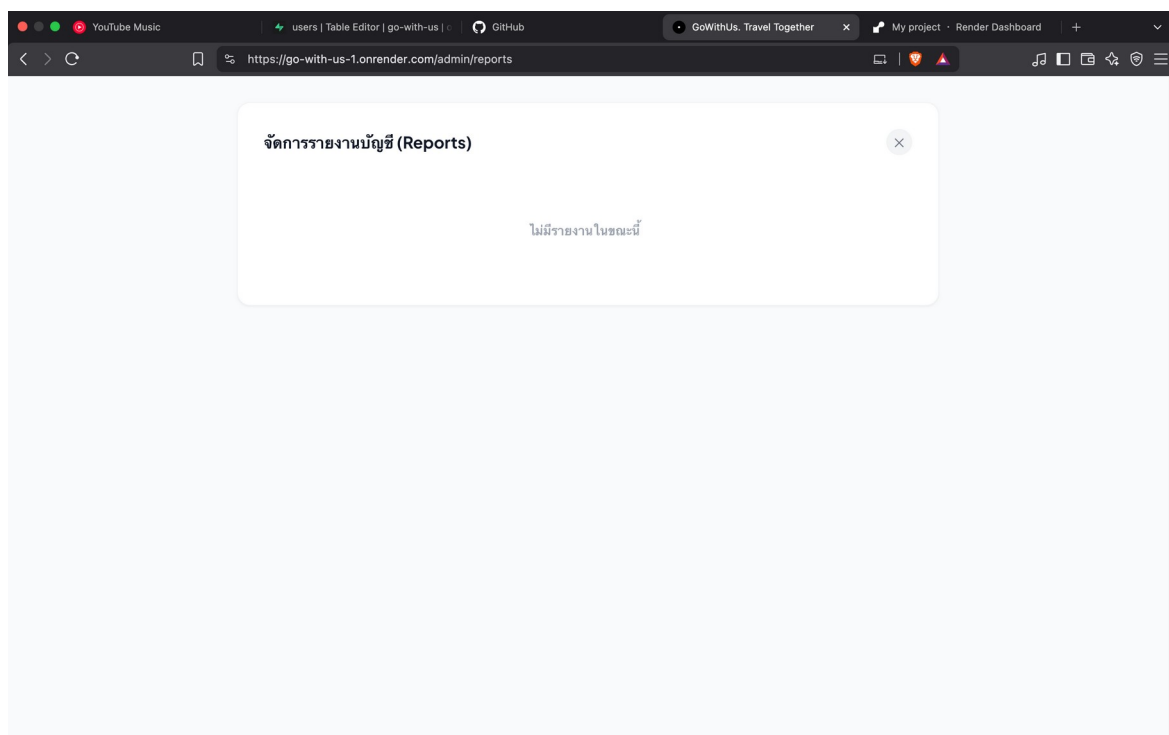
รูปที่ 4.15 หน้าจัดการผู้ใช้

- หน้าจัดการทริป (Trip Management) วัตถุประสงค์: ใช้ดูแลความเรียบร้อยของกิจกรรมหรือทริปท่องเที่ยวที่ถูกสร้างขึ้นโดยสมาชิกในระบบ เพื่อคัดกรองเนื้อหาที่ไม่เหมาะสม ฟังก์ชันการทำงานหลัก: ระบบค้นหาทริป: มีแถบค้นหาอัจฉริยะสามารถค้นหาจากชื่อทริป, สถานที่ปลายทาง, ชื่อผู้สร้างทริป หรือหมวดหมู่ทริปได้ ระบบแสดงรายการทริปในรูปแบบการ์ดข้อมูล: แสดงภาพหน้าปกทริปและหัวข้อทริป แสดงรายละเอียดปลายทาง (จังหวัด) วันที่จัดกิจกรรม และงบประมาณ แสดงจำนวนสมาชิกปัจจุบันเทียบกับจำนวนที่เปิดรับ (เช่น สมาชิก 1/4 คน) แท็กหมวดหมู่ความสนใจของทริป (เช่น ทะเล, ผจญภัย, วัฒนธรรม) การจัดการทริป: ปุ่มสำหรับเข้าสู่รายละเอียดเชิงลึกของทริป (รายละเอียดกิจกรรม, แผนการเดินทาง/Itinerary) และปุ่ม "ลบทริป" (Delete Trip) ในกรณีที่ทริปดังกล่าวขัดต่อกฎของแพลตฟอร์มหรือเข้าข่ายการสแปม



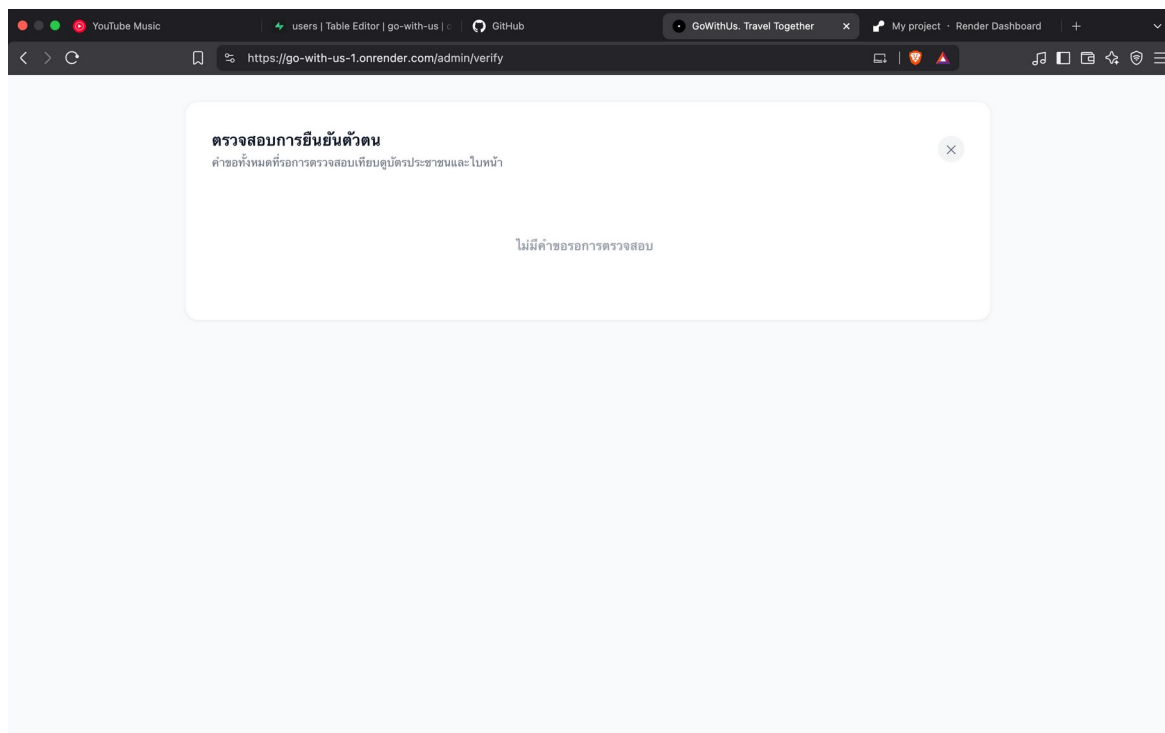
รูปที่ 4.16 หน้าจัดการทริป

4. หน้าจัดการรายงานผู้ใช้ (User Reports) วัตถุประสงค์: เป็นศูนย์กลางรับเรื่องร้องเรียนและการรายงานพฤติกรรมที่ไม่เหมาะสมของสมาชิกจากผู้ใช้ท่านอื่นในแอปพลิเคชัน ฟังก์ชันการทำงานหลัก: ระบบรายการร้องเรียน (Report Tickets): แสดงประวัติและข้อมูลการรายงานปัญหา ประกอบด้วย ชื่อผู้รายงาน (Reporter) และผู้ถูกรายงาน (Reported User) พร้อมเหตุผลในการรายงาน (Reason) และเวลาที่เกิดการรายงาน ระบบคัดกรองพฤติกรรม: ผู้ดูแลระบบสามารถพิจารณาข้อมูลแวดล้อมเพื่อประกอบการตัดสินใจ การดำเนินการลงโทษ (Action Panel): สามารถสั่งการได้โดยตรงจากหน้ารายงาน เช่น: การส่งข้อความแจ้งเตือน/ตักเตือน (Warn) ไปยังกล่องข้อความของผู้ใช้ที่ถูกร้องเรียน การสั่งระงับบัญชีใช้งาน (Ban) ในกรณีที่เป็นการผิดร้ายแรง ปลดการระงับบัญชีใช้งาน (Unban) หากตรวจสอบแล้วไม่พบความผิดปกติ



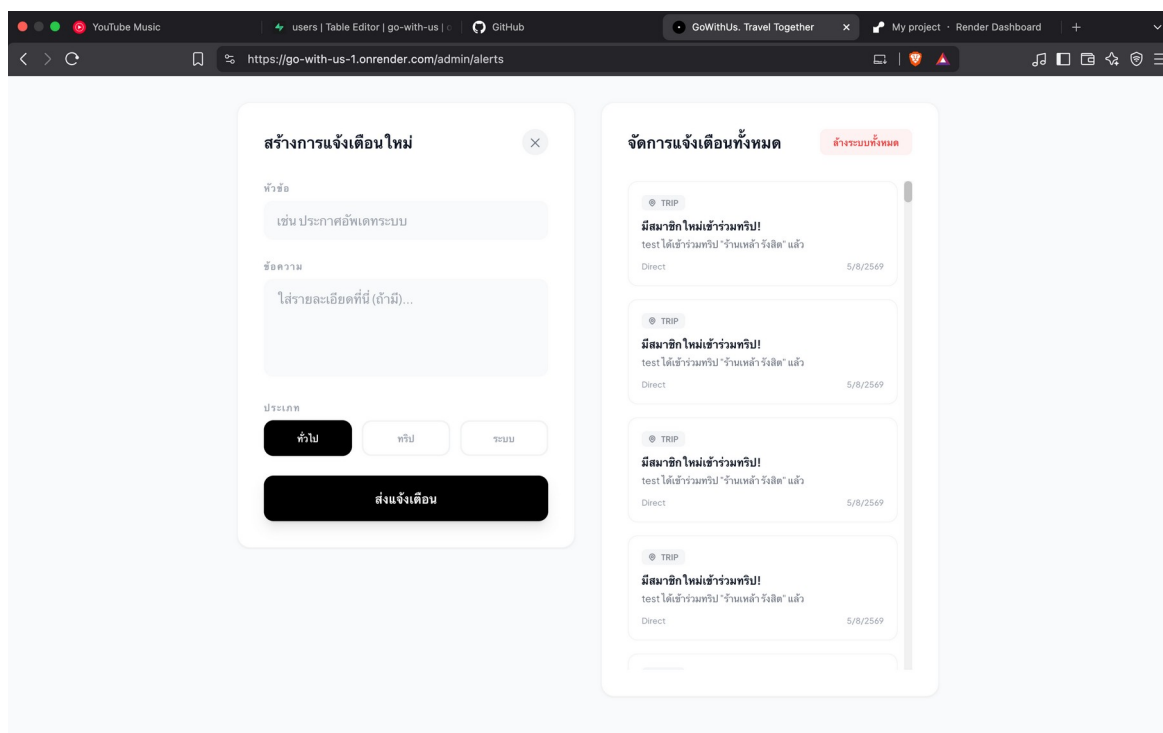
รูปที่ 4.17 หน้าจัดการรายงานผู้ใช้

5. หน้าตรวจสอบคำขอยืนยันตัวตนใน **Backoffice** ช่วยให้ผู้ดูแลระบบตรวจสอบข้อมูลและสถานะคำขอที่ผู้ใช้ส่งมา แล้วอนุมัติหรือปฏิเสธตามข้อมูลที่ตรวจได้ การตรวจการเคลื่อนไหวของใบหน้าบนอุปกรณ์ไม่ใช่ผลเปรียบเทียบอัตลักษณ์กับภาพเอกสารโดยอัตโนมัติ ผู้ดูแลระบบจึงไม่ควรใช้ผลดังกล่าวเพียงอย่างเดียวเพื่อรับรองตัวตน



รูปที่ 4.18 หน้าตรวจสอบการยืนยันตัวตน

6. หน้าจัดการและประกาศการแจ้งเตือนระบบ (System Alerts & Notifications) วัตถุประสงค์: สำหรับส่งประกาศหรือข้อมูลข่าวสารสำคัญจากส่วนกลางไปยังกล่องข้อความแจ้งเตือน (Push Notifications / In-app Alerts) ของผู้ใช้ในระบบทุกคนหรือกลุ่มเป้าหมายเฉพาะ ฟังก์ชันการทำงานหลัก: ฟормสร้างการแจ้งเตือนใหม่ (Create Alert Form): สามารถระบุหัวข้อ (Title), ข้อความรายละเอียด (Message) และเลือกประเภทการแจ้งเตือน (เช่น แจ้งเตือนทั่วไป, แจ้งเตือนทริป, ประกาศระบบ) ส่วนควบคุมการแจ้งเตือนทั้งหมด (Management Panel): รายการประวัติการแจ้งเตือนทั้งหมดที่ระบบส่งออกไป พร้อมปุ่มดำเนินการลบแจ้งเตือนบางรายการหรือปุ่ม "ล้างระบบทั้งหมด" สำหรับจัดการล้างประวัติการแจ้งเตือนในระบบ



รูปที่ 4.19 หน้าจัดการและประกาศการแจ้งเตือนระบบ

4.1.3 ผลการเชื่อมต่อ Backend และฐานข้อมูล

ระบบ Go With Us ทำงานแบบ Client-Server โดยแอป iOS และ Backoffice ส่งคำขอผ่าน Backend API ซึ่งพัฒนาด้วย Node.js และ Express Backend ตรวจสอบสิทธิ์ ประมวลผลกฎของระบบ และใช้ Prisma เชื่อมต่อ PostgreSQL บน Supabase

แอปพลิเคชัน iOS ใช้ Service แยกตามหน้าที่ เช่น AuthService สำหรับบัญชีผู้ใช้ TripService สำหรับข้อมูลทริป MatchService สำหรับระบบจับคู่ และ MessageService สำหรับระบบสนทนา เมื่อผู้ใช้ดำเนินการผ่านแอปพลิเคชัน Service จะส่งคำขอไปยัง Backend API ในรูปแบบ HTTP Request

ระบบ Backoffice ติดต่อ Backend API เพื่อเรียกดูและจัดการข้อมูลที่เกี่ยวข้องกับการบริหารระบบ เช่น ข้อมูลผู้ใช้ ข้อมูลทริป การตรวจสอบตัวตน รายงาน การแจ้งเตือน และข้อมูลสรุปในหน้า Dashboard โดย Backoffice ไม่เชื่อมต่อฐานข้อมูลโดยตรง แต่ต้องดำเนินการผ่าน API เพื่อให้สามารถตรวจสอบสิทธิ์และควบคุมความถูกต้องของข้อมูลได้

เมื่อ Backend ได้รับคำขอ ระบบจะส่งคำขอไปยัง Controller ที่เกี่ยวข้อง จากนั้น Controller จะใช้ Prisma Client สำหรับเพิ่ม อ่าน แก้ไข หรือลบข้อมูลในฐานข้อมูล PostgreSQL เมื่อดำเนินการเสร็จสิ้น Backend จะส่งผลลัพธ์กลับไปยัง iOS หรือ Backoffice ในรูปแบบ JSON

การยืนยันตัวตนด้วย JWT

ระบบใช้ JSON Web Token หรือ JWT สำหรับยืนยันตัวตนและควบคุมสิทธิ์การเข้าถึง API เมื่อผู้ใช้เข้าสู่ระบบด้วยข้อมูลที่ถูกต้อง Backend จะตรวจสอบบัญชีผู้ใช้และสร้าง JWT ส่งกลับไปยังแอปพลิเคชัน หลังจากนั้นแอปพลิเคชันจะแนบ Token ไปกับคำขอที่จำเป็นต้องยืนยันตัวตน

รูปแบบการส่ง Token มีดังนี้ `Authorization: Bearer <JWT Token>` ลำดับการทำงานของระบบยืนยันตัวตนเป็นดังนี้

1. ผู้ใช้กรอกข้อมูลเข้าสู่ระบบผ่านแอปพลิเคชัน
2. แอปพลิเคชันส่งข้อมูลไปยัง Backend API
3. Backend ตรวจสอบบัญชีและรหัสผ่านกับข้อมูลในฐานข้อมูล
4. เมื่อข้อมูลถูกต้อง Backend จะสร้าง JWT
5. แอปพลิเคชันจัดเก็บ Token และแนบไปกับคำขอครั้งถัดไป
6. Backend ตรวจสอบ Token ก่อนอนุญาตให้เข้าถึงฟังก์ชันที่มีการป้องกัน

หากไม่พบ Token, Token ไม่ถูกต้อง หรือ Token หมดอายุ ระบบจะปฏิเสธการเข้าถึงและส่งข้อความแจ้งเตือนผิดพลาดกลับไปยังผู้ใช้ วิธีดังกล่าวช่วยป้องกันไม่ให้บุคคลที่ไม่ได้รับอนุญาตเข้าถึงข้อมูลส่วนตัวหรือดำเนินการแทนผู้ใช้งาน

การรับส่งข้อมูล

ข้อมูลระหว่างแอปพลิเคชัน Backend และฐานข้อมูลรับส่งในรูปแบบ JSON โดยครอบคลุมข้อมูลบัญชีผู้ใช้ โปรไฟล์ ความสนใจ สไตล์การเดินทาง ทริป สมาชิกทริป Match Score ข้อความ และการแจ้งเตือน

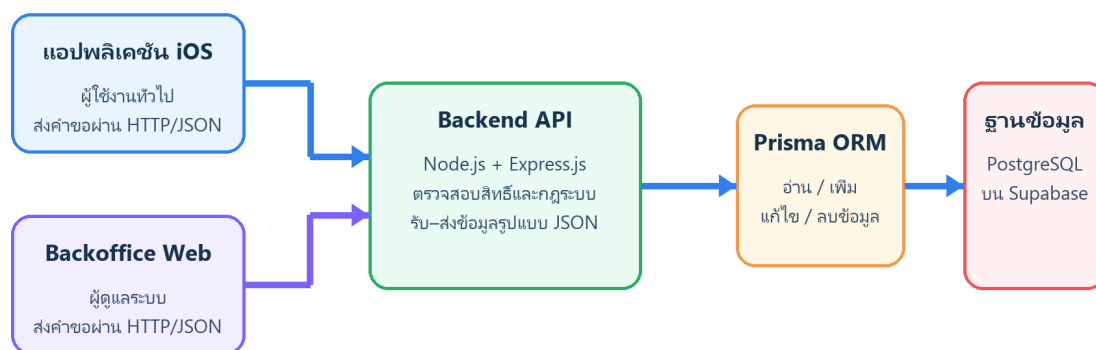
ตัวอย่างกระบวนการสร้างทริปมีลำดับดังนี้

1. ผู้ใช้กรอกข้อมูลทริปในแอปพลิเคชัน
2. แอปพลิเคชันแปลงข้อมูลเป็น JSON และส่งพร้อม JWT ไปยัง Backend API
3. Backend ตรวจสอบ Token และความครบถ้วนของข้อมูล
4. Controller ส่งคำสั่งบันทึกข้อมูลผ่าน Prisma
5. PostgreSQL บันทึกข้อมูลทริป
6. Backend ส่งข้อมูลทริปที่สร้างแล้วกลับไปยังแอปพลิเคชัน
7. แอปพลิเคชันนำข้อมูลมาแสดงในหน้ารายการทริป

สำหรับระบบ Smart Matching แอปพลิเคชันจะส่งคำขอไปยัง Backend เพื่อขอรายการทริปแนะนำ Backend จะอ่านข้อมูลความสนใจและสไตล์การเดินทางของผู้ใช้จากฐานข้อมูล แล้วนำไปเปรียบเทียบกับคุณลักษณะของแต่ละทริป หลังจากคำนวณ Match Score แล้ว ระบบจะเรียงลำดับทริปจากคะแนนสูงไปต่ำและส่งผลลัพธ์กลับไปแสดงบนแอปพลิเคชัน

จากการเชื่อมต่อดังกล่าว ระบบสามารถแยกหน้าที่ของแต่ละส่วนได้อย่างชัดเจน โดย iOS และ Backoffice ทำหน้าที่รับข้อมูลและแสดงผล Node.js ทำหน้าที่ประมวลผลและตรวจสอบกฎการทำงาน ส่วน Supabase/PostgreSQL ทำหน้าที่จัดเก็บข้อมูลอย่างถาวร ส่งผลให้ข้อมูลจากแอปพลิเคชันและ Backoffice สามารถใช้งานร่วมกันผ่าน Backend API ชุดเดียวกันได้

โครงสร้างการเชื่อมต่อ Backend และฐานข้อมูล



หมายเหตุ: แอป iOS และ Backoffice ไม่เชื่อมต่อฐานข้อมูลโดยตรง แต่ต้องดำเนินการผ่าน Backend API

รูปที่ 4.20 โครงสร้างการเชื่อมต่อระหว่างแอปพลิเคชัน Backend และฐานข้อมูล

รูปที่ 4.21 ผลการทดสอบการยืนยันตัวตนด้วย JWT โดยปิดบังค่า Token

GoWithUs

Admin Console

ภาพรวม

จัดการผู้ใช้

จัดการทรัพย์สิน

รายงานผู้ใช้

ยืนยันตัวตน

แจ้งเตือน

จัดการผู้ใช้

ค้นหา ตรวจสอบสถานะ และจะลบผู้ใช้

ค้นหาชื่อ อีเมล หรือชื่อผู้ใช้

ทุกสถานะ

ผู้ใช้	สถานะ	กิจกรรม	รายงาน	วันสิ้นสุด	จัดการ
U Userstest userstest@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	23/8/2569	ระงับผู้ใช้
U User1 user1@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	14/8/2569	ระงับผู้ใช้
K Kkkil ppok@gmail.com	ใช้งานปกติ	1 ทริป - เข้าร่วม 1	0	10/8/2569	ระงับผู้ใช้
P Pond kkk@gmail.com	ใช้งานปกติ	1 ทริป - เข้าร่วม 2	0	10/8/2569	ระงับผู้ใช้
P Pppp ppp@gmail.com	ใช้งานปกติ	1 ทริป - เข้าร่วม 2	0	10/8/2569	ระงับผู้ใช้
O Ohm o@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	5/8/2569	ระงับผู้ใช้
test test@gmail.com	ใช้งานปกติ	1 ทริป - เข้าร่วม 1	0	31/7/2569	ระงับผู้ใช้
J John jatsadakov@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	20/7/2569	ระงับผู้ใช้
Pondz jkg@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 1	0	20/7/2569	ระงับผู้ใช้
B Bot Tester test-bot-xyz-12345@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	17/7/2569	ระงับผู้ใช้
T Test2 test@test.test	ใช้งานปกติ	1 ทริป - เข้าร่วม 2	0	14/7/2569	ระงับผู้ใช้
K KaNomto_om gugugaga@email.com	ใช้งานปกติ	1 ทริป - เข้าร่วม 1	0	8/7/2569	ระงับผู้ใช้
Pondmin ppk@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 1	0	8/7/2569	ระงับผู้ใช้
m maxxxx pom@gmail.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	5/7/2569	ระงับผู้ใช้
p pondzzzz pppk@gmail.com	ใช้งานปกติ	1 ทริป - เข้าร่วม 1	0	5/7/2569	ระงับผู้ใช้
p pondxz jak@gmail.com	ยืนยันแล้ว	0 ทริป - เข้าร่วม 0	0	5/7/2569	ระงับผู้ใช้
P Pondzz mya@gmail.com	ยืนยันแล้ว	0 ทริป - เข้าร่วม 0	0	5/7/2569	ระงับผู้ใช้
T Test User user@gowithus.com	ใช้งานปกติ	0 ทริป - เข้าร่วม 0	0	5/7/2569	ระงับผู้ใช้
A Admin ADMIN admin@gowithus.com	ยืนยันแล้ว	0 ทริป - เข้าร่วม 1	0	5/7/2569	ระงับผู้ใช้

รูปที่ 4.22 ผลการรับข้อมูลจาก Backend API มาแสดงในระบบ Backoffice

4.2 ผลการทำงานของระบบ Smart Matching

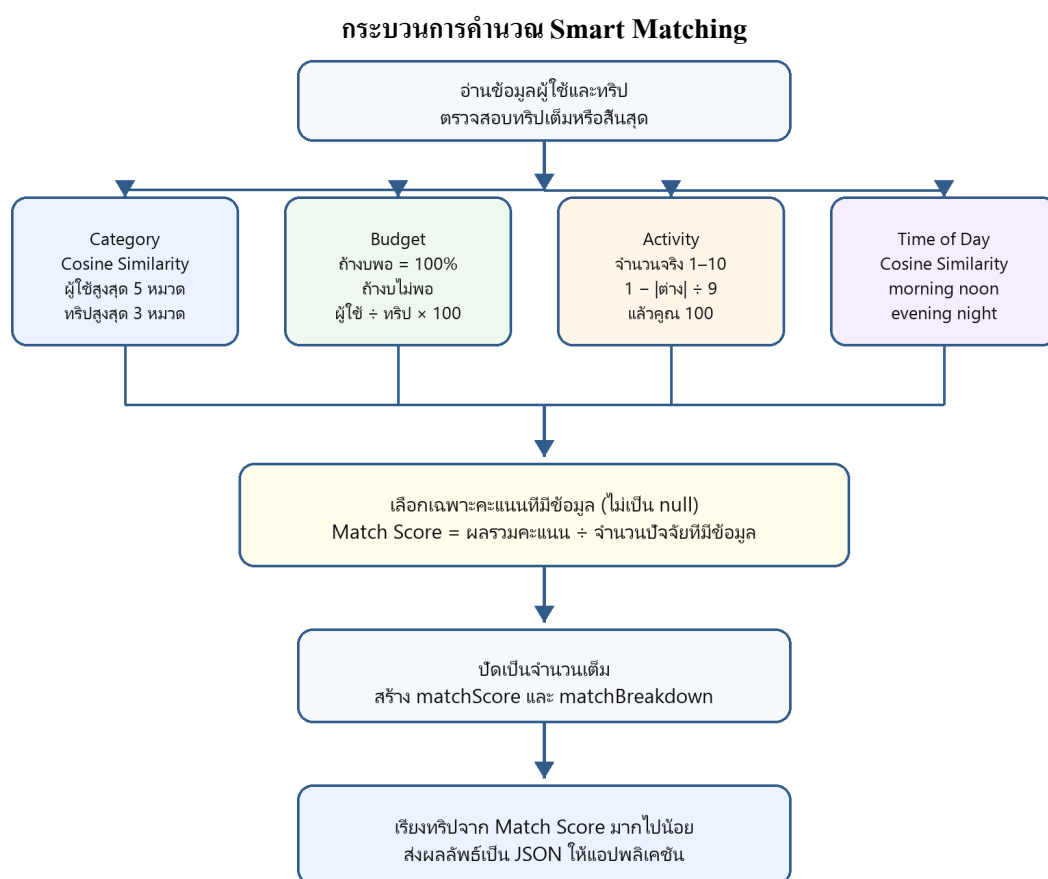
ระบบ Smart Matching วิเคราะห์ผู้ใช้งานกับทริปจาก 4 ปัจจัย ได้แก่ หมวดหมู่งบประมาณ จำนวนกิจกรรมเฉลี่ยต่อวัน และช่วงเวลา ระบบคำนวณคะแนนย่อยแยกตามชนิดข้อมูล แล้วหาค่าเฉลี่ยเลขคณิตของเฉพาะปัจจัยที่มีข้อมูลเป็น Match Score 0–100 เพื่อเรียงลำดับทริป

เมื่อข้อมูลครบทั้ง 4 ปัจจัย ระบบหาค่าเฉลี่ยของคะแนนย่อยทั้ง 4 ค่า หากข้อมูลบางด้านไม่ครบ ระบบจะคำนวณจากจำนวนปัจจัยที่มีข้อมูลจริง

ปัจจัยที่ใช้ในการจับคู่	วิธีรวมคะแนน
หมวดหมู่ (Category)	ค่าเฉลี่ยเท่ากัน
งบประมาณ (Budget)	ค่าเฉลี่ยเท่ากัน
จำนวนกิจกรรมต่อวัน (Activity Style)	ค่าเฉลี่ยเท่ากัน
ช่วงเวลา (Time of Day)	ค่าเฉลี่ยเท่ากัน
คะแนนรวม	ผลรวมคะแนนที่มีข้อมูล ÷ จำนวนปัจจัยที่มีข้อมูล

ตัวอย่าง เมื่อมีคะแนน Category=71, Budget=75, Activity=67 และ Time=100 คะแนนรวมเท่ากับ $(71+75+67+100) \div 4 = 78.25$ และปัดเป็น 78% แต่ถ้าไม่มี Time จะใช้ $(71+75+67) \div 3 = 71\%$

ก่อนคำนวณ ระบบตรวจสอบสถานะของทริป หากทริปมีสมาชิกครบจำนวนสูงสุดหรือสิ้นสุดไปแล้ว ระบบจะกำหนด Match Score เป็น 0 เพราะผู้ใช้ไม่สามารถเข้าร่วมทริปนั้นได้



ภาพที่ 4.23 กระบวนการทำงานของระบบ Smart Matching

4.2.1 ข้อมูลความชอบของผู้ใช้ (User Preferences)

ระบบรวบรวมข้อมูลจากแบบสอบถามสไตล์การเดินทางและจัดเก็บไว้กับข้อมูลผู้ใช้ โดยนำข้อมูลต่อไปนี้มาใช้ในการคำนวณ

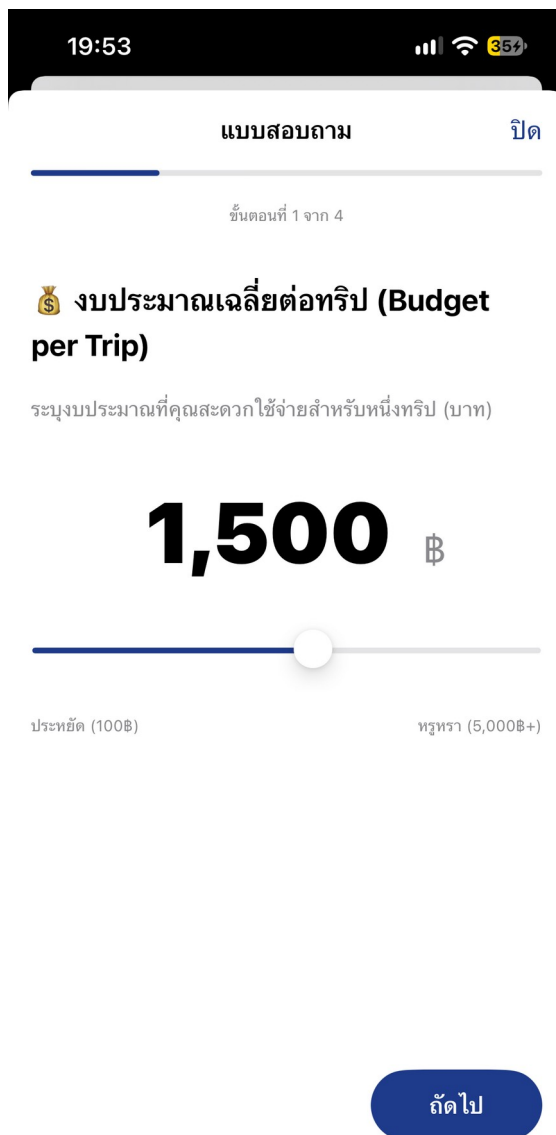
1. **Budget** คืองบประมาณต่อทริปที่ผู้ใช้สะดวกจ่าย กำหนดได้ตั้งแต่ 100–5,000 บาทขึ้นไป
2. **Activity Style** คือจำนวนสถานที่ที่ผู้ใช้สะดวกเที่ยวต่อวัน เลือกค่าได้ตั้งแต่ 1–10 และจัดเก็บเป็นจำนวนเต็ม
3. **Time of Day** คือช่วงเวลาที่ผู้ใช้สะดวกทำกิจกรรม ได้แก่ เช้า (morning) กลางวัน (noon) เย็น (evening) และกลางคืน (night) โดยเลือกได้มากกว่าหนึ่งช่วงเวลา
4. **Interests** คือหมวดหมู่ที่ผู้ใช้สนใจ เลือกได้ไม่เกิน 5 รายการ จาก 13 หมวด ได้แก่ ทะเล ภูเขา แคมป์ปิ้ง เทียวเมือง คาเฟ่ อาหาร แสงเอดส์ ถ่ายรูป ช้อปปิ้ง คอนเสิร์ต ผจญภัย ไหว้พระ และอื่น ๆ

ข้อมูลแบบสอบถามถูกเก็บในตารางผู้ใช้ โดยฟิลด์ interests เก็บหมวดหมู่ที่สนใจ และฟิลด์ travelStyle (JSON) เก็บรายละเอียดสไตล์การเดินทาง เช่น budget, activityStyle และ timeOfDay การเก็บเป็น JSON ช่วยให้ระบบอ่านค่ารายปัจจัยได้โดยตรงและรองรับข้อมูลหลายค่า เช่น ช่วงเวลาที่ผู้ใช้เลือกมากกว่าหนึ่งช่วง

ตัวอย่าง travelStyle: {"budget": 2000, "activityStyle": 3, "timeOfDay": ["morning", "evening"]} หมายถึงผู้ใช้งบ 2,000 บาทต่อทริป ต้องการเที่ยว 3 สถานที่ต่อวัน และสะดวกช่วงเช้ากับเย็น

ระบบใช้ค่าจำนวนเงินจริงในการคำนวณงบประมาณ ไม่ได้แปลงงบประมาณเป็นเวกเตอร์ลอการิทึม ตัวอย่าง: ผู้ใช้งบ 2,000 บาทและทริปราคา 2,000 บาท ได้คะแนนงบประมาณ 100%; หากผู้ใช้งบ 1,500 บาทและทริปราคา 2,000 บาท ได้ $1,500 \div 2,000 \times 100 = 75\%$

หมวดหมู่และช่วงเวลาถูกแปลงเป็น Multi-hot Vector เพื่อคำนวณ Cosine Similarity ส่วนงบประมาณใช้สัดส่วนจำนวนเงินจริง และกิจกรรมใช้ระยะห่างของค่าจริงในช่วง 1-10 จึงไม่มีเวกเตอร์รวมทุกปัจจัย



19:53

แบบสอบถาม ปิด

ขั้นตอนที่ 1 จาก 4

💰 งบประมาณเฉลี่ยต่อทริป (Budget per Trip)

ระบุงบประมาณที่คุณสะดวกใช้จ่ายสำหรับหนึ่งทริป (บาท)

1,500 ฿

ประหยัด (100฿) หลูหร่า (5,000฿+)

ถัดไป

คำบรรยาย: ภาพที่ 4.24 การกำหนดงบประมาณ

19:53

35%

แบบสอบถาม

ปิด

ขั้นตอนที่ 2 จาก 4

🎯

จำนวนสถานที่ท่องเที่ยวต่อวัน (Places per Day)

เลือกจำนวนสถานที่ที่คุณสะดวกเที่ยวในหนึ่งวัน

1 สถานที่ต่อวัน

✓

2 สถานที่ต่อวัน

○

3 สถานที่ต่อวัน

○

4 สถานที่ต่อวัน

○

5 สถานที่ต่อวัน

○

6 สถานที่ต่อวัน

○

ย้อนกลับ

ถัดไป

คำบรรยาย: ภาพที่ 4.25 การเลือกสถานที่จะไปเที่ยวต่อวัน

19:53

35%

แบบสอบถาม

ปิด

ขั้นตอนที่ 3 จาก 4

🕒

ช่วงเวลาที่ชอบท่องเที่ยว (Time of Day)

เลือกช่วงเวลาที่คุณชอบออกไปทำกิจกรรมหรือท่องเที่ยว (เลือกได้มากกว่า 1 ช่วง)

ช่วงเช้า (06:00 - 11:00 น.)

เหมาะสำหรับคนที่ชอบออกจากที่พักเร็ว เช่น ดูพระอาทิตย์ขึ้น เดินตลาดเช้า หรือรับประทานอาหารเช้า

☐

ช่วงกลางวัน (11:00 - 16:00 น.)

เหมาะสำหรับคนที่ชอบเที่ยวช่วงสายถึงบ่าย เช่น เข้าชมสถานที่ท่องเที่ยว แวะคาเฟ่ หรือรับประทานอาหารกลางวัน

☐

ช่วงเย็น (16:00 - 20:00 น.)

เหมาะสำหรับคนที่ชอบเที่ยวช่วงเย็นก่อนค่ำ เช่น เดินเล่น ชมพระอาทิตย์ตก หรือรับประทานอาหารเย็น

☐

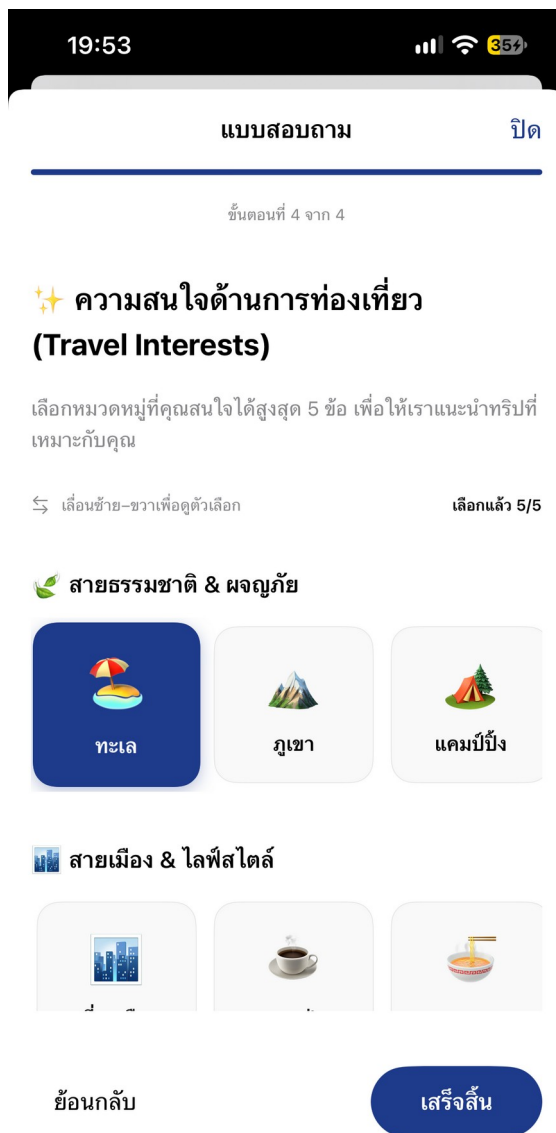
ช่วงกลางคืน (20:00 น. เป็นต้นไป)

☐

ย้อนกลับ

ถัดไป

คำบรรยาย: ภาพที่ 4.26 การกำหนดหมวดหมู่ช่วงเวลาที่สะดวก



19:53

แบบสอบถาม ปิด

ขั้นตอนที่ 4 จาก 4

✨ ความสนใจด้านการท่องเที่ยว
(Travel Interests)

เลือกหมวดหมู่ที่คุณสนใจได้สูงสุด 5 ข้อ เพื่อให้เราแนะนำทริปที่เหมาะสมให้คุณ

← เลื่อนซ้าย-ขวาเพื่อดูตัวเลือก เลือกแล้ว 5/5

สายธรรมชาติ & ผจญภัย

ทะเล ภูเขา แคมป์ปิ้ง

สายเมือง & ไลฟ์สไตล์

เมือง กาแฟ อาหาร

ย้อนกลับ เสร็จสิ้น

คำบรรยาย: ภาพที่ 4.27 ผู้ใช้การกำหนดหมวดหมู่การท่องเที่ยวที่ผู้ใช้สนใจ

แบบสอบถามใช้ข้อมูลงบประมาณต่อทริป จำนวนสถานที่ต่อวัน 1-10 ช่วงเวลา 4 ช่วง และหมวดหมู่ความสนใจ 13 หมวด โดยเลือกความสนใจได้ไม่เกิน 5 รายการ

4.2.2 การเตรียมข้อมูลทริปและการคำนวณคะแนนย่อย

ข้อมูลทริปที่ใช้จับคู่ประกอบด้วย budget, timeOfDay, category, interestTags และ activityStyle โดย category รวมกับ interestTags ได้สูงสุด 3 หมวด ส่วน activityStyle กำหนดจาก itinerary อัตโนมัติ

คะแนนงบประมาณ: หากทริปฟรีหรือผู้ใช้งบไม่ต่ำกว่างบทริป ได้ 100%; หากงบไม่พอ คำนวณ $(\text{งบผู้ใช้} \div \text{งบทริป}) \times 100$ ตัวอย่าง $1,500 \div 2,000 \times 100 = 75\%$ และ $500 \div 2,000 \times 100 = 25\%$

คะแนนกิจกรรม: $\max(1 - |\text{จำนวนผู้ใช้} - \text{จำนวนทริป}| \div 9, 0) \times 100$ ตัวอย่าง 3 กับ 3 ได้ 100%, 3 กับ 6 ได้ 67% และ 1 กับ 10 ได้ 0% ทริปคำนวณจำนวนกิจกรรมเฉลี่ยจาก itinerary ต่อวัน

คะแนนหมวดหมู่: สร้าง Multi-hot Vector 13 ตำแหน่งจากความสนใจผู้ใช้และหมวดหมู่ทริป แล้วคำนวณ CategoryScore ด้วย Cosine Similarity ตัวอย่างผู้ใช้สนใจ [กาแฟ, อาหาร] และทริปมี [กาแฟ] ได้ $1 \div \sqrt{2} \times 100 \approx 70.71\%$ ปัดเป็น 71%

คะแนนช่วงเวลา: สร้าง Multi-hot Vector 4 ตำแหน่งตามลำดับ [เช้า, กลางวัน, เย็น, กลางคืน] แล้วคำนวณ TimeScore ด้วย Cosine Similarity แยกจากหมวดหมู่ ตัวอย่างผู้ใช้และทริปเลือก [เช้า, เย็น] เหมือนกัน ได้เวกเตอร์ [1, 0, 1, 0] ทั้งสองฝั่ง จึงได้ $2 \div (\sqrt{2} \times \sqrt{2}) \times 100 = 100\%$

คะแนนรวมเป็นค่าเฉลี่ยของคะแนนที่มีข้อมูล ตัวอย่าง Category=71, Budget=75, Activity=67 และ Time=100 ได้ $(71+75+67+100) \div 4 = 78.25$ และปัดเป็น 78%

13:10

42

สร้างทริปใหม่

ข้อมูลทริป

ขั้นตอนที่ 1 จาก 4

สไตล์ของทริป

เลือกได้สูงสุด 3 รายการ เพื่อใช้จับคู่กับความสนใจของผู้ใช้ 1/3

ทะเล

ภูเขา

แคมป์ปิ้ง

ผจญภัย

เที่ยวเมือง

กาแฟ

อาหาร

ช้อปปิ้ง

ถ่ายรูป

คอนเสิร์ต

แสงเลาต์

ไหว้พระ

รายละเอียด

AI ช่วยจัดทริป

ถัดไป

13:12

41

สร้างทริปใหม่

วันและงบประมาณ

ขั้นตอนที่ 2 จาก 4

งบประมาณ (บาท)

จำนวนคน

ระบุจำนวนเงิน

10

ประเภทพบ

ต่อคน

ต่อทริป (รวม)

ช่วงเวลาของทริป

ช่วงเช้า (06:00 - 11:00 น.)
เช่น ชมพระอาทิตย์ขึ้น, เยี่ยมชมตลาดเช้า

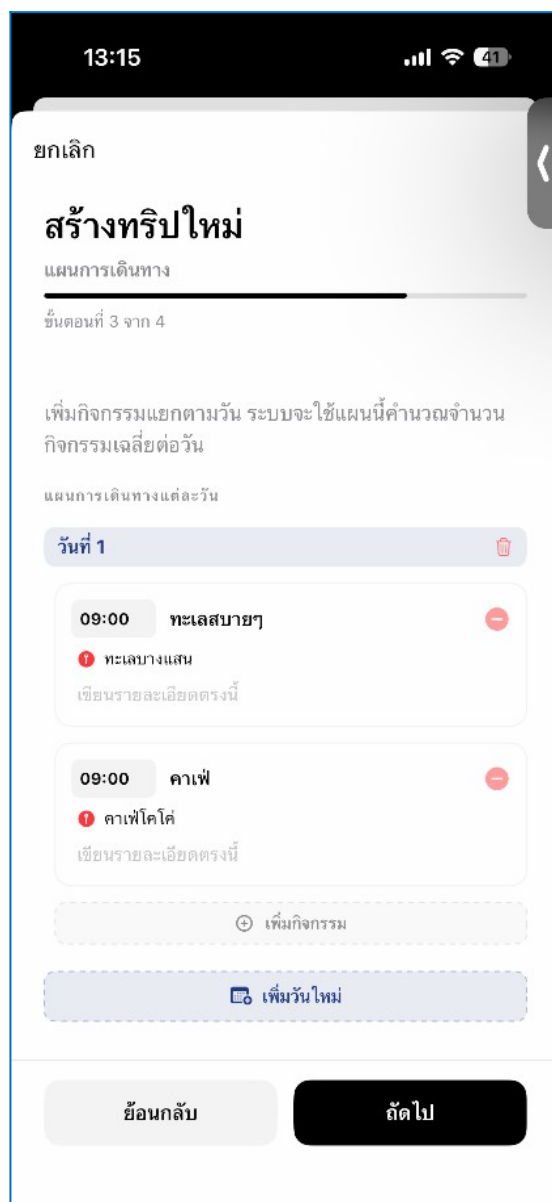
ช่วงกลางวัน (11:00 - 16:00 น.)
เช่น รับประทานอาหาร, พักผ่อนในคาเฟ่, เข้าชมพิพิธภัณฑ์

ช่วงเย็น (16:00 - 20:00 น.)
เช่น เดินพักผ่อน, ชมพระอาทิตย์ตก, รับประทานอาหารค่ำ

ช่วงกลางคืน (20:00 น. เป็นต้นไป)
เช่น ชมแสงสีเมือง, ชมคอนเสิร์ต, รับประทานอาหารค่ำ

ย้อนกลับ

ถัดไป



คำบรรยาย: ภาพที่ 4.27 ข้อมูลผู้ใช้และข้อมูลทริปที่ใช้คำนวณคะแนนย่อยทั้ง 4 ปัจจัย

4.2.3 การคำนวณ Cosine Similarity

การคำนวณ Cosine Similarity แบ่งเป็น 2 กระบวนการ แต่ละกระบวนการใช้ข้อมูลของตนเองและส่งคะแนนย่อยแยกกัน ได้แก่คะแนนหมวดหมู่และคะแนนช่วงเวลา

1) การคำนวณคะแนนหมวดหมู่ความสนใจ

$$\text{CategoryScore} = \text{round}[100 \times (A^c \cdot B^c) \div (||A^c|| \times ||B^c||)]$$

A^c หมายถึงเวกเตอร์ความสนใจของผู้ใช้ 13 ตำแหน่ง และ B^c หมายถึงเวกเตอร์หมวดหมู่ของทริป 13 ตำแหน่ง โดยใช้ลำดับหมวดหมู่เดียวกัน เลือก = 1 และไม่เลือก = 0

$A^c \cdot B^c$ คือผลรวมของผลคูณค่าที่ตำแหน่งเดียวกัน ซึ่งนับจำนวนหมวดหมู่ที่เลือกตรงกัน ส่วน $||A^c||$ และ $||B^c||$ คือรากที่สองของผลรวมกำลังสองของค่าในเวกเตอร์แต่ละชุด ตัวอย่างผู้ใช้เลือก [กาแฟ, อาหาร] และทริปเลือก [กาแฟ] แสดงเฉพาะตำแหน่งกาแฟและอาหาร ได้ $A^c = [1, 1]$ และ $B^c = [1, 0]$ อีก 11 ตำแหน่งเป็น 0 ทั้งสองฝั่ง ผลคูณเชิงจุด = 1 ขนาดเวกเตอร์ผู้ใช้ = $\sqrt{2}$ และขนาดเวกเตอร์ทริป = 1 จึงได้ $\text{CategoryScore} = \text{round}(100 \times 1 \div \sqrt{2}) = 71\%$

2) การคำนวณคะแนนช่วงเวลาที่ชอบทำกิจกรรม

$$\text{TimeScore} = \text{round}[100 \times (A^t \cdot B^t) \div (||A^t|| \times ||B^t||)]$$

A^t หมายถึงเวกเตอร์ช่วงเวลาของผู้ใช้ และ B^t หมายถึงเวกเตอร์ช่วงเวลาของทริป แต่ละชุดมี 4 ตำแหน่งตามลำดับ [เช้า, กลางวัน, เย็น, กลางคืน] เลือก = 1 และไม่เลือก = 0

ตัวอย่างผู้ใช้และทริปเลือก [เช้า, เย็น] เหมือนกัน จึงได้ $A^t = [1, 0, 1, 0]$ และ $B^t = [1, 0, 1, 0]$ ผลคูณเชิงจุด = 2 และขนาดเวกเตอร์ทั้งสอง = $\sqrt{2}$ จึงได้ $\text{TimeScore} = \text{round}(100 \times 2 \div (\sqrt{2} \times \sqrt{2})) = 100\%$

หากทริปเลือกเฉพาะ [เช้า] จะได้ $B^t = [1, 0, 0, 0]$ ผลคูณเชิงจุด = 1 และขนาดเวกเตอร์ทริป = 1 คะแนนช่วงเวลาก็กลายเป็น $\text{round}(100 \times 1 \div \sqrt{2}) = 71\%$

หากเวกเตอร์ด้านใดไม่มีข้อมูลหรือมีค่าเป็นศูนย์ทุกตำแหน่ง ระบบเว้นคะแนนย่อยด้านนั้นเป็น null และไม่นำไปรวมในตัวหารของ Match Score สำหรับเวกเตอร์ที่มีข้อมูล ระบบจำกัดค่า Cosine ให้อยู่ในช่วง 0-1 คูณ 100 และปัดเป็นจำนวนเต็ม

คะแนนหมวดหมู่บอกความสอดคล้องของรายการความสนใจ ส่วนคะแนนช่วงเวลาบอกความสอดคล้องของช่วงที่ชอบทำกิจกรรม แต่ละคะแนนอยู่ในช่วง 0-100 และนำไปคำนวณค่าเฉลี่ยร่วมกับคะแนนงบประมาณและคะแนนกิจกรรม

ระบบส่ง matchBreakdown ได้แก่ category, budget, activityStyle, timeOfDay และ groupMatch พร้อม matchScore ซึ่งเป็นค่าเฉลี่ยของคะแนนย่อยที่คำนวณได้

คะแนนงบประมาณลดลงตามสัดส่วนระหว่างงบผู้ใช้งบกับงบทริปเมื่อผู้ใช้งบต่ำกว่าค่าทริป แล้วนำไปรวมกับคะแนนย่อยด้านอื่นด้วยค่าเฉลี่ย

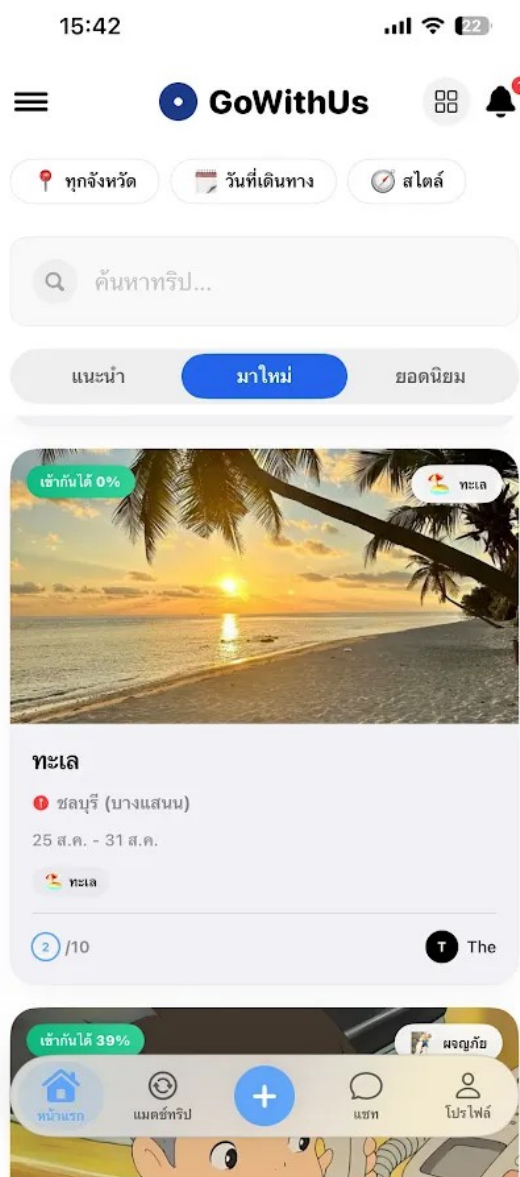
4.2.4 ผลลัพธ์ Match Score

หลังคำนวณ Backend จะเพิ่ม matchScore และ matchBreakdown ลงใน ข้อมูลทริป เรียงทริปจากคะแนนมากไปน้อย แล้วส่งผลลัพธ์ในรูปแบบ JSON ไปแสดงใน แอปพลิเคชัน

หน้ารายการทริปแสดง “เข้ากันได้ X%” จากค่าเฉลี่ยของคะแนนย่อยที่มีข้อมูล

กรณีทดสอบ	Budget	Activity	Category	Time	Match Score
ข้อมูลตรงกันทุกด้าน	100%	100%	100%	100%	100%
ข้อมูลแตกต่างกันสูง	25%	67%	0%	0%	23%

Cosine Similarity ใช้เฉพาะหมวดหมู่และช่วงเวลา งบประมาณใช้สูตรอัตราส่วน กิจกรรมใช้สูตรระยะห่างบนช่วง 1-10 แล้วระบบหาค่าเฉลี่ยคะแนนย่อย



คำบรรยาย: ภาพที่ 4.28 การแสดง Match Score บนหน้ารายการทริปแนะนำ

ภาพหน้าจอแสดงตำแหน่งและรูปแบบการนำเสนอคะแนนความเข้ากันได้บนรายการทริป โดยตัวอย่างในภาพมีค่า 0% ภาพนี้จึงใช้ประกอบการอธิบายส่วนติดต่อผู้ใช้ ไม่ใช่ยืนยันความแม่นยำหรือการกระจายของคะแนนระบบ

4.3 ผลการทดสอบระบบ

การทดสอบระบบ GoWithUs มีวัตถุประสงค์เพื่อตรวจสอบว่าฟังก์ชันแต่ละส่วนสามารถทำงานได้ตามข้อกำหนด รวมถึงตรวจสอบการทำงานร่วมกันระหว่างแอปพลิเคชัน ส่วน Backend API และฐาน

ข้อมูล การทดสอบแบ่งออกเป็น 4 ระดับ ได้แก่ Unit Testing, Integration Testing, System Testing และ User Acceptance Testing

4.3.1 ผลการทดสอบ Unit Testing

Unit Testing เป็นการตรวจสอบฟังก์ชันหรือหน่วยการทำงานย่อย โครงการนี้มีสคริปต์ตัวอย่างในโฟลเดอร์ backend-api สำหรับเรียกคำนวณ Smart Matching ได้แก่

test_budget.js

test_matching.js

test_match_cases.js

test_match_bug.js

test_userb_trip.js

สคริปต์ดังกล่าวเรียกใช้ฟังก์ชัน calculateTripCompatibilityDetailed กับข้อมูลตัวอย่างหลายกรณี แต่ยังไม่มีการ assertion หรือรายงานผลรันอัตโนมัติที่ตรวจสอบย้อนกลับได้ ตารางที่ 4.30 จึงแสดงกรณีทดสอบและผลที่คาดหวัง ไม่ใช่ผลยืนยันว่าทุกกรณีผ่านการทดสอบ

รหัสทดสอบ	กรณีทดสอบ	ผลที่คาดหวัง
UT-01	ข้อมูลผู้ใช้ตรงกับทริปทุกด้าน	ได้ Match Score ระดับสูง
UT-02	งบประมาณและกิจกรรมตรง แต่หมวดหมู่และเวลาไม่ตรง	คะแนนต่ำกว่ากรณี UT-01
UT-03	งบประมาณแตกต่างกันมาก	คะแนนด้าน Budget ลดลง
UT-04	ระดับกิจกรรมแตกต่างกันมาก	คะแนนด้าน Activity Style ลดลง
UT-05	หมวดหมู่ทริปไม่ตรงกับความสนใจ	คะแนนด้าน Category เป็น 0 หรือลดลง
UT-06	ช่วงเวลาของวันไม่ตรงกัน	คะแนนด้าน Time of Day เป็น 0
UT-07	ข้อมูลไม่ตรงกันทุกด้าน	ได้ Match Score ระดับต่ำ

ตัวอย่าง เมื่อ Category=0%, Budget=25%, Activity=67% และ Time=0% คะแนนรวมเท่ากับ $(0+25+67+0) \div 4 = 23\%$ หากปัจจัยใดไม่มีข้อมูลระบบจะไม่นำค่า null มาหารรวม

ผลดังกล่าวแสดงให้เห็นว่าคะแนนลดลงตามจำนวนปัจจัยและระดับความแตกต่างของข้อมูล โดยสามารถตรวจสอบที่มาของผลลัพธ์ได้จากคะแนนย่อยใน matchBreakdown

4.3.2 ผลการทดสอบ Integration Testing

Integration Testing เป็นการตรวจสอบการทำงานร่วมกันระหว่างส่วนประกอบของระบบ GoWithUs ซึ่งประกอบด้วยแอปพลิเคชัน iOS หรือ Web Frontend, Backend API และฐานข้อมูล PostgreSQL ผ่าน Prisma

ลำดับการรับส่งข้อมูลโดยรวมเป็นดังนี้

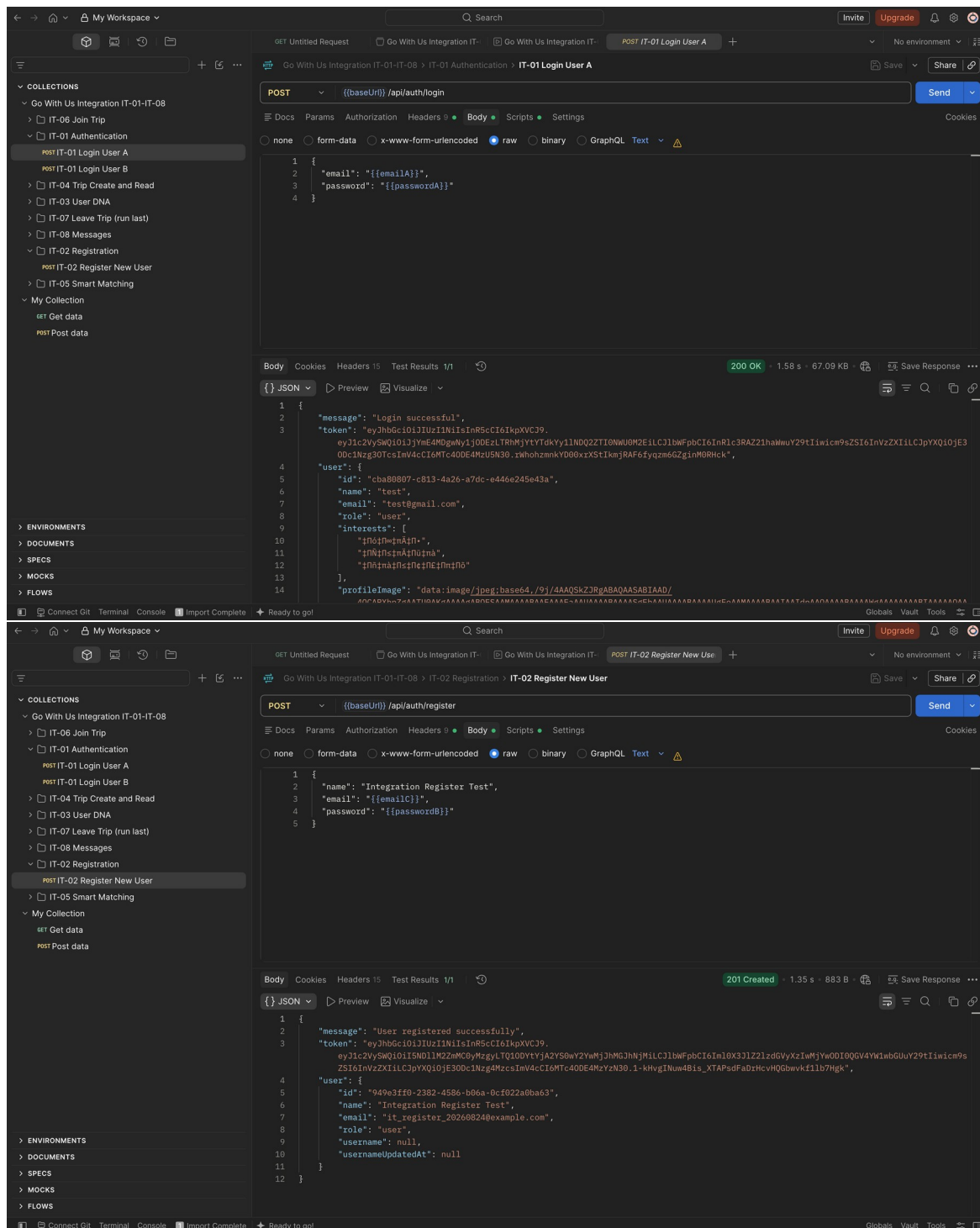
Application > Backend API > Prisma > PostgreSQL

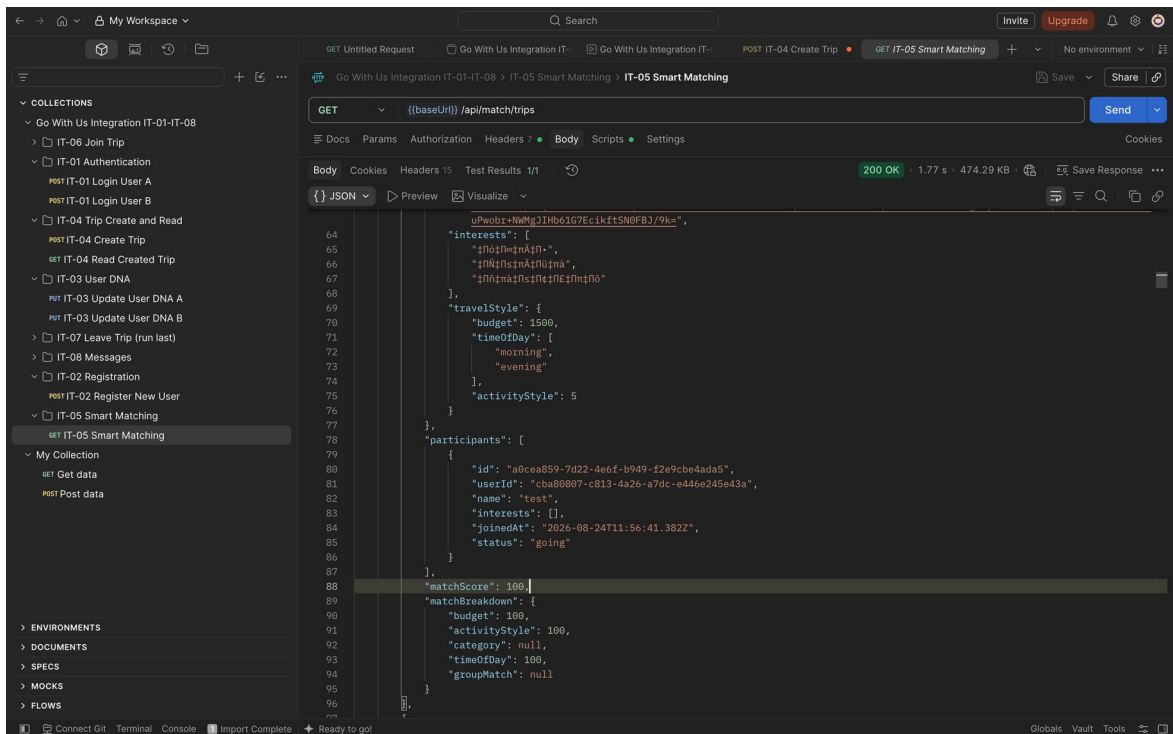
เมื่อผู้ใช้งานดำเนินการผ่านส่วนติดต่อผู้ใช้ แอปพลิเคชันจะส่งคำขอไปยัง Backend API จากนั้น Backend จะตรวจสอบข้อมูล ประมวลผลตามกฎของระบบ และเรียกใช้ Prisma เพื่ออ่านหรือบันทึกข้อมูลในฐานข้อมูล เมื่อดำเนินการเสร็จแล้ว Backend จะส่งผลลัพธ์กลับไปแสดงในแอปพลิเคชันตารางที่ 4.32 แสดงรายการทดสอบการทำงานร่วมกันของระบบ

รหัสทดสอบ	กระบวนการ	ส่วนที่ทำงานร่วมกัน	ผลที่คาดหวัง
IT-01	เข้าสู่ระบบ	App > API > Database	ตรวจสอบบัญชีและได้รับ JWT Token
IT-02	สมัครสมาชิก	App > API > Database	สร้างข้อมูลผู้ใช้ใหม่ได้
IT-03	แก้ไข User DNA	App > API > Database	บันทึก Interests และ Travel Style ได้
IT-04	สร้างทริป	App > API > Database	บันทึกทริปและเรียกดูทริปได้
IT-05	คำนวณ Smart Matching	App > API > Matching Function > Database	ได้ Match Score ของแต่ละทริป
IT-06	เข้าร่วมทริป	App > API > Database	เพิ่มผู้ใช้ในรายชื่อผู้เข้าร่วม
IT-07	ออกจากทริป	App > API > Database	นำผู้ใช้ออกจากรายชื่อผู้เข้าร่วม
IT-08	ส่งข้อความ	App > API/WebSocket > Database	บันทึกและแสดงข้อความในห้องแชต

ผลการตรวจสอบโครงสร้างโปรแกรมพบว่า Frontend และ Native iOS ติดต่อกับ Backend ผ่าน Service ของแต่ละฟังก์ชัน เช่น AuthService, TripService, MatchService และ MessageService ส่วน Backend

ติดต่อฐานข้อมูลผ่าน Prisma Client แสดงว่าระบบมีการแบ่งหน้าที่ระหว่างส่วนติดต่อผู้ใช้ การประมวลผล และการจัดเก็บข้อมูลอย่างชัดเจน

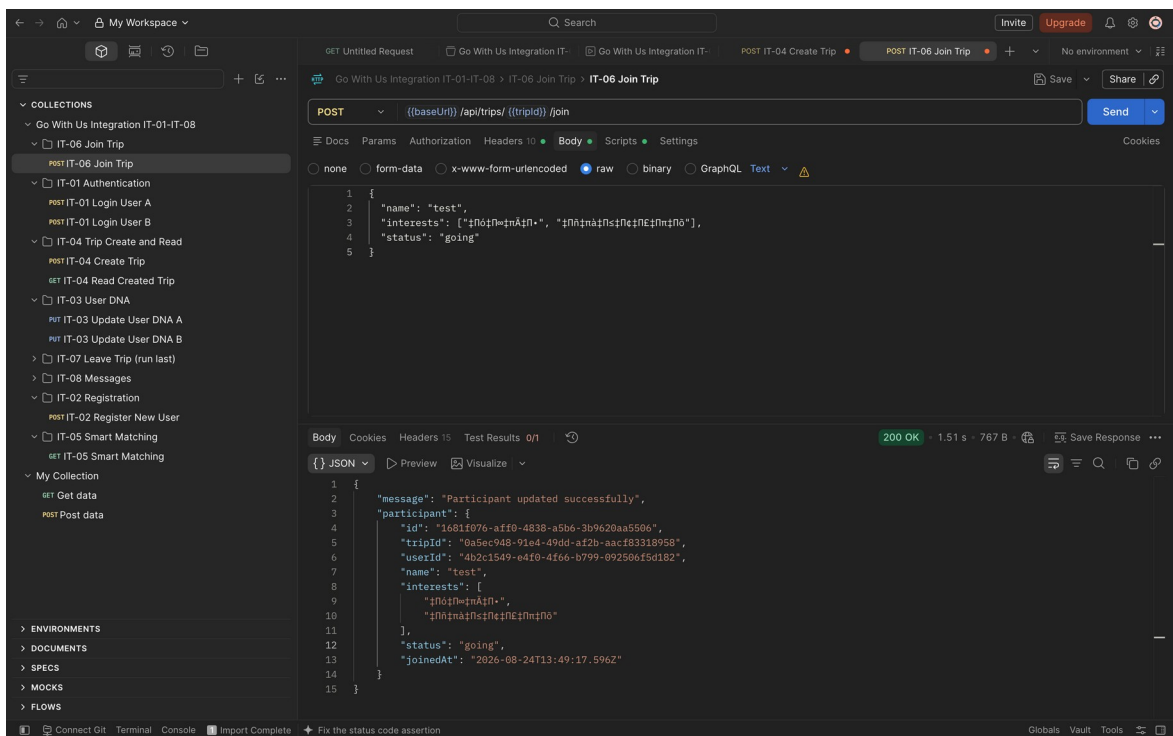




GET /api/match/trips

200 OK · 1.77 s · 474.29 KB

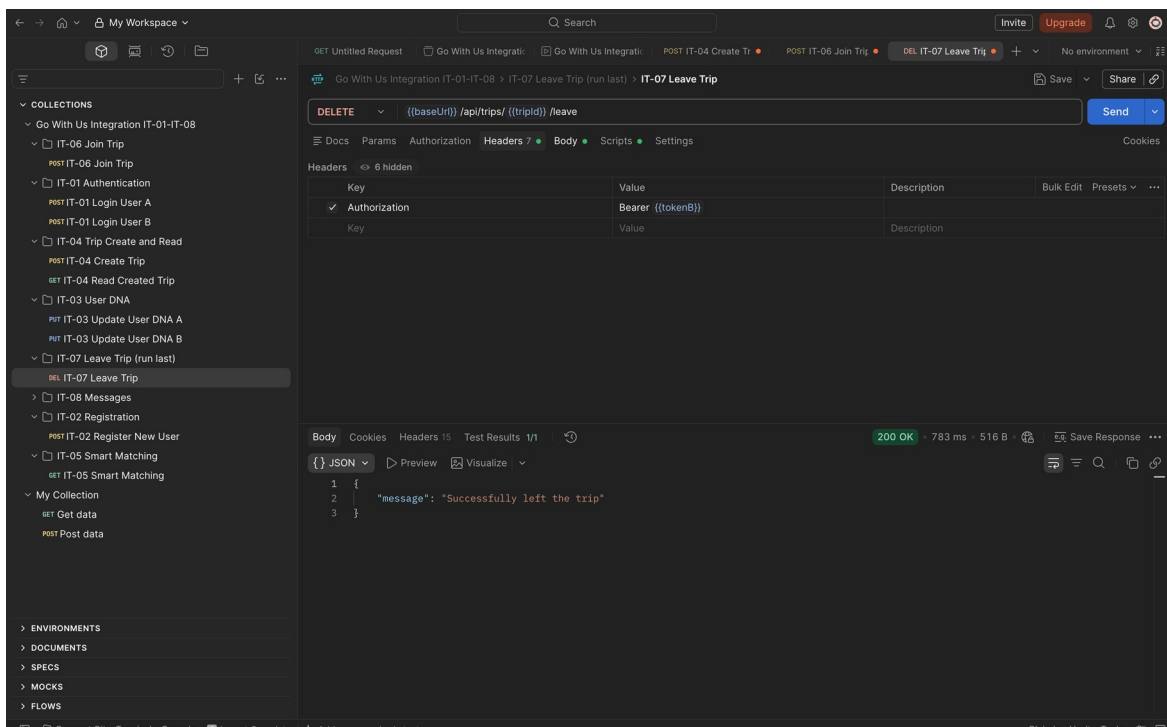
```
{
  "UPwobr+NwMgJlHb61G/Ec1kftSN0F8J/9k=",
  "interests": [
    "iN0iNwMgJlHb61G/Ec1kftSN0F8J/9k=",
    "iN0iNwMgJlHb61G/Ec1kftSN0F8J/9k=",
    "iN0iNwMgJlHb61G/Ec1kftSN0F8J/9k="
  ],
  "travelStyle": {
    "budget": 1500,
    "timeOfDay": [
      "morning",
      "evening"
    ],
    "activityStyle": 5
  },
  "participants": [
    {
      "id": "a0cea859-7d22-4e6f-b949-f2e9cbe4ada5",
      "userId": "cba80807-c813-4a26-a7dc-e446e245e43a",
      "name": "test",
      "interests": [],
      "joinedAt": "2026-08-24T11:56:41.382Z",
      "status": "going"
    }
  ],
  "matchScore": 100,
  "matchBreakdown": {
    "budget": 100,
    "activityStyle": 100,
    "category": null,
    "timeOfDay": 100,
    "groupMatch": null
  }
}
```



POST /api/trips/{{tripId}}/join

200 OK · 1.51 s · 767 B

```
{
  "message": "Participant updated successfully",
  "participant": {
    "id": "1681f076-aff0-4830-a5b6-3b9620aa5506",
    "tripId": "0a5ec948-91e4-49dd-af2b-aacf83318958",
    "userId": "4b2c15d9-e4f0-4f66-b799-092506f5d182",
    "name": "test",
    "interests": [
      "iN0iNwMgJlHb61G/Ec1kftSN0F8J/9k=",
      "iN0iNwMgJlHb61G/Ec1kftSN0F8J/9k="
    ],
    "status": "going",
    "joinedAt": "2026-08-24T13:49:17.596Z"
  }
}
```



My Workspace

Go With Us Integration IT-01-IT-08 > IT-07 Leave Trip (run last) > **IT-07 Leave Trip**

DELETE `{{baseUri}}/api/trips/{{tripId}}/leave` **Send**

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

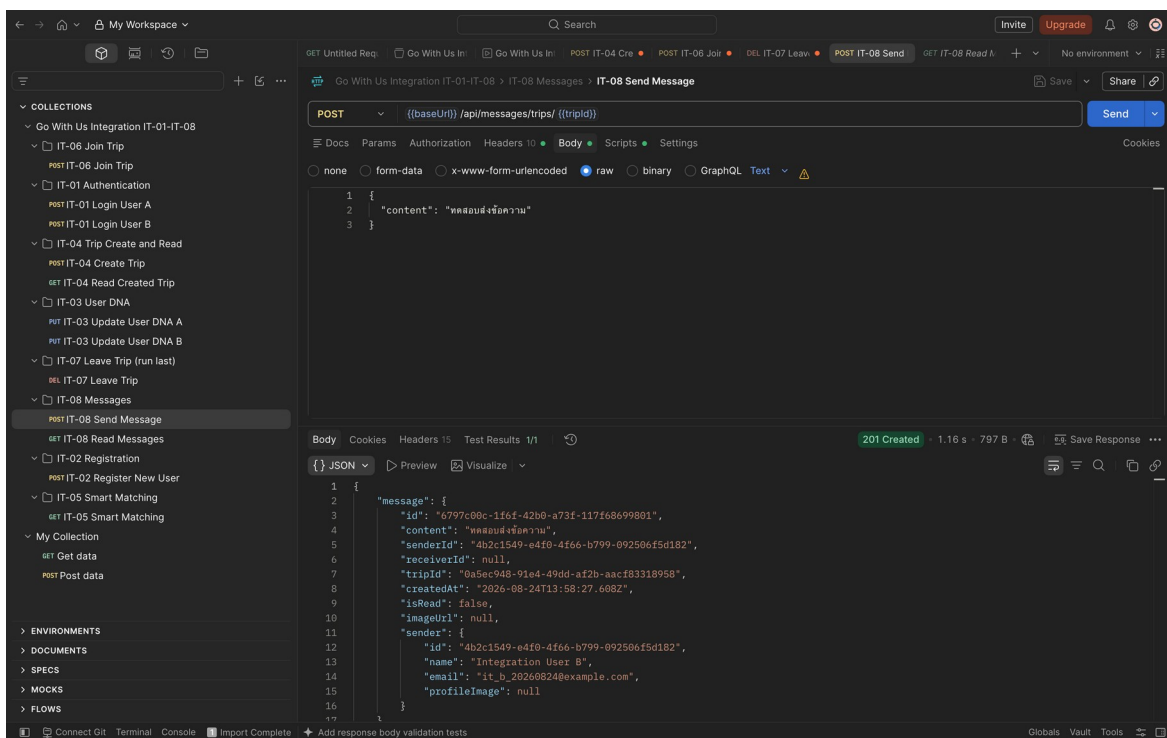
Headers 6 hidden

Key	Value	Description	Bulk Edit	Presets	...
✓ Authorization	Bearer {{tokenB}}				
Key	Value	Description			

Body Cookies Headers 15 Test Results 1/1 **200 OK** · 783 ms · 516 B **Save Response**

JSON Preview Visualize

```
1 {
2   "message": "Successfully left the trip"
3 }
```



My Workspace

Go With Us Integration IT-01-IT-08 > IT-08 Messages > **IT-08 Send Message**

POST `{{baseUri}}/api/messages/trips/{{tripId}}` **Send**

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL Text

```
1 {
2   "content": "ทดสอบส่งข้อความ"
3 }
```

Body Cookies Headers 15 Test Results 1/1 **201 Created** · 1.16 s · 797 B **Save Response**

JSON Preview Visualize

```
1 {
2   "message": {
3     "id": "6797c00c-1f6f-42b0-a73f-117f68699801",
4     "content": "ทดสอบส่งข้อความ",
5     "senderId": "4b2c1549-e4f0-4f66-b799-092586f5d182",
6     "receiverId": null,
7     "tripId": "0a5ec948-91e4-49dd-af2b-aacf83318958",
8     "createdAt": "2026-08-24T13:58:27.688Z",
9     "isRead": false,
10    "imageUrl": null,
11    "sender": {
12      "id": "4b2c1549-e4f0-4f66-b799-092586f5d182",
13      "name": "Integration User B",
14      "email": "it_b_20260824@example.com",
15      "profileImage": null
16    }
17  }
18 }
```

APPLICATION > API > DATABASE



สมัครสมาชิก

เริ่มต้นการผจญภัยของคุณ

ชื่อ

Users1

อีเมล

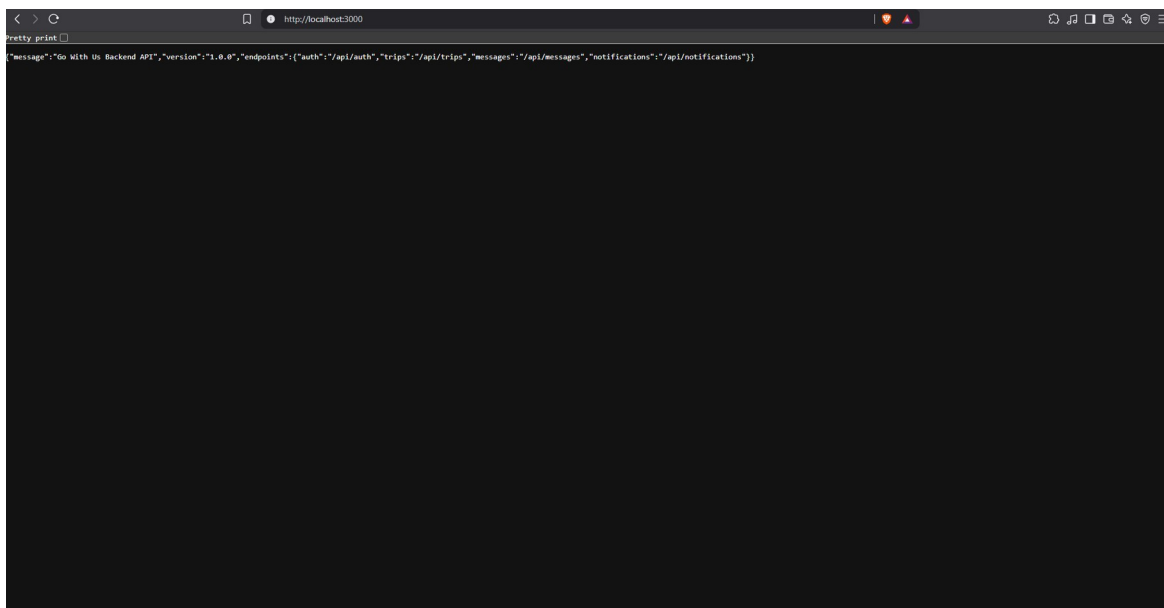
users1@gmail.com

รหัสผ่าน



ยืนยันรหัสผ่าน





id	username	usernameUpdatedAt	name	email	password	role
1bb32e3b-cd76-47fb-aa78-c285958e	NULL	NULL	Pondmin	ppk@gmail.com	\$2b\$10\$6oQZKzc3o2eq9KxtStFuFin	user
29267094-453a-48a4-8a2b-cd483be1	NULL	NULL	Users1	users@gmail.com	\$2b\$10\$Ar0htqXGf8LqkS67aePumvtI	user
32b7647a-5526-46a0-a7a9-512b6db7	jundjan	2026-07-16 07:56:17.528	Test2	test@test.test	\$2b\$10\$SS33BmI890XkqX2dFjR0d	user
358cd4de-f691-4b42-847c-9065b30a	NULL	NULL	Bot Tester	test-bot-xyz-12345@gmail.com	\$2b\$10\$5iOefGnqkELV9EBCCHJbuqM	user
4035d55c-17b6-4762-91bf-5d720009	NULL	NULL	Pondz	jk@gmail.com	\$2b\$10\$d2ZEU6v9b9uEpdqH0Cr9PulE	user
4180ea43-1fed-4384-9087-ea7ba187	Pondz	2026-08-10 05:40:44.124	Pppp	ppp@gmail.com	\$2b\$10\$et87baxmHWTDQzgrEly3wR	user
4b2c1549-e4f0-4f66-b799-092506f5	NULL	NULL	Integration User B	it_b_20260824@example.com	\$2b\$10\$HeqLZEEID5QaHXKccCdAteC	user
689c0605-0977-47e9-90d5-dee4a10c	ponx	2026-08-10 06:03:15.411	Kkkii	ppok@gmail.com	\$2b\$10\$K5BL4n.bOofw.NVPRaBeOew	user
69a4ab42-782d-4497-8898-8ba16fb3	NULL	NULL	Integration User A	it_a_20260824@example.com	\$2b\$10\$HeqLZEEID5QaHXKccCdAteC	user
6c0aa55c-af05-434a-8c09-e6661a5f	user1	2026-08-14 04:37:45.627	User1	user@gmail.com	\$2b\$10\$wq997HdbUVVrm8XzJcJcu	user
70d3a365-50cb-4227-b183-08b9959f	NULL	NULL	pondxz	jak@gmail.com	\$2b\$10\$J3.1A18H3D53DcJg8APQuX9	user
823efc0a-2611-4ce3-b0f0-1d41db481	admin	2026-08-08 14:51:53.638	Admin	admin@gowithus.com	\$2b\$10\$RwCj7Rp38CJbLV8MwUA.y	admin
949e3f0f-2382-4586-b06a-0cf022a0f	NULL	NULL	Integration Register Test	it_register_20260824@example.com	\$2b\$10\$xtHguqJy3Zy06/ONN7/MOk	user
967b2f9e-389d-48bd-9a47-cf514c67	Pondki	2026-08-24 10:30:55.845	Pondki	kij@gmail.com	\$2b\$10\$dZrfaseX8Wp0JkVPHekXT	user
9820ef90-6ab7-48cc-8658-bc5862e	NULL	NULL	maxxxx	pom@gmail.com	\$2b\$10\$DWhwp92VnDA668AHKMaQu	user
9de55196-95be-4d51-80d2-9acce4cd	NULL	NULL	Userstest	ustertest@gmail.com	\$2b\$10\$8Xv70wXof2j8PnE74eyOMg	user
aa577bf1-d6ea-4307-bc06-86ffef544	pond	2026-08-10 05:49:04.568	Pond	kkk@gmail.com	\$2b\$10\$2TnT72050Jb.J2ew.RPYmCuk	user
ac42dc7-4220-4976-85d7-6870312c	NULL	NULL	Ohm	o@gmail.com	\$2b\$10\$w50PX8aUxzaiFT3RouJf	user
b94374c-4ba8-46a6-bbf6-a420d280	NULL	NULL	KaNomto_om	gugugaga@gmail.com	\$2b\$10\$Xo7727Ez3cxaDy89eQnL	user
c753a9d9-1c39-4724-8316-ebf5ba55f	NULL	NULL	Test User	user@gowithus.com	\$2b\$10\$8mMndcOrA7URauuolYyTo	user

คำบรรยาย: ภาพที่ 4.29 โครงสร้างการทดสอบการทำงานร่วมกันของระบบ

4.3.3 ผลการทดสอบ System Testing แบบ Black Box

System Testing เป็นการตรวจสอบลำดับการใช้งานจากข้อมูลนำเข้าและผลที่ปรากฏต่อผู้ใช้ โดยไม่พิจารณารายละเอียดโค้ดภายใน ตารางต่อไปกำหนดกรณีทดสอบตั้งแต่สมัครสมาชิก กรอกข้อมูลความชอบ สร้างหรือค้นหาคำทริป ตรวจสอบ Match Score เข้าร่วมทริป และใช้ห้องสนทนา แต่ยังไม่มีการรันจริงครบทุกกรณี

ลำดับการทดสอบแบบ End-to-End กำหนดดังนี้

ตารางที่ 4.34 แสดงกรณีทดสอบการทำงานของระบบแบบ Black Box

รหัส	ขั้นตอนทดสอบ	ผลที่คาดหวัง
ST-01	กรอกข้อมูลสมัครสมาชิกครบถ้วน	ระบบสร้างบัญชีและเข้าสู่ระบบได้

รหัส	ขั้นตอนทดสอบ	ผลที่คาดหวัง
ST-02	กรอกข้อมูลไม่ครบหรือใช้อีเมลซ้ำ	ระบบปฏิเสธและแสดงข้อความแจ้งเตือน
ST-03	กำหนดงบประมาณ ระดับกิจกรรม เวลา และความสนใจ	ระบบบันทึก User DNA ได้
ST-04	สร้างทริปด้วยข้อมูลครบถ้วน	ทริปปรากฏในรายการทริป
ST-05	เปิดหน้าแนะนำทริป	ระบบแสดงและเรียงทริปตาม Match Score
ST-06	เลือกเข้าร่วมทริปที่ยังไม่เต็ม	ระบบเพิ่มผู้ใช้เป็นสมาชิกทริป
ST-07	เลือกเข้าร่วมทริปที่สมาชิกเต็ม	ระบบไม่อนุญาตให้เข้าร่วม
ST-08	ออกจากทริป	รายชื่อสมาชิกและจำนวนผู้เข้าร่วมถูกปรับปรุง
ST-09	ส่งข้อความในห้องแชต	ผู้เข้าร่วมเห็นข้อความที่ส่ง

13:30

←

สมัครสมาชิก

เริ่มต้นการผจญภัยของคุณ

ชื่อ

Pondg

อีเมล

pon@gmail.com

รหัสผ่าน

•••••

ยืนยันรหัสผ่าน

•••••

สมัครสมาชิก

13:32

←

สมัครสมาชิก

เริ่มต้นการผจญภัยของคุณ

ชื่อ

The

อีเมล

ppp

รหัสผ่าน

•••••

ยืนยันรหัสผ่าน

•••••

รูปแบบอีเมลไม่ถูกต้อง

สมัครสมาชิก

13:30

แบบสอบถาม

ข้าม

ขั้นตอนที่ 1 จาก 5


ข้อมูลส่วนตัวของคุณ

ข้อมูลนี้ช่วยให้ผู้ร่วมทริบ์จิกคุณ และใช้สร้างโปรไฟล์ของคุณ

Username

@ username

ใช้อักษรอย่างน้อย 3 ตัว และต้องไม่ซ้ำกับผู้อื่น

วันเกิด

เลือกวันเกิด

25 ส.ค. 2551

อายุ 18 ปี

13:33


ยืนยันอีเมล

กรุณากรอกรหัส 6 หลักที่ส่งไปที่
ppl@gmail.com

ยืนยันรหัส

ไม่ได้รับรหัส? ส่งใหม่ใน 57 วิ

ถัดไป

คำบรรยาย: “ภาพที่ 4.30 การทดสอบสมัครสมาชิก”

13:42

57%

แบบสอบถาม

ข้าม

ขั้นตอนที่ 2 จาก 5

💰 งบประมาณเฉลี่ยต่อวัน

(Budget / บาท)

ระบุงบประมาณที่คุณพึงพอใจในการใช้จ่ายระหว่างทริป (ต่อวัน)

1,500 ฿

ประหยัด (100฿)

หรูหรา (5,000฿+)

ย้อนกลับ

ถัดไป

13:42

57%

แบบสอบถาม

ข้าม

ขั้นตอนที่ 3 จาก 5

🎯 วันนีงชอบเที่ยวประมาณกี่ที่?

สไลด์การจัดตารางเที่ยวในแต่ละวันของคุณเป็นแบบไหน?

1-2 ที่ (สายชิล)

เน้นพักผ่อน อยู่แต่ยาวนานๆ ไม่รีบร้อน

☐

3-4 ที่ (สายพอดี้)

เที่ยวสบายๆ ได้ไปหลายที่แต่ไม่เหนื่อยเกินไป

☐

5 ที่ขึ้นไป (สายลุย)

ตารางแน่น เน้นเก็บใจครบ ไปหลายที่

☒

ย้อนกลับ

ถัดไป

13:42

แบบสอบถาม

ข้าม

ขั้นตอนที่ 4 จาก 5

🕒 ช่วงเวลาที่ชอบท่องเที่ยว (Time of Day)

เลือกช่วงเวลาที่ชอบออกไปทำกิจกรรมหรือท่องเที่ยว (เลือกได้มากกว่า 1 ช่วง)

ช่วงเช้า (06:00 - 11:00 น.)
เช่น ชมพระอาทิตย์ขึ้น, เยี่ยมชมตลาดเช้า

ช่วงกลางวัน (11:00 - 16:00 น.)
เช่น รับประทานอาหาร, พักผ่อนในคาเฟ่, เข้าชมพิพิธภัณฑ์

ช่วงเย็น (16:00 - 20:00 น.)
เช่น เดินพักผ่อน, ชมพระอาทิตย์ตก, รับประทานอาหารค่ำ

ช่วงกลางคืน (20:00 น. เป็นต้นไป)
เช่น สัมผัสบรรยากาศยามค่ำคืน, เข้าร่วมงานสังสรรค์

ย้อนกลับ

ถัดไป

13:42

แบบสอบถาม

ข้าม

ขั้นตอนที่ 5 จาก 5

✨ สไตล์การท่องเที่ยวของคุณ

เลือกสไตล์การท่องเที่ยวที่คุณชอบ (เลือกได้สูงสุด 5 ข้อ) เพื่อให้เราแนะนำทริปที่โดนใจคุณมากที่สุด

🌿 สายธรรมชาติ & ผจญภัย

ทะเล

ภูเขา

แคมป์ปิ้ง

ผจญภัย

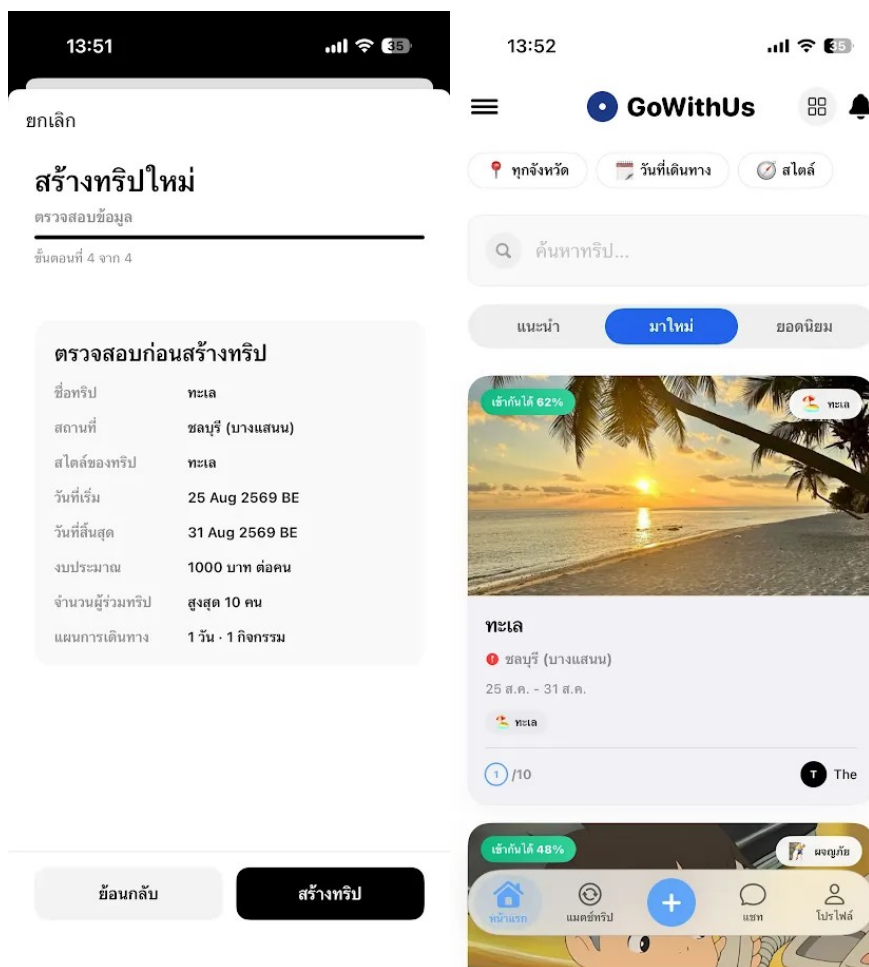
🏙️ สายเมือง &ไลฟ์สไตล์

ย้อนกลับ

เสร็จสิ้น



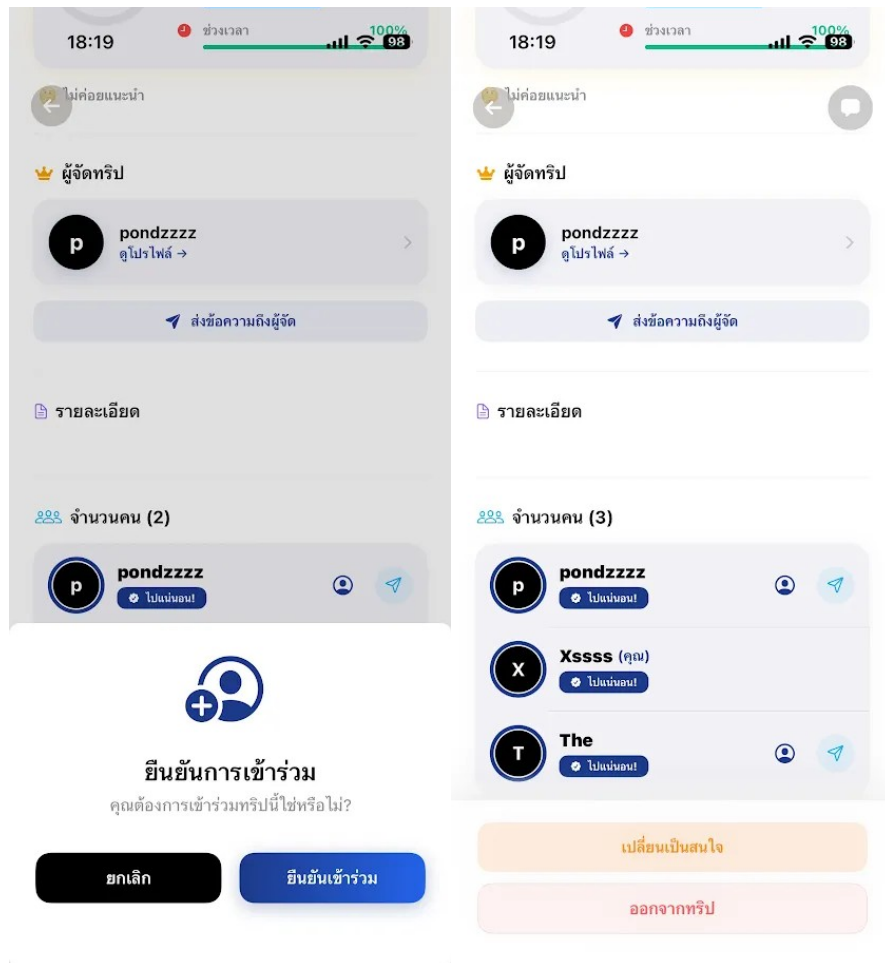
คำบรรยาย: “ภาพที่ 4.31 การกำหนดความสนใจสำหรับสร้าง User DNA”



คำบรรยาย: “ภาพที่ 4.32 การทดสอบสร้างทริป”



คำบรรยาย: “ภาพที่ 4.33 การแสดง Match Score บนหน้ารายการทริป”



14:10

←



ทริปทดสอบ Integration

📌 ชลบุรี (พัทยา)

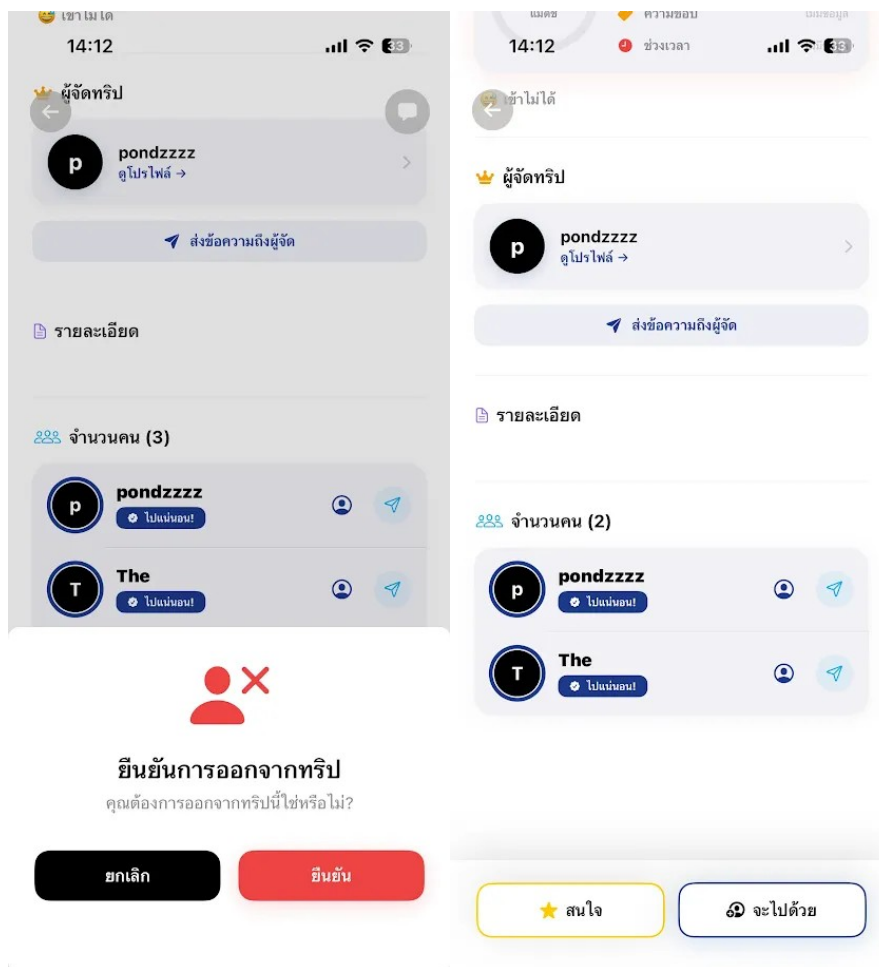
🍷 ทะเล 🍷 ดำน้ำ 🍷 อาหาร

• เต็มแล้ว

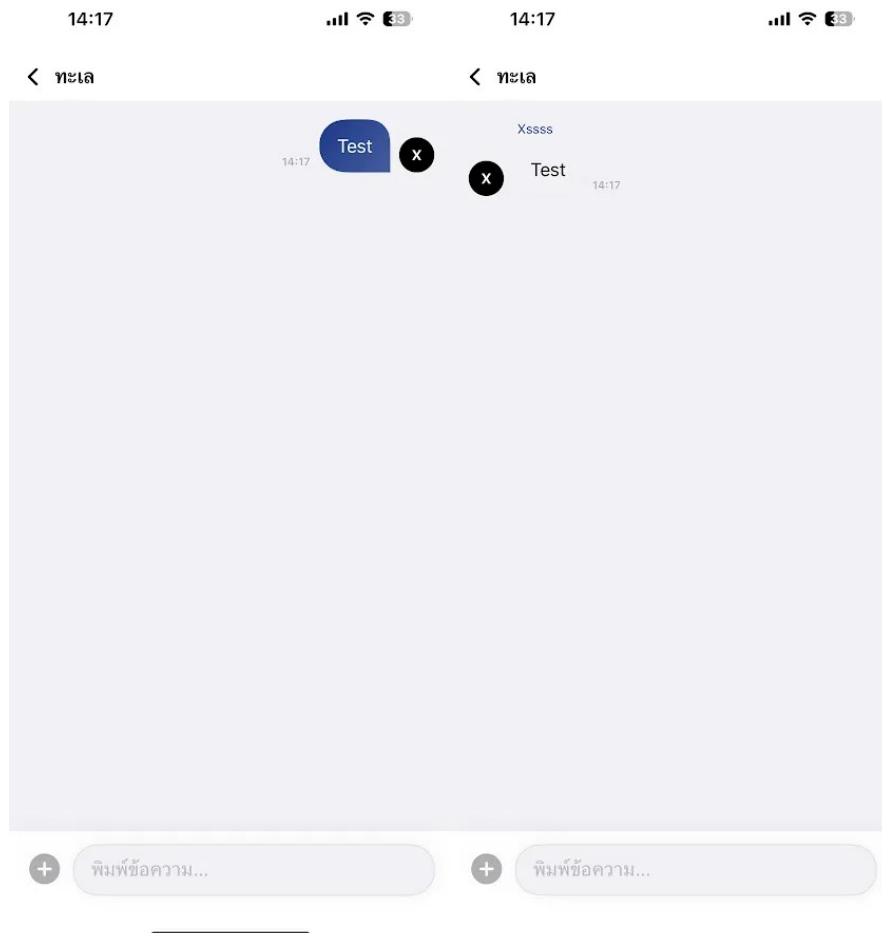
 งบประมาณ (ต่อคน) 1,000 B	 จำนวนคน 4/4	 ระยะเวลา 20 ก.ย. - 20 ก.ย.
---	--	---

♥ ความเข้ากันของคุณ

ทริปเต็มแล้ว



คำบรรยาย: “ภาพที่ 4.34 การทดสอบดูรายละเอียดและเข้าร่วมทริป”



คำบรรยาย: “ภาพที่ 4.35 การทดสอบระบบสนทนา”

4.3.4 ขอบเขตการประเมินการยอมรับของผู้ใช้

การประเมินการยอมรับของผู้ใช้ควรให้กลุ่มเป้าหมายทำภารกิจจริง ได้แก่ สมัครสมาชิก ระบุความต้องการ สร้างและค้นหาทริป ตรวจสอบคะแนน เข้าร่วมทริป และใช้ห้องสนทนา แล้วบันทึกจำนวนผู้ทำสำเร็จ ปัญหาที่พบ และข้อเสนอแนะ

4.4 ตัวอย่างการตรวจสอบสูตร Smart Matching

หัวข้อนี้แสดงกรณีตัวอย่างเพื่ออธิบายผลของฟังก์ชัน `calculateTripCompatibilityDetailed` ใน Backend โดยเปรียบเทียบข้อมูลผู้ใช้งานกับทริปตามคะแนนย่อย 4 ปัจจัย การแสดงค่าที่คำนวณตามสูตรไม่ใช่หลักฐานการรันชุดทดสอบอัตโนมัติ

ระบบ Smart Matching ใช้ Category, Budget, Activity Style และ Time of Day โดย Category กับ Time ใช้ Cosine Similarity, Budget ใช้อัตราส่วน และ Activity ใช้ระยะห่างบนช่วง 1-10 ก่อนหาค่าเฉลี่ยของคะแนนที่มีข้อมูล

Cosine Similarity ใช้แปลงความสอดคล้องของหมวดหมู่และช่วงเวลาเป็นคะแนนย่อย ส่วน **Match Score** คำนวณจากค่าเฉลี่ยของคะแนนย่อยที่มีข้อมูลและปัดเป็นจำนวนเต็ม

ค่าที่เข้าใกล้ร้อยละ 100 หมายถึงข้อมูลผู้ใช้และทริปมีความสอดคล้องกันสูง ส่วนค่าที่เข้าใกล้ร้อยละ 0 หมายถึงมีความสอดคล้องกันต่ำ

4.4.1 กรณีข้อมูลมีความคล้ายคลึงสูง

ตัวอย่างกำหนดให้ผู้ใช้และทริปมีงบประมาณ 2,000 บาท ต้องการเที่ยว 3 สถานที่ต่อวัน สนใจกาแฟ และเลือกช่วงเช้ากับกลางวันตรงกัน

เมื่อข้อมูลของทั้งสองฝ่ายตรงกันในแต่ละด้าน คะแนนย่อยของทั้ง 4 ปัจจัยเท่ากับ 100%

เมื่อข้อมูลทั้ง 4 ด้านตรงกัน คะแนนย่อยทั้งหมดเท่ากับ 100% ดังนั้นค่าเฉลี่ย $(100+100+100+100) \div 4 = 100\%$

ปัจจัย	ข้อมูลผู้ใช้	ข้อมูลทริป	คะแนนย่อย
งบประมาณ	2,000 บาท	2,000 บาท	100%
กิจกรรม	3 แห่ง/วัน	3 กิจกรรม/วัน	100%
หมวดหมู่	กาแฟ	กาแฟ	100%
ช่วงเวลา	เช้าและกลางวัน	เช้าและกลางวัน	100%
Match Score		ค่าเฉลี่ย 4 ปัจจัย	100%

4.4.2 กรณีข้อมูลมีความแตกต่างกันบางส่วน

การทดสอบกรณีข้อมูลแตกต่างกันบางส่วนใช้สำหรับตรวจสอบว่า Match Score สามารถลดลงตามปัจจัยที่ไม่ตรงกันได้หรือไม่ ตัวอย่างเช่น ผู้ใช้มีงบประมาณและสไตล์กิจกรรมตรงกับทริป แต่เลือกหมวดหมู่ภูเขาและช่วงเวลากลางคืน ขณะที่ทริปอยู่ในหมวดหมู่กาแฟและจัดกิจกรรมในช่วงเช้ากับช่วงกลางวัน

ในกรณี Budget=100%, Activity=100%, Category=0% และ Time=0% ค่าเฉลี่ยเท่ากับ $(100+100+0+0) \div 4 = 50\%$

ผลดังกล่าวแสดงให้เห็นว่าระบบไม่ได้พิจารณาเฉพาะคุณลักษณะที่ตรงกัน แต่ลดคะแนนตามปัจจัยที่ไม่สอดคล้องกันด้วย

ปัจจัย	ข้อมูลผู้ใช้	ข้อมูลทริป	คะแนนย่อย
งบประมาณ	2,000 บาท	2,000 บาท	100%
กิจกรรม	3 แห่ง/วัน	3 กิจกรรม/วัน	100%
หมวดหมู่	ภูเขา	กาแฟ	0%
ช่วงเวลา	กลางคืน	เช้าและกลางวัน	0%

ปัจจัย	ข้อมูลผู้ใช้	ข้อมูลทริป	คะแนนย่อย
Match Score		$(100+100+0+0) \div 4$	50%

4.4.3 กรณีข้อมูลมีความแตกต่างกันสูง

กรณีข้อมูลแตกต่างกันสูงใช้ทดสอบเมื่อผู้ใช้งบต่ำกว่าทริป สนใจคนละหมวด ต้องการจำนวนกิจกรรมต่างกัน และเลือกช่วงเวลาไม่ตรงกัน คะแนนแต่ละด้านจะลดลงตามสูตรของด้านนั้น

ตัวอย่างผู้ใช้งบ 500 บาทและทริปใช้งบ 2,000 บาท จะได้ Budget Score = $500 \div 2,000 \times 100 = 25\%$

เมื่อ Budget=25%, Activity=0%, Category=0% และ Time=0% ค่าเฉลี่ยเท่ากับ 6.25 และปัดเป็น Match Score 6%

ปัจจัย	ข้อมูลผู้ใช้	ข้อมูลทริป	คะแนนย่อย
งบประมาณ	500 บาท	2,000 บาท	25%
กิจกรรม	1 แห่ง/วัน	10 กิจกรรม/วัน	0%
หมวดหมู่	ผจญภัย	คาเฟ่	0%
ช่วงเวลา	กลางคืน	เช้าและกลางวัน	0%
Match Score		$(25+0+0+0) \div 4$	6%

ผลลัพธ์แสดงว่าระบบลดคะแนนตามระดับความแตกต่างจริงของแต่ละปัจจัย และสามารถตรวจสอบที่มาของคะแนนได้จาก matchBreakdown

4.4.4 การเปรียบเทียบผลลัพธ์

รหัส	Test Case	Expected	ผลคำนวณ	Result
TC-01	ข้อมูลตรงกันทุกด้าน	100%	100%	ตัวอย่างตามสูตร
TC-02	Budget และ Activity ตรง อีก 2 ด้านไม่ตรง	50%	50%	ตัวอย่างตามสูตร
TC-03	งบ 500 เทียบกับทริป 2,000	Budget 25%	25%	ตัวอย่างตามสูตร
TC-04	กิจกรรม 3 เทียบกับ 6	Activity 67%	67%	ตัวอย่างตามสูตร

รหัส	Test Case	Expected	ผลคำนวณ	Result
TC-05	หมวดหมู่ไม่ตรงกัน	Category 0%	0%	ตัวอย่างตามสูตร
TC-06	ช่วงเวลาไม่ตรงกัน	Time 0%	0%	ตัวอย่างตามสูตร
TC-07	คะแนนย่อย 25, 0, 0, 0	Match Score 6%	6%	ตัวอย่างตามสูตร

กรณีกิจกรรม ผู้ใช้ 3 แห่งต่อวันและทริป 6 กิจกรรมต่อวันจะได้ Activity Score 67% หากอีก 3 ปีจัยได้ 100% คะแนนรวมจะเท่ากับ $(100+100+67+100) \div 4 = 91.75$ และปัดเป็น 92%

เมื่อข้อมูลตรงกันทุกด้านจะได้ 100% ส่วนการเปลี่ยนเพียงปัจจัยเดียวทำให้คะแนนลดลงตามค่าคะแนนย่อยของด้านนั้น เนื่องจากทุกปัจจัยที่มีข้อมูลถูกนำมาเฉลี่ยเท่ากัน จึงควรอ่าน MatchScore ร่วมกับ matchBreakdown เพราะคะแนนรวมเดียวกันอาจเกิดจากรูปแบบคะแนนย่อยที่แตกต่างกัน

สรุปผลการทดสอบ

ผลการคำนวณแสดงว่าระบบให้คะแนนสูงเมื่อข้อมูลสอดคล้องกันและลดคะแนนตามความแตกต่างของแต่ละด้าน คะแนนงบประมาณลดลงตามอัตราส่วนจริง ส่วนคะแนนรวมเป็นค่าเฉลี่ยของปัจจัยที่มีข้อมูล จึงสามารถใช้เรียงทริปตามความเหมาะสมและตรวจสอบที่มาจากรูป matchBreakdown ได้

ตัวอย่างในหัวข้อนี้นี้ใช้แสดงวิธีตรวจสอบผลคำนวณตามสูตรของระบบ ไม่ใช่ผลประเมินจากผู้ใช้งานจริง

4.5 ผลการประเมินความง่ายในการใช้งาน

ส่วนนี้อธิบายการใช้ System Usability Scale (SUS) เพื่อประเมินความง่ายในการใช้งานหลังผู้ทดสอบทำภารกิจหลักของแอปพลิเคชัน โดยไม่ระบุคะแนนก่อนเก็บคำตอบจริง

แบบประเมินประกอบด้วยคำถามจำนวน 10 ข้อ ใช้มาตราส่วนประมาณค่า 5 ระดับ ตั้งแต่ 1 หมายถึงไม่เห็นด้วยอย่างยิ่ง ถึง 5 หมายถึงเห็นด้วยอย่างยิ่ง จากนั้นนำคำตอบมาคำนวณเป็นคะแนน SUS ระหว่าง 0–100 คะแนน

4.5.1 วิธีคำนวณคะแนน System Usability Scale (SUS)

การประเมิน SUS ใช้คำถาม 10 ข้อตาม Brooke (1996) ข้อคี่นำคะแนนคำตอบลบ 1 ข้อคู่ นำ 5 ลบคะแนนคำตอบ รวมคะแนนที่ปรับแล้วและคูณ 2.5 เพื่อให้ได้คะแนนรายคนในช่วง 0–100

$$SUS = [\text{sum}(\text{คะแนนข้อคี่} - 1) + \text{sum}(5 - \text{คะแนนข้อคู่})] \times 2.5$$

คะแนนเฉลี่ยคำนวณจากสมการ

คะแนนเฉลี่ย **SUS** คำนวณจากผลรวมคะแนน **SUS** ของผู้ทดสอบแต่ละคนหารด้วยจำนวนผู้ทดสอบทั้งหมด

คะแนน SUS	การแปลผล
80.3–100	ดีเยี่ยม
68–80.2	ดีและยอมรับได้
51–67.9	ปานกลางและควรปรับปรุง
0–50.9	ต่ำและไม่ผ่านเกณฑ์

4.6 สรุปผลการทดสอบ

จากหลักฐานที่แสดงไว้ก่อนหน้านี้ ระบบ **Go With Us** สามารถรองรับกระบวนการหลัก ได้แก่ การสมัครสมาชิกและเข้าสู่ระบบ การบันทึกความต้องการของผู้ใช้ การสร้างและค้นหาทริป การคำนวณ **Match Score** การเข้าร่วมทริป และการสนทนาระหว่างสมาชิก

ระบบ **Smart Matching** คำนวณจากหมวดหมู่ งบประมาณ จำนวนกิจกรรมต่อวัน และช่วงเวลา โดยใช้ **Cosine Similarity** กับหมวดหมู่และช่วงเวลา ใช้อัตราส่วนกับงบประมาณ และใช้ระยะห่างบนช่วง **1–10** กับกิจกรรม ก่อนหาค่าเฉลี่ยของคะแนนย่อยที่มีข้อมูล

เมื่อข้อมูลผู้ใช้ตรงกับทริปทุกด้าน ระบบให้ **Match Score 100%** และเมื่อข้อมูลแตกต่างกัน คะแนนจะลดลงตามสูตรของแต่ละปัจจัย โดยสามารถตรวจสอบรายละเอียดได้จาก **matchBreakdown**

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

โครงการพัฒนาแอปพลิเคชันวางแผนการเดินทางอัจฉริยะ Go With Us มีวัตถุประสงค์เพื่อพัฒนาแอปพลิเคชันสำหรับช่วยค้นหาและจับคู่ผู้ใช้งานกับทริปการเดินทางที่มีความเหมาะสมกับความสนใจและรูปแบบการเดินทางของแต่ละบุคคล โดยประยุกต์ใช้อัลกอริทึม Cosine Similarity ในการวิเคราะห์ความคล้ายคลึงระหว่างข้อมูล User DNA ของผู้ใช้งานกับคุณลักษณะของทริป และนำเทคโนโลยีที่เกี่ยวข้องมาประยุกต์ใช้เพื่อสนับสนุนการวางแผนการเดินทาง

5.1 สรุปผลการดำเนินโครงการ

จากการดำเนินโครงการ สามารถพัฒนาแอปพลิเคชัน Go With Us ในรูปแบบ Native iOS Application โดยใช้ภาษา Swift และ SwiftUI เป็นเทคโนโลยีหลักในการพัฒนาแอปพลิเคชัน และใช้ Xcode เป็นเครื่องมือสำหรับการพัฒนาและทดสอบระบบ แอปพลิเคชันสามารถรองรับกระบวนการทำงานหลัก ได้แก่ การสมัครสมาชิกและเข้าสู่ระบบ การจัดเก็บข้อมูลความสนใจและรูปแบบการเดินทางของผู้ใช้งาน การค้นหาและสร้างทริป การเข้าร่วมทริป การสนทนาระหว่างสมาชิกภายในทริป และการแสดงข้อมูลที่เกี่ยวข้องกับการเดินทาง

ระบบนำข้อมูลผู้เข้ามาเปรียบเทียบกับทริป 4 ด้าน โดยใช้ Cosine Similarity สำหรับหมวดหมู่และช่วงเวลา ใช้อัตราส่วนสำหรับงบประมาณ และใช้ระยะทางบนช่วง 1-10 สำหรับกิจกรรม แล้วหาค่าเฉลี่ยของคะแนนย่อยที่มีข้อมูลเป็น Match Score

นอกจากนี้ ระบบมีการประยุกต์ใช้ MapKit เพื่อสนับสนุนการแสดงผลแผนที่และตำแหน่งสถานที่ภายในแอปพลิเคชัน และประยุกต์ใช้ Gemini API สำหรับสร้างคำแนะนำและวางแผนการเดินทาง (Itinerary Planning) ตามข้อมูลและเงื่อนไขของผู้ใช้งาน

ระบบ Backend พัฒนาด้วย Node.js และเชื่อมต่อกับฐานข้อมูล PostgreSQL บน Supabase สำหรับจัดเก็บและบริหารจัดการข้อมูลที่เกี่ยวข้องกับระบบ รวมถึงมีระบบ Backoffice Web สำหรับสนับสนุนการบริหารจัดการข้อมูลโดยผู้ดูแลระบบ

ผลที่แสดงในบทที่ 4 ครอบคลุมการพัฒนาฟังก์ชันและตัวอย่างการตรวจสอบสูตร ส่วนผลการยอมรับของผู้ใช้และคะแนน SUS ยังต้องอาศัยข้อมูลที่เก็บจากผู้ทดสอบจริง

5.2 อภิปรายผล

5.2.1 ผลการพัฒนาแอปพลิเคชัน

การพัฒนาแอปพลิเคชันในรูปแบบ Native iOS ด้วย Swift และ SwiftUI ทำให้สามารถออกแบบส่วนติดต่อผู้ใช้งานให้เหมาะสมกับอุปกรณ์ iOS และสามารถเชื่อมต่อกับความสามารถของระบบปฏิบัติการ

การและ Framework ของ Apple ได้โดยตรง เช่น การใช้งานแผนที่ผ่าน MapKit และการเข้าถึงความสามารถของอุปกรณ์ที่เกี่ยวข้องกับการทำงานของระบบ

การแยกระบบออกเป็นแอปพลิเคชัน iOS ระบบ Backend ระบบฐานข้อมูล และระบบ Backoffice Web ช่วยให้แต่ละส่วนมีหน้าที่ในการทำงานที่ชัดเจน และสามารถรับส่งข้อมูลระหว่างกันผ่าน API ได้อย่างเป็นระบบ

5.2.2 ผลการทำงานของระบบ Smart Matching

ระบบ Smart Matching คำนวณ Category, Budget, Activity Style และ Time of Day แยกกัน แล้วหาค่าเฉลี่ยของคะแนนที่มีข้อมูล ผลลัพธ์แสดงเป็น Match Score และ matchBreakdown

คะแนนช่วยจัดลำดับทริปตามข้อมูลที่ใช้ระบุ แต่ไม่รับประกันว่าผู้ร่วมเดินทางจะเข้ากันได้ ในสถานการณ์จริง คุณภาพของผลลัพธ์ขึ้นอยู่กับความครบถ้วนของข้อมูลผู้ใช้และรายละเอียดทริป

5.2.3 ผลการประยุกต์ใช้ Generative AI และ MapKit

การประยุกต์ใช้ Gemini API ช่วยสนับสนุนกระบวนการสร้างคำแนะนำและวางแผนการเดินทางตามข้อมูลที่ใช้ใช้งานกำหนด ทำให้ระบบสามารถนำเสนอข้อมูลการเดินทางที่มีความยืดหยุ่นมากกว่าการใช้ข้อมูลที่กำหนดไว้ล่วงหน้าเพียงอย่างเดียว

ในส่วนของ MapKit สามารถนำมาใช้สนับสนุนการแสดงผลแผนที่ ตำแหน่ง และสถานที่ที่เกี่ยวข้องกับการเดินทางภายในแอปพลิเคชัน ทำให้ผู้ใช้งานสามารถพิจารณาข้อมูลด้านตำแหน่งร่วมกับข้อมูลของทริปได้สะดวกยิ่งขึ้น

5.2.4 ผลการประเมินและทดสอบระบบ

การตรวจสอบในเอกสารนี้แสดงตัวอย่างการคำนวณ Smart Matching และการทำงานของกระบวนการหลักตามหลักฐานที่มีอยู่ ยังไม่ควรสรุปผล User Acceptance Testing โดยไม่มีข้อมูลผู้ทดสอบจริง

5.3 ปัญหาและข้อจำกัดของระบบ

จากการพัฒนาและทดสอบระบบ พบข้อจำกัดบางประการที่ควรนำมาพิจารณาในการพัฒนาระบบต่อไปได้แก่

1. แอปพลิเคชันหลักในปัจจุบันพัฒนาสำหรับระบบปฏิบัติการ iOS จึงยังไม่รองรับผู้ใช้งานบนระบบปฏิบัติการ Android
2. ประสิทธิภาพของระบบ Smart Matching ขึ้นอยู่กับความครบถ้วนและความถูกต้องของข้อมูล User DNA ที่ผู้ใช้งานระบุ หากข้อมูลมีจำนวนน้อยหรือไม่สอดคล้องกับความต้องการจริง อาจส่งผลกระทบต่อความเหมาะสมที่ระบบคำนวณได้

3. การสร้างแผนการเดินทางด้วย Generative AI อาจให้ผลลัพธ์แตกต่างกันตามข้อมูลและคำสั่งที่ส่งให้โมเดล จึงควรมีการตรวจสอบความเหมาะสมของข้อมูลก่อนนำไปใช้ในการเดินทางจริง
4. การทำงานของฟังก์ชันที่เกี่ยวข้องกับบริการออนไลน์ เช่น Backend, ฐานข้อมูล, Generative AI และข้อมูลแผนที่ จำเป็นต้องอาศัยการเชื่อมต่อเครือข่ายอินเทอร์เน็ต
5. รายงานฉบับนี้ยังไม่มีผลประเมินจากผู้ใช้งานจริงและจำนวนผู้ทดสอบที่ตรวจสอบย้อนกลับได้ จึงยังไม่สามารถสรุปการยอมรับระบบหรืออ้างว่าผลทดสอบสะท้อนพฤติกรรมของผู้ใช้งานมากในสภาพแวดล้อมจริง

5.4 ข้อเสนอแนะและแนวทางการพัฒนาต่อ

สำหรับการพัฒนาระบบ Go With Us ในอนาคต สามารถขยายขีดความสามารถของระบบได้หลายด้าน เช่น การพัฒนาแอปพลิเคชันสำหรับระบบปฏิบัติการ Android เพื่อเพิ่มจำนวนผู้ใช้งาน การปรับปรุงระบบ Smart Matching โดยนำข้อมูลจากพฤติกรรมการใช้งานจริงมาประกอบการคำนวณ การพัฒนาระบบแนะนำให้สามารถเรียนรู้ความสนใจที่เปลี่ยนแปลงของผู้ใช้งาน และการปรับปรุงกระบวนการสร้างแผนการเดินทางด้วย Generative AI ให้สามารถตรวจสอบข้อจำกัดของสถานที่และข้อมูลการเดินทางได้มากยิ่งขึ้น

นอกจากนี้ สามารถพัฒนาระบบด้านความปลอดภัยและความน่าเชื่อถือของชุมชนเพิ่มเติม เช่น การปรับปรุงกระบวนการยืนยันตัวตน ระบบรายงานผู้ใช้งาน ระบบจัดการเหตุการณ์ผิดปกติ และกลไกการประเมินความน่าเชื่อถือของสมาชิก เพื่อเพิ่มความมั่นใจในการค้นหาและเดินทางร่วมกับผู้ใช้งานรายอื่น

ในด้านระบบ Smart Matching สามารถศึกษาวิธีการหรืออัลกอริทึมเพิ่มเติมเพื่อเปรียบเทียบกับ Cosine Similarity และประเมินว่าวิธีการใดเหมาะสมกับข้อมูลและพฤติกรรมของผู้ใช้งานมากที่สุดเมื่อระบบมีจำนวนผู้ใช้งานและข้อมูลเพิ่มขึ้นในอนาคต

อ้างอิง

- [1] กิตติพงษ์ กลมเกลี้ยง, การวิเคราะห์ข้อมูลด้วยเทคนิคการเรียนรู้ของเครื่อง (Machine Learning Techniques). กรุงเทพฯ: สำนักพิมพ์มหาวิทยาลัยเกษตรศาสตร์, 2565.
- [2] นัทธพล เหลืองอร่าม, ภูมินันท์ รักยิ้ม, และ สุนันทา ศรีม่วง, "การพัฒนาระบบแนะนำสถานที่ท่องเที่ยวรายบุคคล," วารสารวิทยาสารสนเทศและเทคโนโลยี, ปีที่ 4, ฉบับที่ 1, หน้า 33–43, 2566. [ออนไลน์]. เข้าถึงได้จาก: https://so09.tci-thaijo.org/index.php/jait_ssru/article/view/2344
- [3] ประสงค์ ปรานิตพลกรัง และคณะ, ระบบสารสนเทศเพื่อการจัดการ (Management Information Systems). กรุงเทพฯ: ซีเอ็ดดูเคชั่น, 2562.
- [4] ศูนย์พัฒนาวิชาการด้านตลาดการท่องเที่ยว (TAT Academy), "ทฤษฎีพฤติกรรมผู้บริโภคด้านการท่องเที่ยว," TAT Review Magazine, ปีที่ 48, ฉบับที่ 2, หน้า 12–25, 2565–2566.
- [5] สุมาลี แสงอุไรวุฒิ, การวิเคราะห์การถดถอยและสหสัมพันธ์สำหรับการวิจัยเชิงวิชาการ. กรุงเทพฯ: โรงพิมพ์แห่งจุฬาลงกรณ์มหาวิทยาลัย, 2564.
- [6] A. Agresti, *An Introduction to Categorical Data Analysis*, 3rd ed. Hoboken, NJ: Wiley-Interscience, 2018.
- [7] Apple Inc., "SwiftUI Documentation," 2024. [Online]. Available: <https://developer.apple.com/documentation/swiftui/>
- [8] C. Bianchi, "Solo holiday travellers: Motivations and experiences of academics and managers," *International Journal of Tourism Research*, vol. 18, no. 2, pp. 197–206, 2016.
- [9] S. Chaudhuri and U. Dayal, "An overview of data warehousing and OLAP technology," *ACM SIGMOD Record*, vol. 26, no. 1, pp. 65–74, 1997.
- [10] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [11] A. K. Dey and G. D. Abowd, "Towards a better understanding of context and context-awareness," *GVU Technical Report GIT-GVU-99-22*, Georgia Institute of Technology, 1999.
- [12] Express.js, "Express - Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>

- [13] Google, "Gemini API Documentation," 2024. [Online]. Available: https://ai.google.dev/docs/gemini_api
- [14] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.
- [15] D. Jannach, M. Zanker, A. Felfernig, and G. Friedrich, Recommender Systems: An Introduction. Cambridge, UK: Cambridge University Press, 2010.
- [16] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.
- [17] X. Li and Y. Wang, "A Survey of Recommendation Systems: Techniques, Applications and Challenges," *IEEE Access*, vol. 8, pp. 45–60, 2020.
- [18] E. D. Liddy, "Natural Language Processing," in *Encyclopedia of Information Science and Technology*, Hershey, PA: Idea Group Reference, 2001, pp. 2126–2136.
- [19] J. B. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.
- [20] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [21] Node.js, "Node.js Documentation," 2024. [Online]. Available: <https://nodejs.org/en/docs/>
- [22] C. M. Paris, "Social Media and the Solo Traveler: A Case Study of Flashpackers," *Information and Communication Technologies in Tourism*, pp. 127–138, 2012.
- [23] R. Paulavičius, L. Stripinis, S. Sutavičiūtė, D. Kočegarov, and E. Filatovas, "A novel greedy genetic algorithm-based personalized travel recommendation system," *Expert Systems with Applications*, vol. 230, art. 120580, 2023. doi: 10.1016/j.eswa.2023.120580.

- [24] G. Peersman, "Introduction to API Security," *Journal of Cloud Computing*, vol. 3, no. 1, pp. 10–19, 2014.
- [25] Prisma, "Prisma ORM Documentation," 2024. [Online]. Available: <https://www.prisma.io/docs/>
- [26] React Community, "React: The library for web and native user interface," 2024. [Online]. Available: <https://react.dev/>
- [27] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2021.
- [28] A. Singhal, "Modern Information Retrieval: A Brief Overview," *IEEE Data Engineering Bulletin*, vol. 24, no. 4, pp. 35–43, 2001.
- [29] W. Stallings, *Network Security Essentials: Applications and Standards*, 7th ed. Hoboken, NJ: Pearson, 2022.
- [30] Supabase, "Supabase: Open Source Firebase Alternative," 2024. [Online]. Available: <https://supabase.com/docs/>
- [31] Tailwind CSS, "Tailwind CSS Documentation," 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [32] P. N. Tan, M. Steinbach, and V. Kumar, *Introduction to Data Mining*, 2nd ed. Hoboken, NJ: Pearson, 2018.
- [33] S. Tiwari, *Professional NoSQL*. Indianapolis, IN: Wrox Press, 2020.
- [34] Vite.js, "Vite: Next Generation Frontend Tooling," 2024. [Online]. Available: <https://vitejs.dev/guide>
- [35] World Tourism Organization (UNWTO), "Global Tourism Barometer and Statistical Annex," 2023. [Online]. Available: <https://www.unwto.org/>
- [36] D. Zeng, H. Chen, R. Lusch, and S. H. Li, "Social media analytics and intelligence," *IEEE Intelligent Systems*, vol. 25, no. 6, pp. 13–16, 2010.

- [37] J. Brooke, "SUS: A quick and dirty usability scale," in Usability Evaluation in Industry, Taylor & Francis, 1996, pp. 189–194.
- [38] G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," Communications of the ACM, vol. 18, no. 11, pp. 613–620, 1975. doi:10.1145/361219.361220.